

定时,首先给定一个较大的积分时间常数,如果不满足要求,减小比例系数 K_p ,通常减小到初始值的 80%,然后逐渐减小 T_i ,直到消除了系统的静态偏差,同时能够保证系统的动态性能。

③在加入积分环节消除了系统的静态偏差后,如果动态性能不能够满足要求,可以加入微分环节,构成比例积分微分控制器。通常先将微分时间常数 T_d 初始值设成零,然后逐步增大,同时相应地调整 K_p 与 T_i ,直到获得满意的控制效果。

采用试凑法整定参数,需要对 PID 控制算法的三个环节的作用有深入的了解,结合工程实践的经验,工程师总结出下面的整定口诀:

参数整定找最佳,从小到大顺序查。
先是比例后积分,最后再把微分加。
曲线振荡很频繁,比例度盘要放大。
曲线漂浮绕大弯,比例度盘往小扳。
曲线偏离回复慢,积分时间往下降。
曲线波动周期长,积分时间再加长。
曲线振荡频率快,先把微分降下来。
动差大来波动慢,微分时间应加长。
理想曲线两个波,前高后低四比一。
一看二调多分析,调节质量不会低。

在工程实际中,由于环境、被控对象以及控制要求等各方面因素,标准的 PID 控制算法有时无法满足要求,这就需要对控制算法进行改进。于是产生了不完全微分 PID 控制、微分先行 PID 控制算法、积分分离算法等。

4.4 本章小结

本章主要介绍了控制系统的基本概念,控制系统可以有很多分类方式,按照系统是否有反馈可以分为开环控制系统和闭环控制系统,要熟悉开环和闭环的分类原则。控制系统还可以按照控制目标分为恒值控制系统、程序控制系统和随动控制系统;按照系统传递信号的特点可以分为连续系统和离散系统。

一个典型的控制系统是由控制器、驱动器、执行器以及测量系统构成,在设计控制系统时,首先要根据系统的应用背景、性能要求、成本要求等选择合适的方案,系统构成的硬件方面,本章重点介绍了微型计算机、可编程控制器、嵌入式系统等控制器的特点,简要介绍了几种常用的电气、气动、液压驱动与执行元件,比较不同形式驱动的优缺点,并进行了应用举例,对选取合适的驱动和执行方式有一定的帮助。执行机构能够实现运动与力的转换与传递,执行机构种类很多,本章重点介绍连杆机构、齿轮机构、链传动、带传动、螺旋机构、凸轮机构以及棘轮机构等常用机构,使读者了解传动设计与传动特点。

控制器是控制系统的大脑,而控制理论和控制方法决定了控制效果。本章简要介绍了控制理论的发展历程和特点,重点介绍了比例积分微分控制算法,以及在实际控制中的具体实施方法。

由于篇幅有限,控制系统各个组成部分只能作简要介绍,如果想获得更多系统设计以及详细应用,可以参考自动控制以及机械原理与设计相关书籍。

思考题

1. 列举身边的一些控制系统,并判断系统为开环或者闭环。
2. 画出一些工业控制系统的框图,并分析各个环节输入与输出。
3. 比较几种数字控制器的优缺点,并举例。
4. 查阅气动传动资料,归纳特点,并列举一些应用。
5. 比较齿轮传动、带传动与链传动优缺点。
6. 列举一些齿轮传动与螺旋机构的应用。
7. 查阅资料,简述模糊控制的控制思想。

第5章 计算机控制系统

工业现场有两大类量——开关量和模拟量,在对这些量的控制过程中,计算机与工业现场信号无法进行直接传递,那么,工业现场的开关量是如何传送到计算机里呢?计算机所发出的开关量控制信号又是如何送达到工业现场呢?工业现场的模拟量是如何转换为计算机所能接受的数字量传送到计算机里呢?计算机计算的结果为数字量,它又是如何控制现场模拟量执行器去动作呢?这些将是本章解决的主要问题。

5.1 概述

现代控制理论的发展给自动控制系统增添了理论工具,而计算机技术的发展为新型控制规律的实现、构造高性能的控制系统提供了物质基础,两者的结合极大地推动了自动控制技术的发展。

计算机控制系统是建立在自动控制技术和计算机技术的基础上的。计算机控制系统利用计算机的硬件和软件代替自动控制系统的控制器,综合了自动控制理论、计算机技术、检测技术、通信与网络技术等,并将这些技术集成起来用于工业生产过程,对生产过程实现检测、控制、优化、调度、管理和决策,以达到提高质量与产量,确保安全生产等目的。

计算机控制系统具有丰富的指令系统和很强的逻辑判断功能。用计算机代替控制器,只要选择合适的控制算法,依据偏差计算出相应的控制量,就可以由软件编程来实现控制功能,能够实现模拟电路不能实现的复杂控制规律,因此它的适应性和灵活性很高。

对于工业生产过程的自动控制方法常分为以流水作业为主的顺序逻辑控制和以生产过程控制为主的物理量的控制,前者常常属于开关量开环控制系统,后者则常常属于模拟量闭环控制系统。在计算机控制系统中,往往大多数是开关量和模拟量同时存在的混合系统。

要对工业生产的过程及装置进行控制,就要检测被控对象当前的状态信息,并将此信息传递给计算机;计算机经过计算、处理后,将控制量以数字量的形式输出,并转换为适合于对生产过程进行控制的量。而计算机与工业现场信号无法进行直接传递,这就需要在二者之间设置进行信息传递和交换的装置,即过程通道。

在自动控制系统的框图中,若用计算机做比较器与控制器,再加上过程通道,就构成了计算机控制系统,其基本框图如图 5-1 所示。

从本质上来看,计算机控制系统的控制过程可以归结为以下三个步骤:

- ①实时数据采集。对被控参数的状态信息进行检测并输入。
 - ②实时决策。对采集到的被控参数的状态信息进行分析,并按照已经设定好的控制规律计算出下一步的控制量。
 - ③实时控制。根据决策,计算机输出控制量,实时控制执行机构动作。
- 计算机控制系统按照上述步骤不断重复来控制整个系统按照一定的动态品质指标进行工

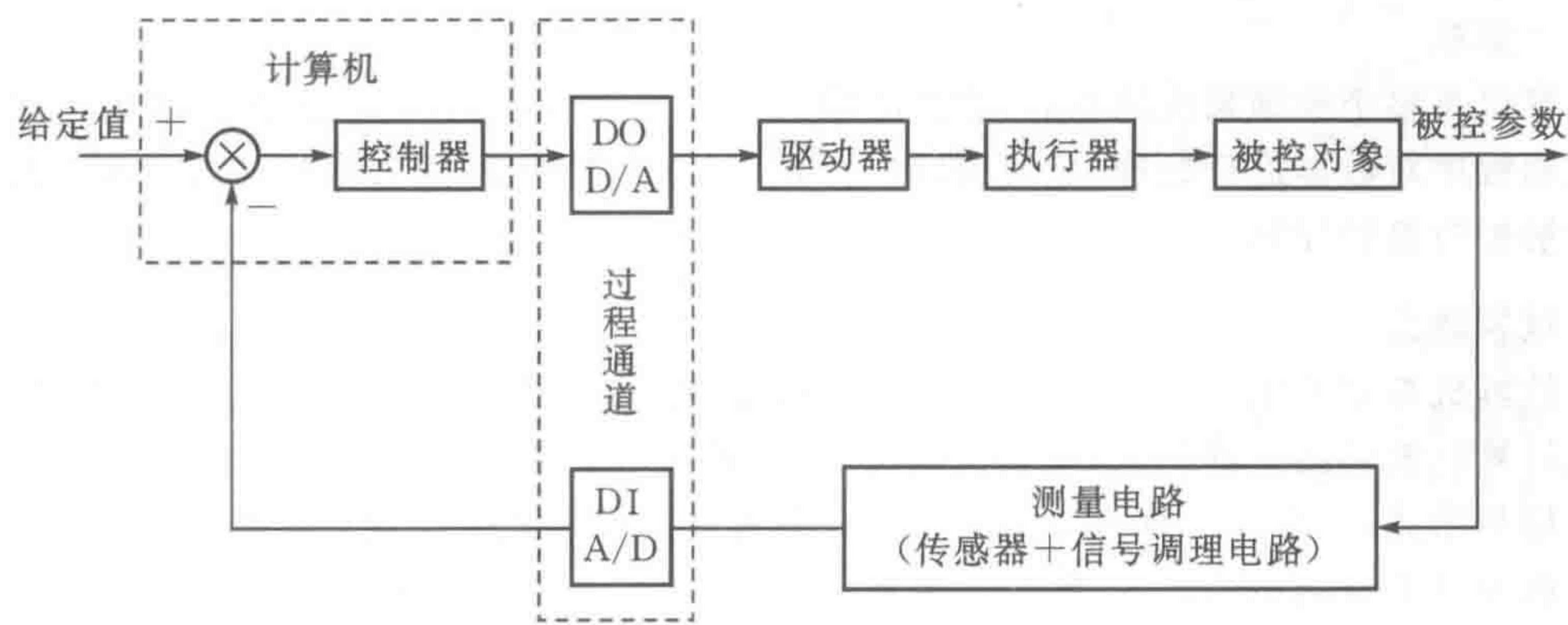


图 5-1 计算机控制系统的基本框图

作,并且对被控参数本身出现的异常状态进行实时监控和及时处理。

5.2 计算机控制系统的组成

计算机控制系统包括硬件、软件两大部分,硬件由主机、接口电路、外部设备组成,是计算机控制系统的基础;软件是安装在主机中的程序,能够完成对其接口和外部设备的控制,完成对信息的处理,它包含维持计算机主机工作的系统软件 and 为完成控制而进行信息处理的应用软件两大部分,软件是计算机控制系统的关键。

5.2.1 计算机控制系统的硬件

计算机控制系统的硬件一般由人机交互设备、计算机、过程通道、检测装置、执行机构、被控对象(生产装置或生产过程)等组成。计算机控制系统的硬件组成框图如图 5-2 所示。

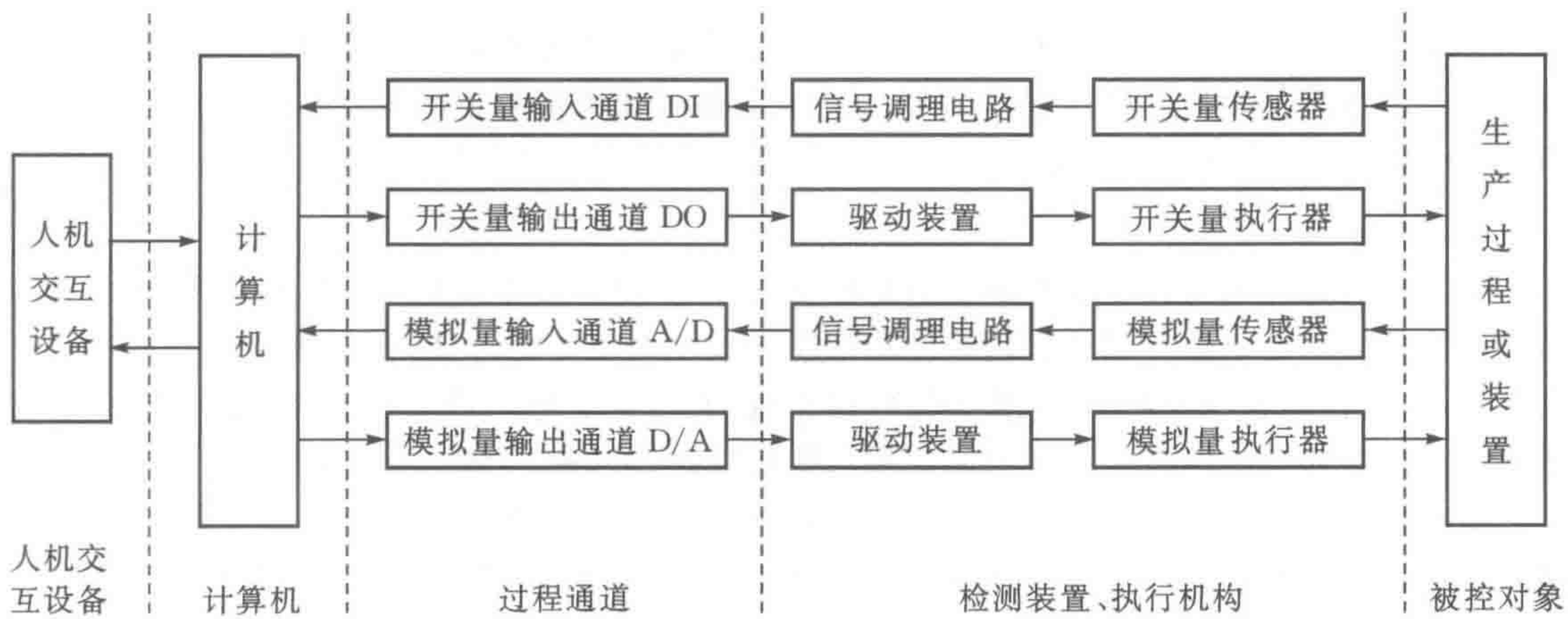


图 5-2 计算机控制系统的硬件组成框图

1. 计算机

计算机是整个控制系统的核心,它接收输入通道发送的数据,实现被测参数的巡回检测,通过控制程序对数据进行处理、比较、判断、计算后,得出控制量,通过输出通道发出控制指令,实现对被控对象的控制。

2. 过程通道

在计算机控制系统中,为了实现对生产过程或装置的控制,要将生产现场的各种被测参数转换成计算机能够接受的形式,计算机经过计算、处理后的结果还须变换成适合于对生产进行控制的信号形式,这个在计算机和生产过程或装置之间传递和变换信息的装置称为过程通道。

过程通道配合相应的输入、输出控制程序,使计算机和被控对象间能进行信息交换,从而实现对被控对象的控制。根据信号的方向和形式,过程通道又可分为:

(1) 开关量输入通道

开关量输入通道是把过程和被控对象的开关量或通过传感器已转换的数字量输入计算机。

(2) 开关量输出通道

开关量输出通道是将计算机运算、决策之后的数字信号输出给被控对象或外部设备。

(3) 模拟量输入通道

模拟量输入通道是将经由传感器得到的工业对象的生产过程参数变换成数字量传送给计算机。

(4) 模拟量输出通道

模拟量输出通道是将计算机输出的数字信号变换为控制执行机构的模拟信号,以实现对生产过程的控制。

在本章的训练中,过程通道由 PCI-1710 采集卡和 ADAM-3968 端子板组成。PCI-1710 采集卡和 ADAM-3968 端子板将在 5.3 节中进行介绍。

3. 检测装置

工业过程的参数一般是非电量,必须经过传感器变换为等效的电信号,再经过适当的信号调理才能进行信号采集。例如用 AD590 温度传感器将温度信号转变为电流信号,然后经过电流/电压转换电路转变为电压信号,再经模拟量输入通道送入计算机。

4. 执行机构

执行机构的作用是接受计算机发出的控制信号,并把它转换成执行机构的动作,使被控对象按预先规定的要求进行调整。执行机构往往与被控对象连为一体,控制被控参数的变化过程。例如,在液位控制系统中,计算机输出控制量,通过控制电动调节阀的开度控制进入容器的液体流量进而来控制液位的变化。常用的执行机构有电动、气动、液压等方式。

有些执行机构需要较大的驱动功率,即需向执行机构提供大电流或高电压驱动信号,以驱动其动作;另一方面,由于各种执行机构的动作原理不尽相同,有的用电动,有的用气动或液压,如何使计算机输出的信号与之匹配,也是执行机构必须解决的重要问题。为了实现与执行机构的功率配合,一般都要在计算机输出板卡与执行机构之间配置驱动装置。

5. 人机交互设备

人机交互设备是实现计算机与人进行信息交互的设备。按其功能可分为输入设备和输出

设备。输入设备用来输入程序、数据或操作命令,如键盘、鼠标等;输出设备用来向操作人员提供各种反映生产过程工况的信息和数据,以便操作人员及时了解控制过程,如打印机、记录仪、图形显示器(CRT)等。外存储器等主要用来存储程序和数据,兼有输入与输出功能。

5.2.2 计算机控制系统的软件

计算机控制系统的硬件是完成控制任务的设备基础,软件是实现控制任务的关键,它关系到计算机运行和控制效果以及硬件功能的发挥。软件是指计算机控制系统中具有各种功能的计算机程序的总和,如完成操作、监控、管理、控制、计算和自诊断等功能的程序,整个系统在软件指挥下协调工作。软件由系统软件和应用软件组成。

1. 系统软件

系统软件一般随硬件一起由计算机的制造厂商提供,是用来管理计算机本身的资源、方便用户使用计算机的软件。作为开发应用软件的工具,系统软件提供了计算机运行和管理的基本环境。常用的有操作系统、开发系统等,它们一般不需用户自行设计编程,只需掌握使用方法或根据实际需要加以适当设置即可。

2. 应用软件

应用软件是用户根据要解决的实际问题而编写的各种程序,例如各种数据采集、数据处理、控制算法等。应用软件通常采用模块化结构进行设计,一个模块就是一个子函数,通过子函数的调用实现控制功能。应用软件的优劣,将给控制系统的功能、精度和效率带来很大的影响,它的设计是本章主要介绍的内容之一。

5.3 采集卡 PCI-1710 简介

5.3.1 简介

PCI-1710 是一款功能强大的低成本多功能 PCI 总线数据采集卡,如图 5-3 所示,其中包含五种最常用的测量和控制功能,即 12 位 A/D 转换、D/A 转换、数字量输入、数字量输出,以及计数器/定时器功能。

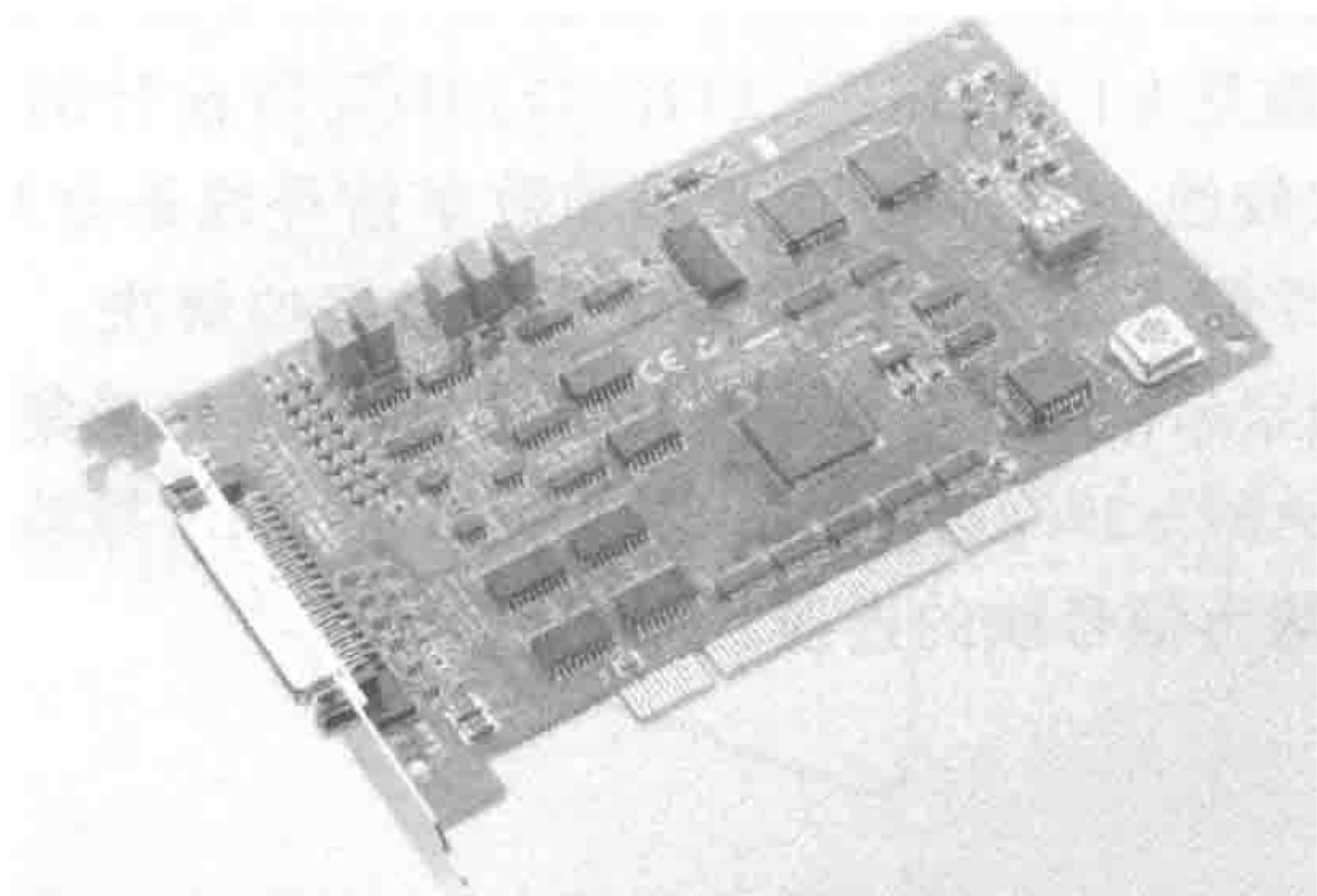


图 5-3 PCI-1710 数据采集卡

PCI-1710 支持即插即用。在安装插卡时,用户不需要设置任何跳线和 DIP 拨码开关。实际上,所有与总线相关的配置,例如基地址、中断,均由即插即用功能完成。

5.3.2 特性

PCT-1710 的特性总结如下:

- ①16 路单端或 8 路差分模拟量输入或组合方式输入。
- ②12 位 A/D 转换器,采样速率可达 100 kHz。
- ③可编程设置每个通道的增益。
- ④板载 4 K 采样 FIFO 缓存器。
- ⑤2 路 12 位模拟量输出。
- ⑥16 路开关量输入及 16 路开关量输出。
- ⑦可编程计数器/定时器。

5.3.3 基于采集卡的控制系统的组成

在基于采集卡的控制系统中,采集卡和端子板构成过程通道,如图 5-4 所示。

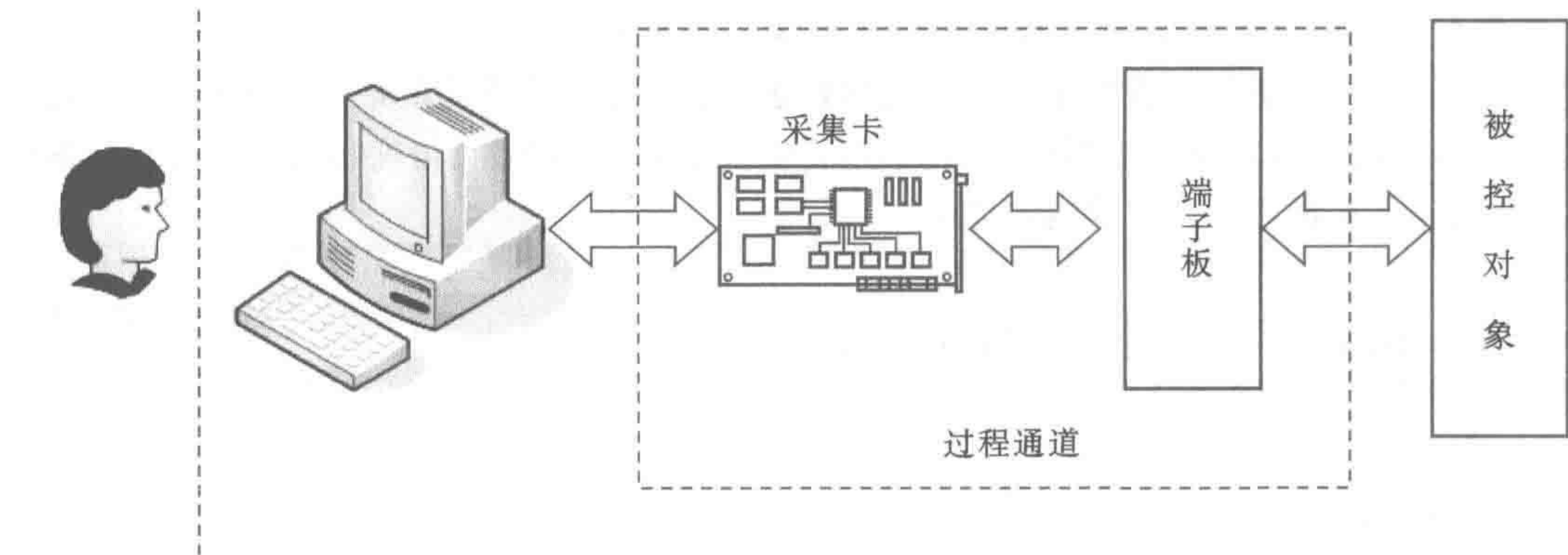


图 5-4 由采集卡和端子板构成过程通道

用数据采集卡构成完整的控制系统还需要接线端子板、通信电缆。

PCL-10168 屏蔽电缆是专门为 PCI-1710/1710HG 所设计的,它用来降低模拟信号的输入噪声。该电缆采用双绞线,并且模拟信号线和数字信号线是分开屏蔽的。这样能使信号间的交叉干扰降到最小,并使 EMI/EMC 问题得到了最终的解决。PCL-10168 屏蔽电缆如图 5-5 所示,是两端针型接口的 68 芯 SCSI-II 电缆,用于连接采集卡与接线端子板。

接线端子板采用 ADAM-3968 型,如图 5-6 所示,是 DIN 导轨安装的 68 芯 SCSI-II 接线端子板,用于各种输入输出信号线的连接。

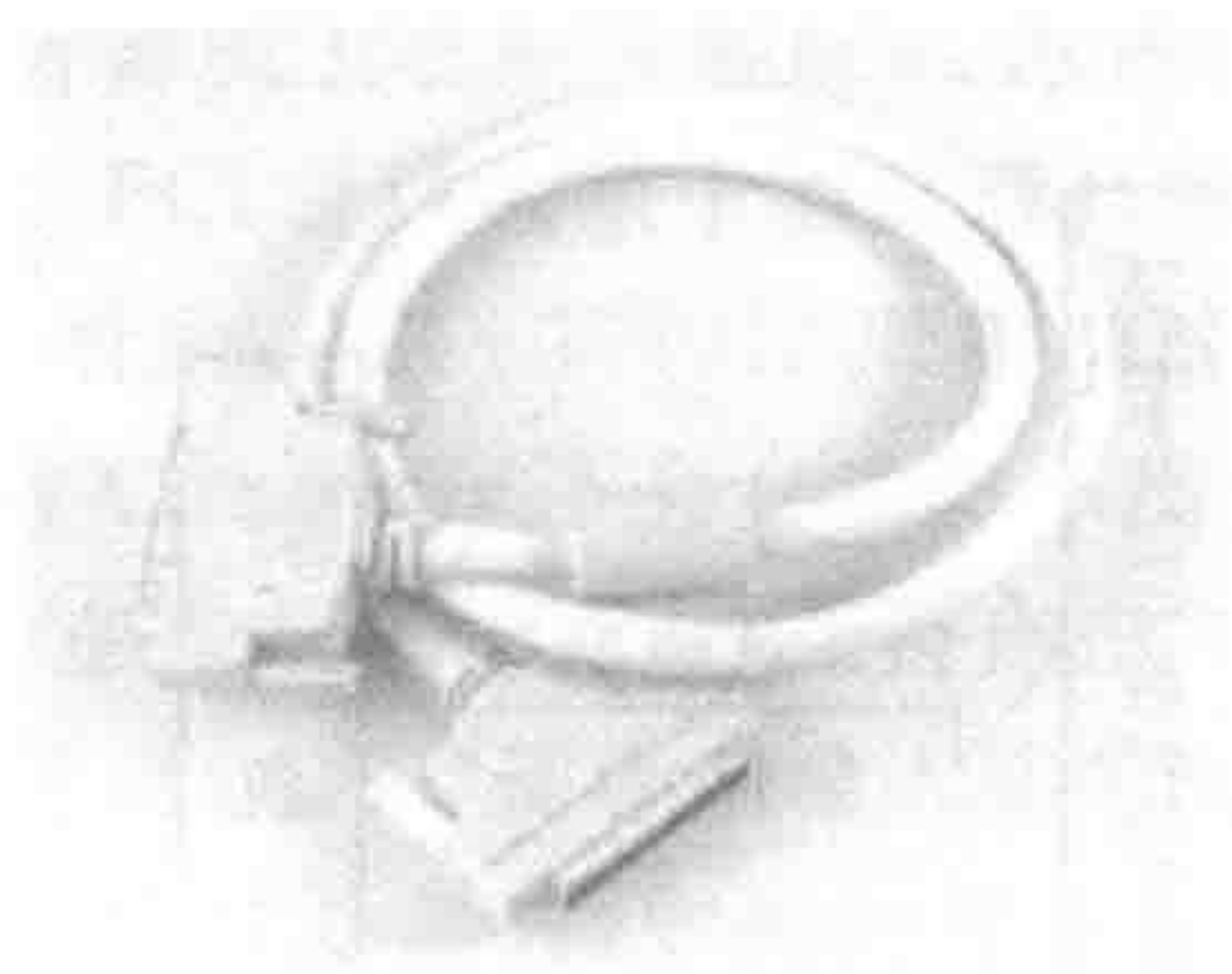


图 5-5 PCL-10168 电缆

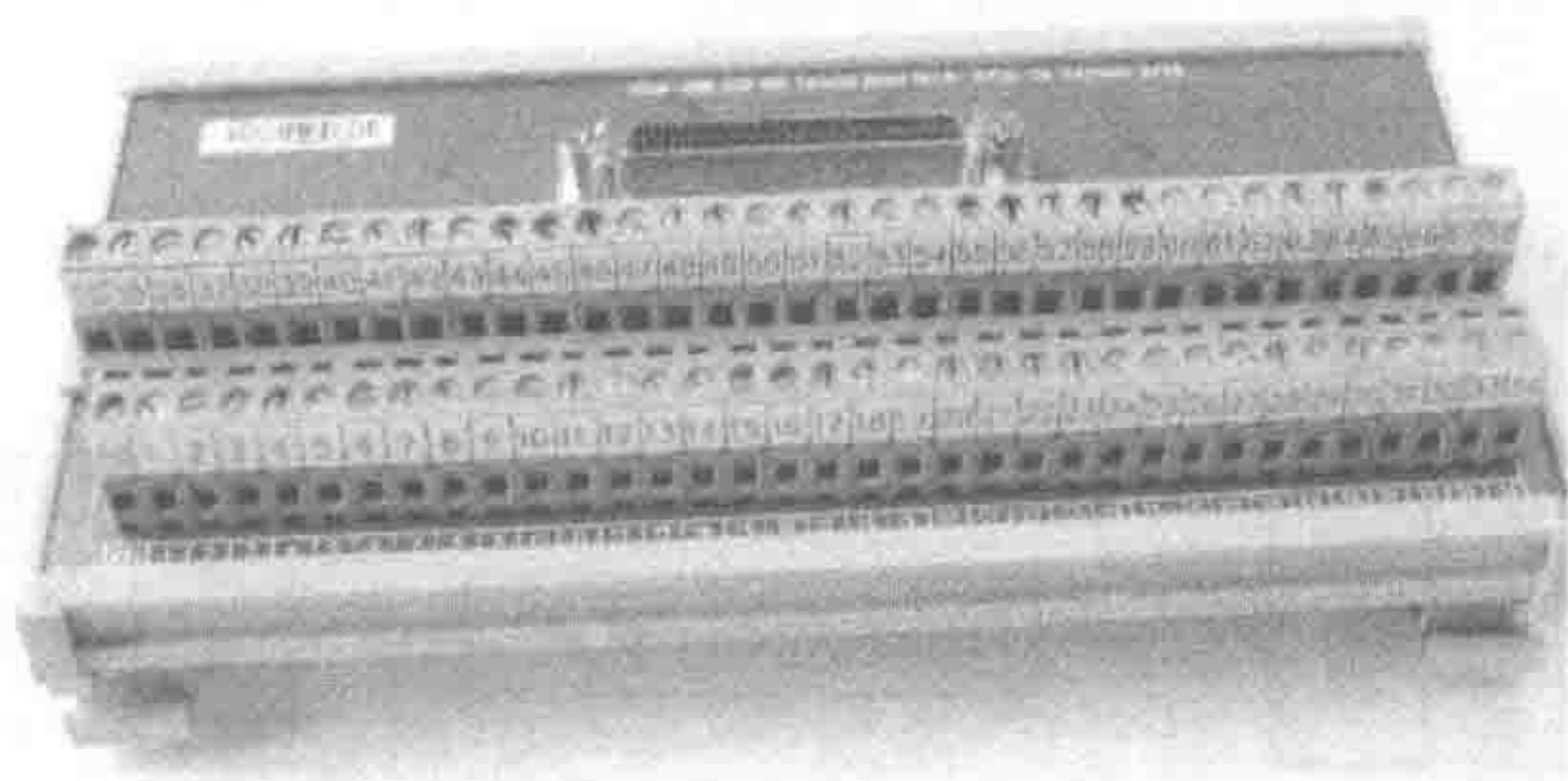


图 5-6 ADAM-3968 接线端子板

用 PCL-10168 电缆将 PCI-1710 采集卡与 ADAM-3968 端子板连接,这样 PCL-1710 的 68 个针脚和ADAM-3968的 68 个接线端子一一对应,对应关系如图 5-7 所示,使用时将输入信号连接到端子板相应接线柱上即可。

AI0	68	34	AI1
AI2	67	33	AI3
AI4	66	32	AI5
AI6	65	31	AI7
AI8	64	30	AI9
AI10	63	29	AI11
AI12	62	28	AI13
AI14	61	27	AI15
AIGND	60	26	AIGND
AO0_REF*	59	25	AO1_REF*
AO0_OUT*	58	24	AO1_OUT*
AOGND*	57	23	AOGND*
DI0	56	22	DI1
DI2	55	21	DI3
DI4	54	20	DI5
DI6	53	19	DI7
DI8	52	18	DI9
DI10	51	17	DI11
DI12	50	16	DI13
DI14	49	15	DI15
DGND	48	14	DGND
DO0	47	13	DO1
DO2	46	12	DO3
DO4	45	11	DO5
DO6	44	10	DO7
DO8	43	9	DO9
DO10	42	8	DO11
DO12	41	7	DO13
DO14	40	6	DO15
DGND	39	5	DGND
CNTO_CLK	38	4	PACER_OUT
CNTO_OUT	37	3	TRG_GATE
CNTO_GATE	36	2	EXT_TRG
+12 V	35	1	+ 5V

图 5-7 PCI-1710 卡 68 针 I/O 接口的针脚定义

用 PCI-1710 采集卡构成的控制系统框图如图 5-8 所示。

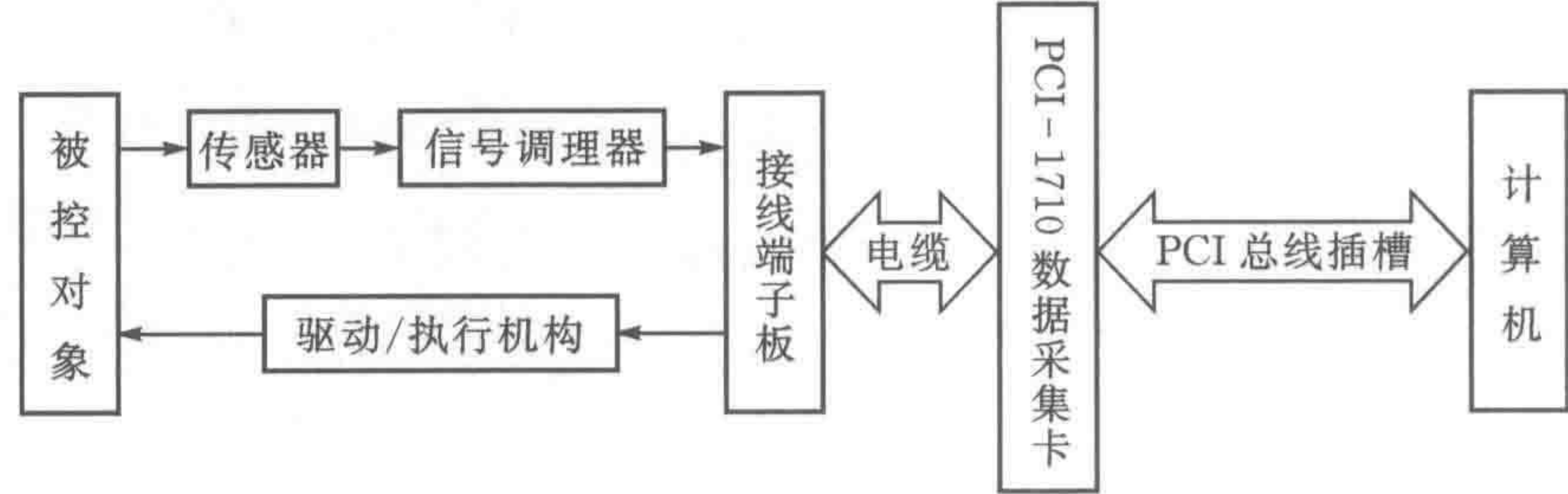


图 5-8 基于 PCI-1710 采集卡的控制系统的组成

5.3.4 采集卡 PCI-1710 Demo 程序应用介绍

利用采集卡附带的测试程序对采集卡的各项功能进行测试。

运行设备测试程序：在研华设备管理程序 Advantech Device Manager 对话框中点击“Test”按钮，出现“Advantech Device Test”对话框，通过不同选项可以对采集卡的“Analog Input”“Analog Output”“Digital Input”“Digital Output”“Counter”等功能进行测试。

1. 设置

(1)从开始菜单/程序/Advantech Automation/ Device Manager,打开 Advantech Device Manager,如图 5-9 所示。

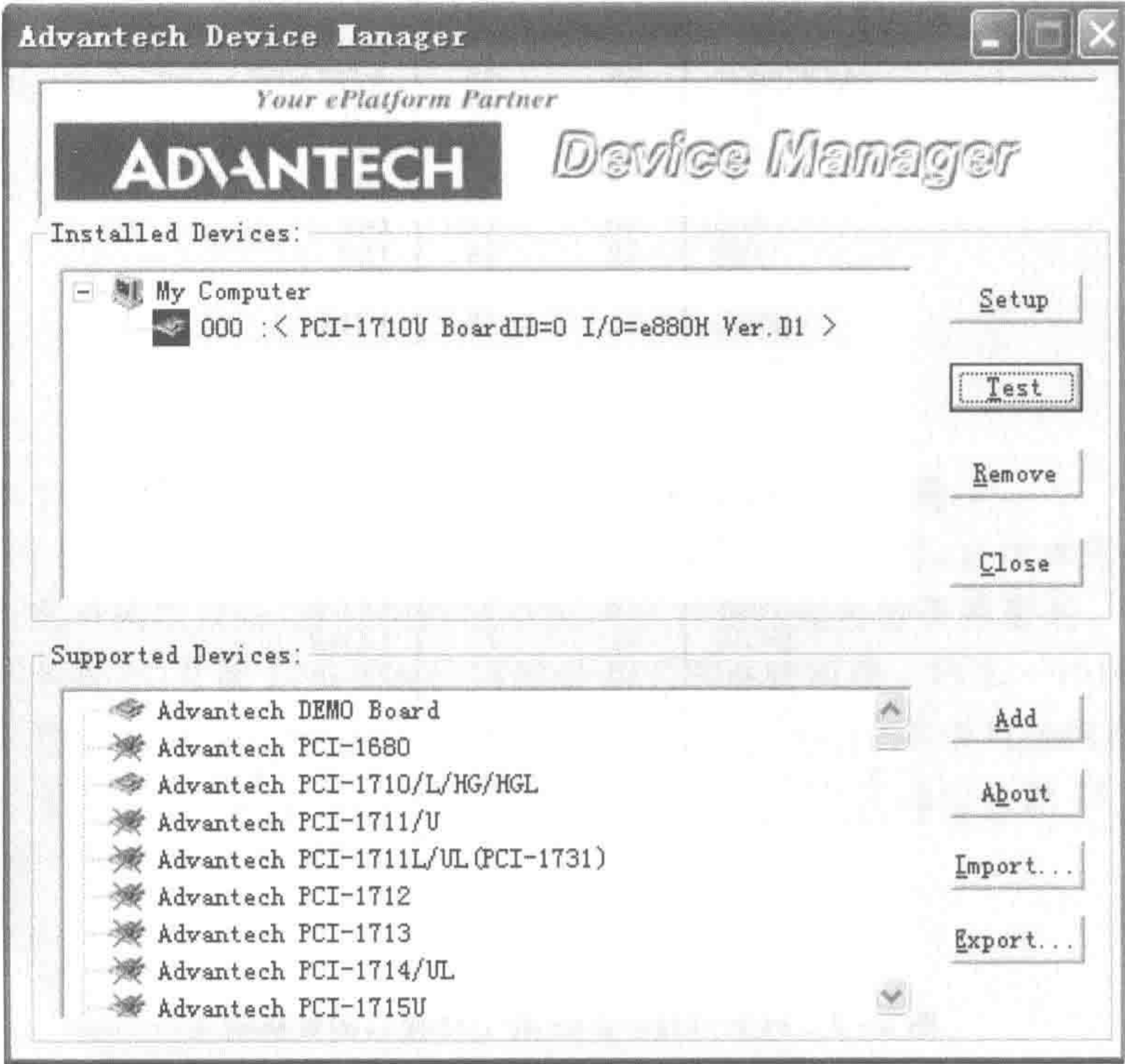


图 5-9 Demo 程序界面

当计算机上已经安装好某个产品的驱动程序后,它前面将没有叉号,说明驱动程序已经安装成功。PCI 总线的板卡插好后计算机操作系统会自动识别,并显示分配给板卡的基地址,如图 5-9 所示,PCI-1710U 采集卡的基地址为 E880H。Device Manager 在 Installed Devices 栏中 My Computer 下会自动显示出所插入的设备,这一点和 ISA 总线的板卡不同。

(2)在图 5-9 所示界面中点击“Setup”弹出设置界面,如图 5-10 所示。用户可设置模拟量输入通道(A/D)是单端输入或是差分输入;还可设置模拟量输出通道(D/A)的参考电压是使用内部或者外部的,如果使用内部参考电压,可选择“0-5 V”或者“0-10 V”。设置完成后点击“OK”即可。

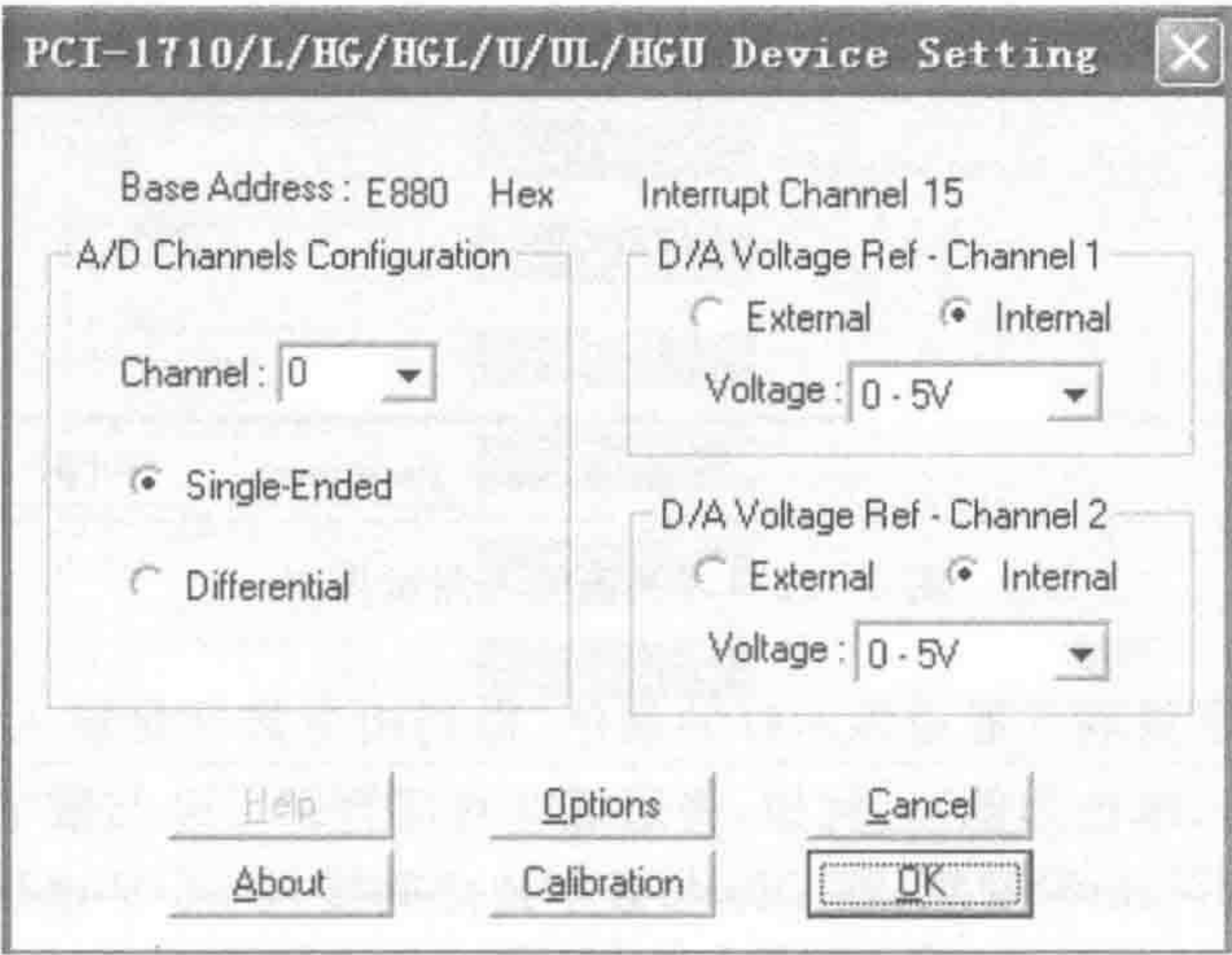


图 5-10 设置界面

2. 测试

测试时用 PCL-10168 电缆将 PCI-1710 采集卡与 ADAM-3968 端子板连接,如图 5-11 所示。PCI-1710 采集卡已安装在计算机的 PCI 插槽里。这样 PCL-1710 的 68 个针脚和 ADAM-3968 的 68 个接线端子一一对应,可通过将输入信号连接到接线端子来测试 PCI-1710 的功能。

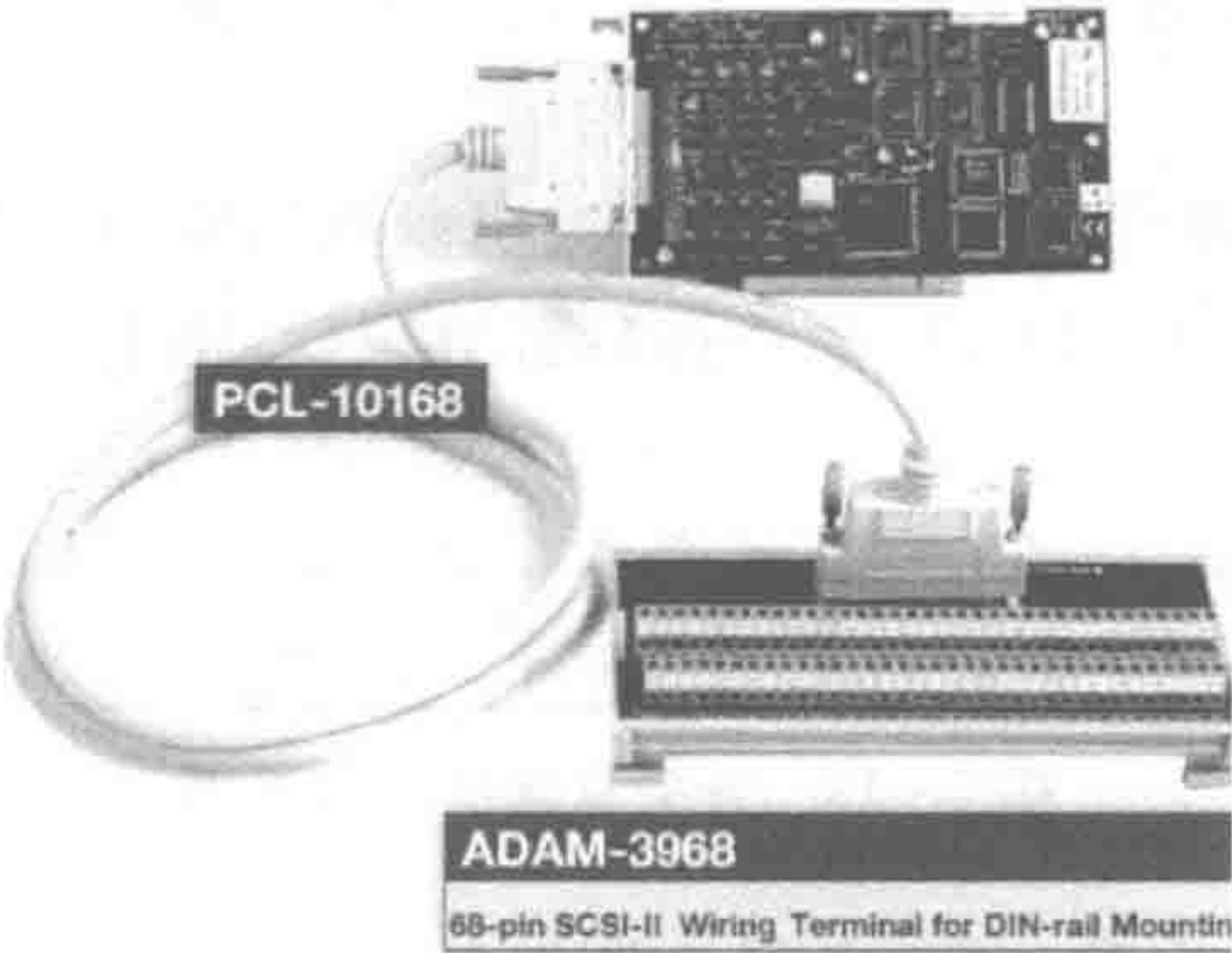


图 5-11 采集卡与端子板的连接

①开关量输入功能的测试。在图 5-9 的界面中点击“Test”，弹出界面后选择“Digital input”，进入开关量输入测试界面，如图 5-12 所示。

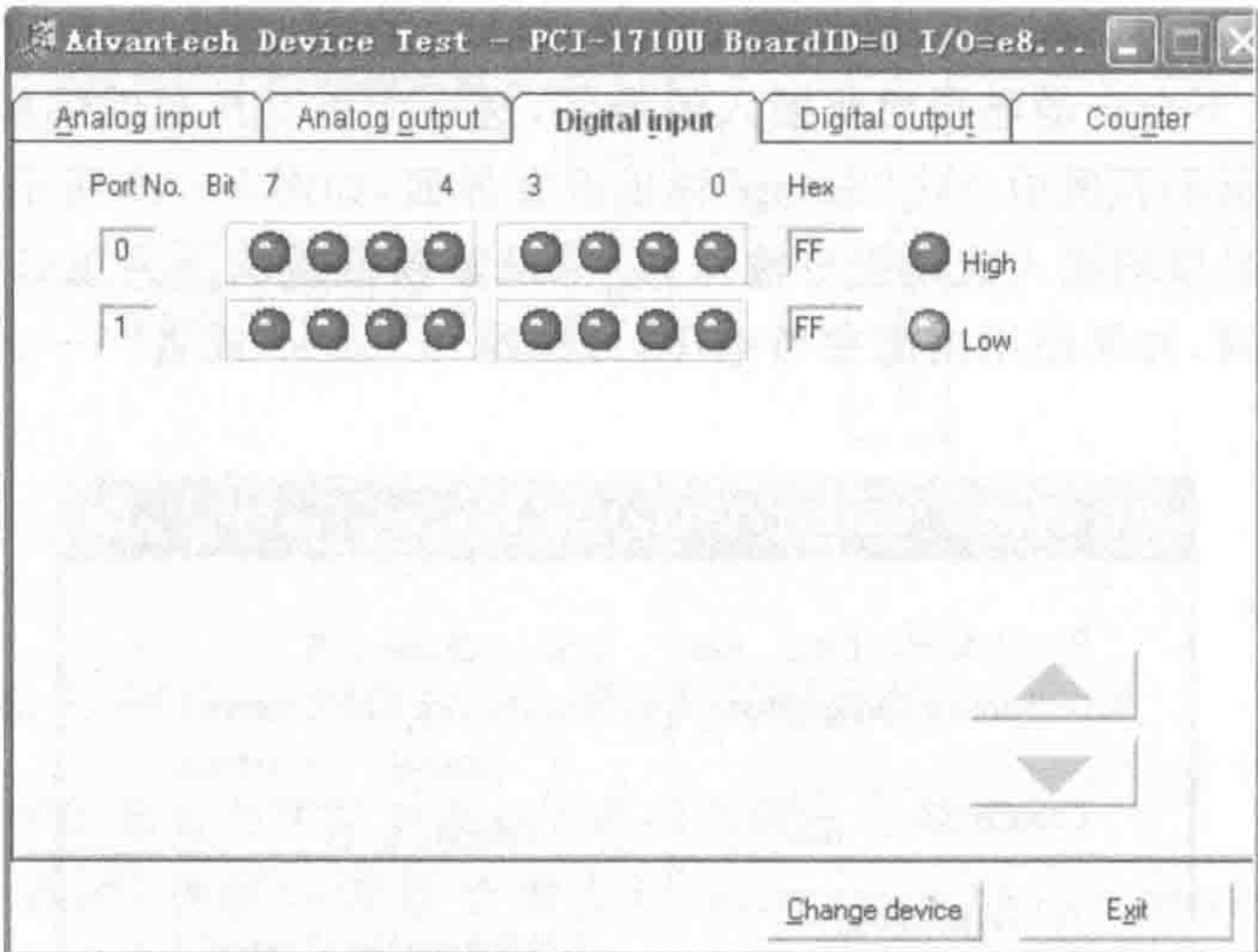


图 5-12 开关量输入测试界面

用户可以通过数字量输入通道指示灯的颜色，得到相应数字量输入通道输入的是低电平还是高电平（红色为高，绿色为低）。例如，将通道 0 对应管脚 DI0 与数字地 DGND 短接，则通道 0 对应的状态指示灯（Bit0）变绿，在 DI0 与数字地之间接入 +5 V 电压，则指示灯变红。

②开关量输出功能的测试。选择“Digital output”，进入开关量输出测试界面，如图 5-13 所示。

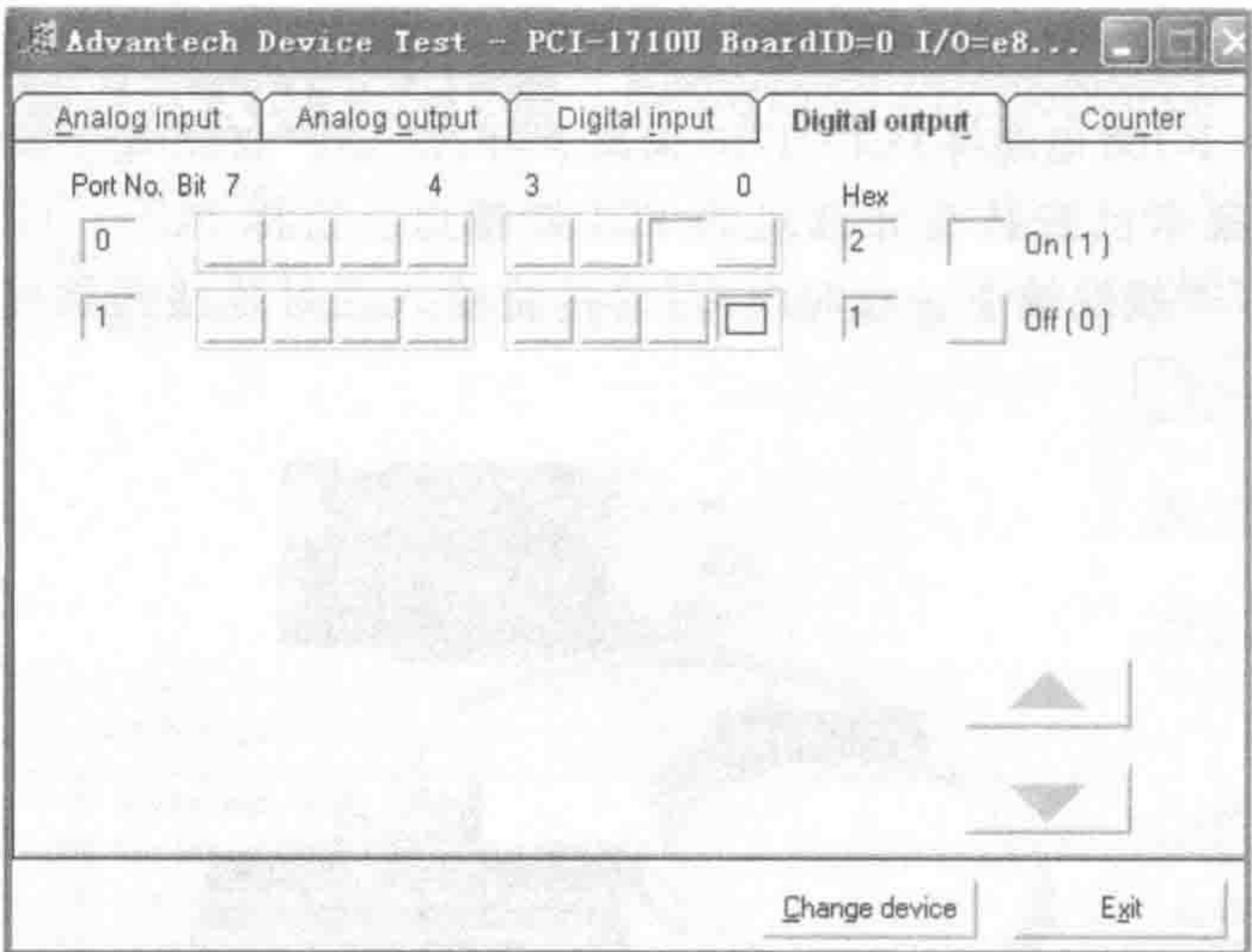


图 5-13 开关量输出测试界面

用户可以通过按动界面中的方框，方便地将相对应的输出通道设为高电平输出或低电平输出。用万用表测试相应管脚，可以测得高电平输出时为 4.5 V，低电平输出时为 0.15 V。用电压表测试相应管脚，可以测到这个电压。例如图 5-13 中，低八位输出 2，高八位输出 1（十六进制），即 DO1 输出高电平，DO8 输出高电平，其余输出通道输出均为低电平。

③模拟量输入功能的测试。选择“Analog input”，进入模拟量输入测试界面，如图 5 - 14 所示。

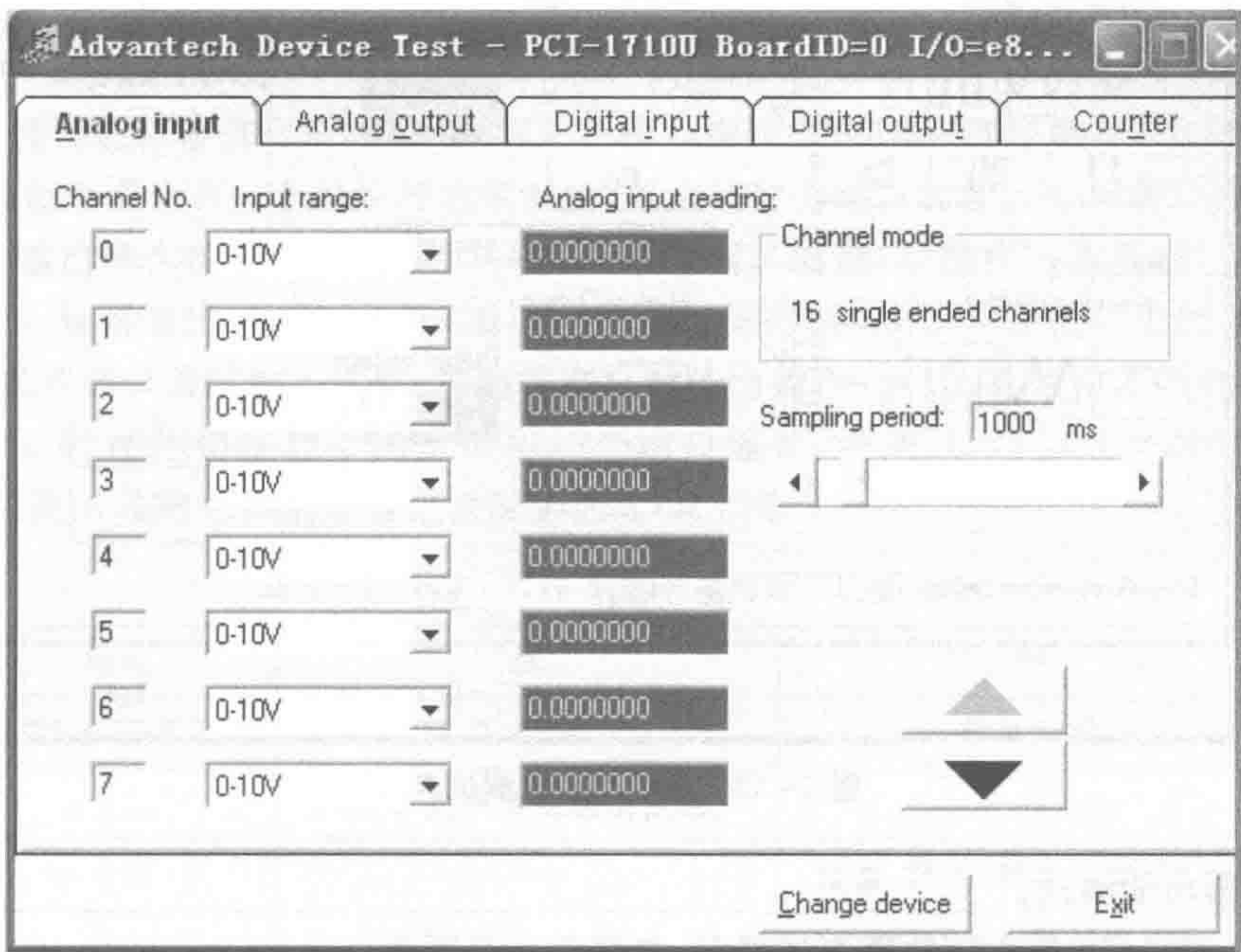


图 5 - 14 模拟量输入测试界面

- 测试界面说明：
- Channel No:模拟量输入通道号(0—15)。
 - Input range:输入范围选择。
 - Analog input reading :模拟量输入通道读取的数值。
 - Channel mode:通道设定模式。
 - sampling period :采样时间间隔。

④模拟量输出功能的测试。选择“Analog output”，进入模拟量输出测试界面，如图5 - 15 所示。

两个模拟量输出通道可以分别选择输出正弦波、三角波或方波,各波形的最低电平和最高电平也可以设置,然后按“▷”按钮,即可在相应通道输出。也可以直接设置输出电压幅值,点击“Out”按钮,即可在相应通道输出。例如,要使通道 0 输出 4.5 V 电压,在“Manual Output”中设置输出值为 4.5 V,点击“Out”按钮,即可在管脚 AO0_OUT 与 AO_GND 之间输出 4.5 V 电压,这个值可用万用表测得,在图 5 - 15 的界面“Output Voltage”项也可以观察到相应的输出。

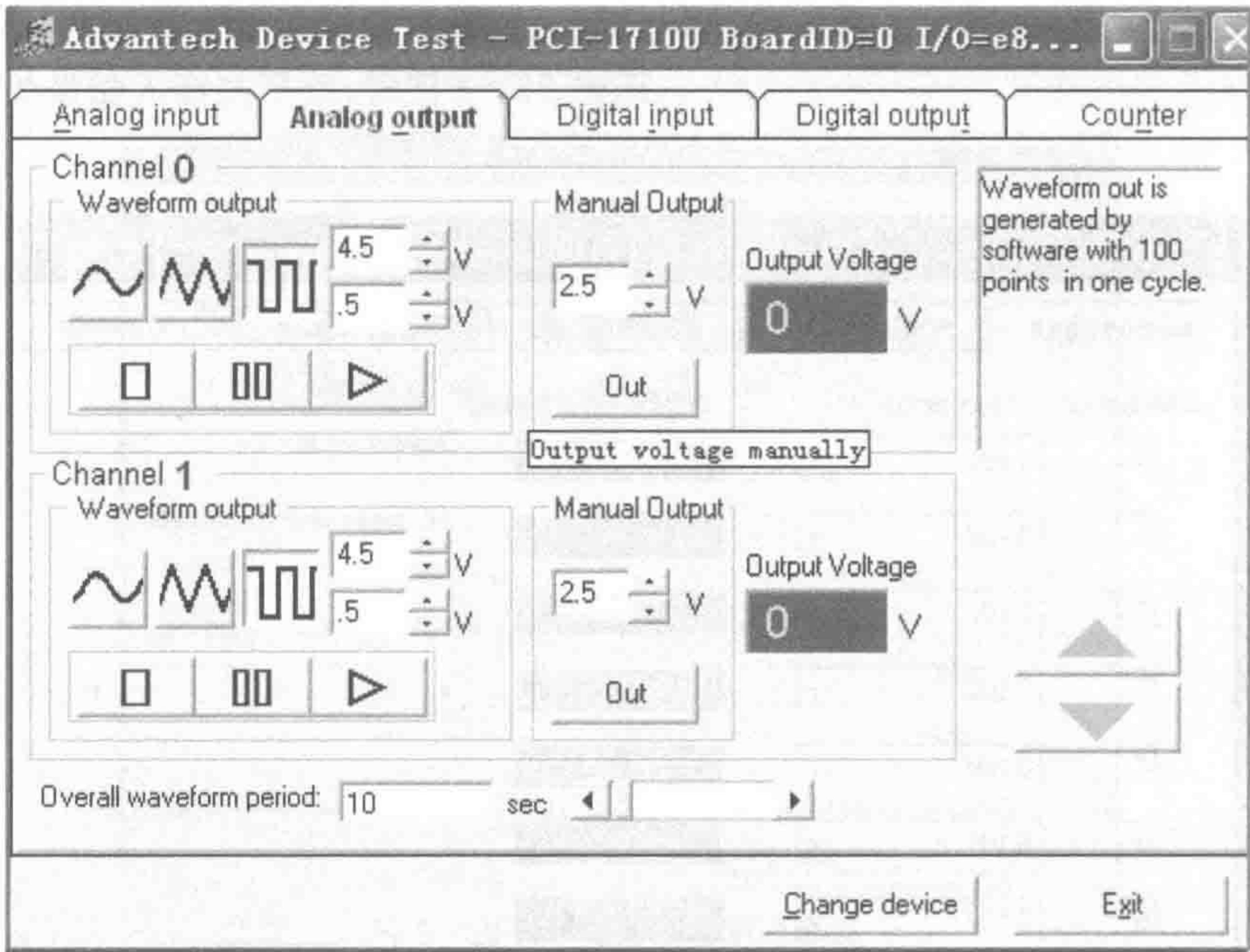


图 5-15 模拟量输出测试界面

⑤计数器功能测试。

选择“Counter”，进入计数器测试界面，如图 5-16 所示。

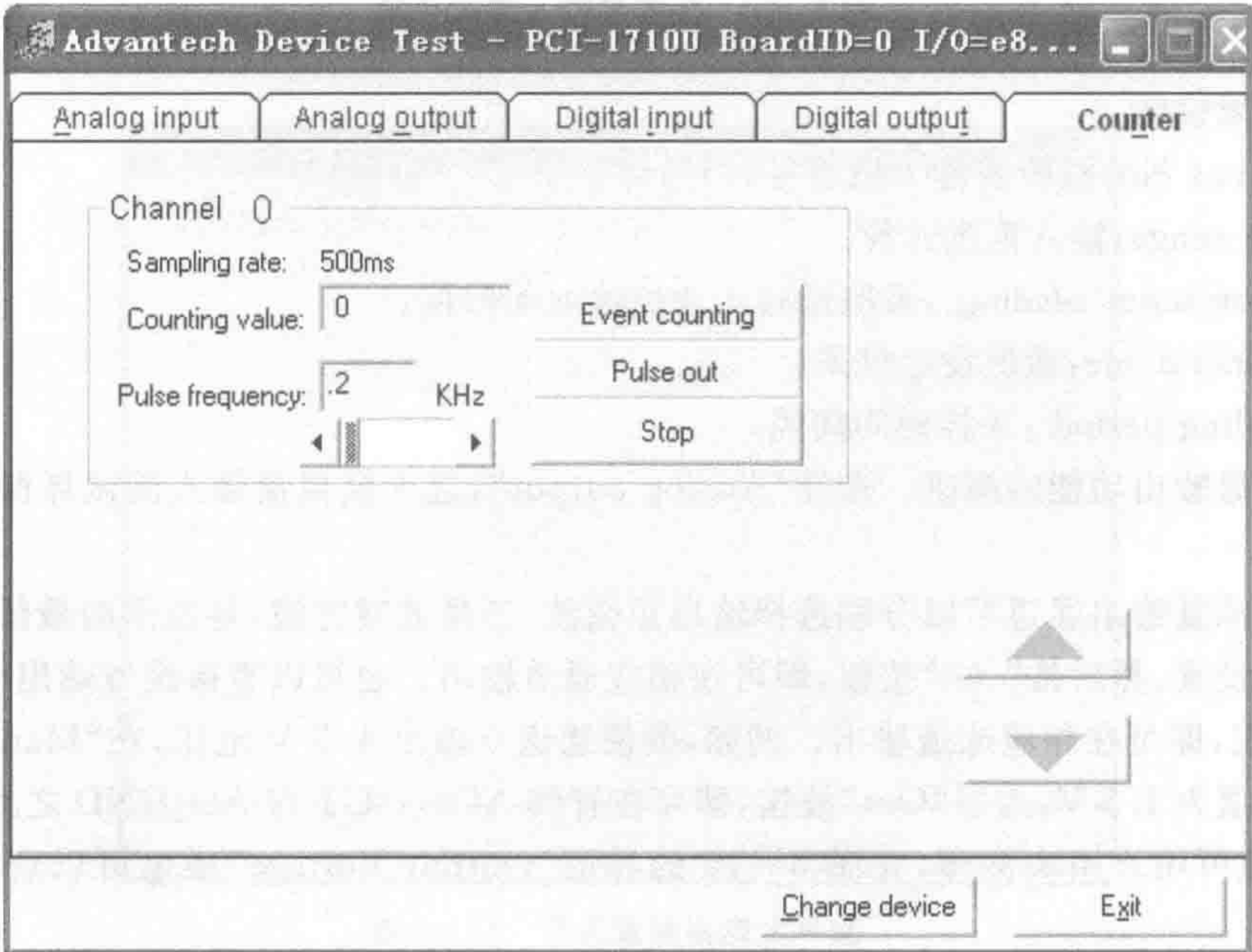


图 5-16 计数器测试界面

用户可以选择 Event counting(事件计数)或者 pulse out(脉冲输出)两种功能。选择事件计数时,将信号发生器接到管脚 CNT0-CLK,当 CNT0-GATE 悬空或接+5 V 时,事件计数

器将开始计数。例如：在管脚 CNT0 - CLK 接 100 Hz 的方波信号，计数器将累加方波信号的个数。如果选择脉冲输出，管脚 CNT0 - OUT 将输出频率信号，输出信号的频率可以设置。例如图上显示，设置输出信号的频率为 0.2 kHz。

5.3.5 采集卡 PCI-1710 端口地址分配

CPU 与外设进行信息交换时，各类信息在接口中存入不同的寄存器，一般称这些寄存器为 I/O 端口，每个端口有一个地址与之相对应，该地址称为端口地址。有了端口地址，CPU 对外设的输入/输出操作实际上就是对 I/O 接口中各端口的读/写操作。数据端口一般是双向的，数据是输入还是输出，取决于对该端口地址进行操作时 CPU 发往接口电路的读/写控制信号。由于状态端口只做输入操作，控制端口只做输出操作，因此，有时为了节省系统地址空间，在设计接口时往往将这两个端口共用一个端口地址，再用读/写信号来分别选择访问。PCI-1710 采集卡常用端口地址分配情况如表 5-1 所示。

表 5-1 PCI-1710 采集卡常用端口地址分配

地址	读	写
Base+0	A/D 低字节	软件触发
+1	A/D 高字节及通道信息	—
+2	—	增益、极性、单端与差动控制
+4	—	多路开关起始通道控制
+5	—	多路开关结束通道控制
+6	—	A/D 工作模式控制
+7	A/D 状态信息	—
+9		FIFO 清空
+10	—	D/A 通道 0 低字节
+11	—	D/A 通道 0 高字节
+12	—	D/A 通道 1 低字节
+13	—	D/A 通道 1 高字节
+14	—	D/A 参考控制
+16	DI 低字节	DO 低字节
+17	DI 高字节	DO 高字节

- 需要注意的是：
- ①一个端口在某一时刻只能读或只能写。
 - ②一个端口对应一个寄存器，一个寄存器可以存放一个字节的数据。
 - ③每一个端口对应的寄存器都有规定的数据格式，定义了每一位的意义(见附录 3)。
 - ④每一个采集卡有一个起始地址，称为基地址(BASE)。PCI 总线的板卡插好后计算机操作系统会自动识别，并显示分配给板卡的基地址。在本章的训练中，PCI-1710 采集卡的基地址为 0xE880。

⑤访问端口时的完整地址为：

采集卡各端口的地址=基地址+偏移量

⑥为了系统更为安全,Windows 2000 以上操作系统对系统底层操作采取了屏蔽的策略,由于 Windows 对系统的保护,绝对不允许任何的直接 I/O 动作发生。WinIO 库通过使用内核模式下设备驱动程序和其他一些底层编程技巧绕过 Windows 安全保护机制,允许 32 位 Windows 程序直接对 I/O 口进行操作。所以,在对端口进行读写操作时,需调用 WinIO 库,具体参见附录 2 中 F2.7 节。

⑦在 VC 环境下对端口的读写函数：

对端口进行读操作：_inp(端口地址)

对端口进行写操作：_outp(端口地址,变量名)

5.4 开关量的输入、输出通道

5.4.1 开关量的概念

开关量顾名思义就是只有开和关两种状态的工程量,如开关的闭合与断开,指示灯的亮与灭,继电器或接触器的吸合与释放,电机的启动与停止,阀门的打开与关闭等。这些信号的共同特征是以二进制的逻辑“1”和“0”出现的,代表生产过程的一个状态。开关量只要用一位二进制数即可表示,也就是说这种变量的值要么是 0,要么是 1。

5.4.2 开关量输入通道的功能

开关量输入通道的基本功能是将生产装置或生产过程产生的开关量信号转换成计算机需要的电平信号,以二进制数的形式输入计算机。这些开关量信号的形式一般是电压、电流和开关的触点,因此容易引起瞬时高压、过电流或接触抖动等现象,因此为使信号安全可靠,在开关量输入电路中,主要是考虑信号调理技术,如电平转换、RC 滤波、过电压保护、反电压保护、光电隔离等。

①电平转换是用电阻分压法把现场的电流信号转换为电压信号。

②RC 滤波是用 RC 滤波器滤除高频干扰。

③过电压保护是用稳压管和限流电阻作过电压保护;用稳压管或压敏电阻把瞬态尖峰电压箝位在安全电平上。

④反电压保护是串联一个二极管防止反极性电压输入。

⑤光电隔离是用光耦隔离器实现计算机与外部的完全电隔离。

5.4.3 开关量输出通道的功能

开关量输出通道的功能是把计算机输出的数字信号传送给开关器件(如继电器或指示灯),控制它们的通、断或亮、灭。开关量输出通道简称 DO 通道。

在输出通道中,为防止现场强电磁干扰或工频电压通过输出通道反串到 CPU 系统中,一般需要采用通道隔离技术。目前,光电耦合器件和继电器常用作开关量输出隔离器件。

计算机输出的是微弱数字信号,为了能对生产过程中的开关量执行器进行控制,需根据现

场负荷的不同,如指示灯、继电器、接触器、电机、阀门等,可以选用不同的功率放大器件构成不同的开关量驱动电路。常用的有三极管输出驱动电路、继电器输出驱动电路、晶闸管输出驱动电路、固态继电器输出驱动电路等。

5.4.4 开关量用于顺序控制

顺序控制是以预先规定好的时间或条件为依据,按照预先规定好的动作次序顺序地完成工作的自动控制。简而言之,顺序控制就是按时序或事序规定工作的自动控制。在工业生产中,许多生产工序,如运输、加工、检验、装配、包装等,都要求顺序控制。在一些复杂的大型计算机控制系统中,许多环节需要采用顺序控制方法,例如,有些生产机械要求在现场输入信号(如行程开关、按钮、光电开关、各种继电器等)的作用下,根据一定的转换条件实现有顺序的开关动作;而有些生产机械则要求按照一定的时间先后次序实现有顺序的开关动作。

顺序控制系统的实现相对容易些,对于按事序工作的环节(系统),计算机从生产现场获取信号,然后按工艺要求对有关的输入信号进行“与”“或”“非”等基本逻辑运算与判断,然后将结果通过开关量输出通道向执行机构发出控制指令,即可实现顺序控制;对于按时序工作的环节(系统),由计算机产生必要的时序信号,再判断按工艺要求规定的时间间隔是否已到,判断结果通过开关量输出通道输出,控制执行器动作,从而实现顺序控制。

开关量控制的特点如下:

- ①为了完成某一特定任务,常常需要进行多个被控对象的多步动作的控制。
- ②动作有比较固定的规律,而且不随意改变。
- ③动作有时不仅根据条件进行,还要根据前一个动作的持续时间去进行。
- ④动作通常是有始有终或周期性的。

5.4.5 开关量输入的实现

PCI-1710 采集卡提供有 16 路开关量输入通道。因为一路开关量信号只要用一位二进制数即可表示,所以 16 路开关量输入通道就需对应两个八位寄存器。由表 5-1 可知,16 路开关量输入所对应的端口地址为 BASE+16 和 BASE+17。开关量输入通道号与寄存器各位的对应关系如表 5-2 所示。

表 5-2 开关量输入通道号与寄存器各位的对应关系

BASE+16	D7	D6	D5	D4	D3	D2	D1	D0
DI 低字节	DI7	DI6	DI5	DI4	DI3	DI2	DI1	DI0
BASE+17	D7	D6	D5	D4	D3	D2	D1	D0
DI 高字节	DI15	DI14	DI13	DI12	DI11	DI10	DI9	DI8

若输入信号接在 0~7 号输入通道,则实现语句为:

```
DI_in=_inp(BASE+16);
```

若输入信号接在 8~15 号输入通道,则实现语句为:

```
DI_in=_inp(BASE+17);
```

例 开关量输入的简单程序。

```
#include <windows.h>
#include <iostream>
#include <conio.h>
#include "winio.h"           //winio 头文件
#pragma comment(lib,"winio.lib") //包含 winio 库
using namespace std;
void main(void)
{
    unsigned short BASE= 0xE880;
    int iPort=16;
    //初始化 WinIo
    if (! InitializeWinIo())
    {
        cout<<"Error In InitializeWinI!"<<endl;
        exit(1);
    }
    int DI_data;
    DI_data =_inp(BASE+iPort);
    cout<<" DI_data "<<"="<< DI_data<<endl;
    ShutdownWinIo();          //关闭 WinIo
}
```

5.4.6 开关量输出的实现

PCI-1710 采集卡提供有 16 路开关量输出通道。因为一路开关量控制信号只要用一位二进制数即可表示,所以 16 路开关量输出通道就需对应两个八位寄存器。由表 5-1 可知,16 路开关量输出所对应的端口地址为 BASE+16 和 BASE+17。开关量输出通道号与寄存器各位的对应关系如表 5-3 所示。

表 5-3 开关量输出通道号与寄存器各位的对应关系

BASE+16	D7	D6	D5	D4	D3	D2	D1	D0
D0 低字节	D07	D06	D05	D04	D03	D02	D01	D00

BASE+17	D7	D6	D5	D4	D3	D2	D1	D0
D0 高字节	D015	D014	D013	D012	D011	D010	D09	D08

若输出信号由 0~7 号输出通道输出,则实现语句为:

```
_outp(BASE+16,out_data);
```

若输出信号由 8~15 号输出通道输出,则实现语句为:

```
_outp(BASE+17,out_data);
```

5.5 模拟量的输入、输出通道

5.5.1 模拟量的概念

在某一时间段,时间与幅值都是连续的量称为模拟量。例如,在工业系统中,力、温度、位移、流量、压力、温度、速度等量都是模拟量。

在工业生产中,需要测量和控制的量往往是模拟量。为了利用计算机实现对工业生产过程的自动监测和控制,首先要能够将生产过程中监测设备输出的连续变化的模拟量转变为计算机能够识别和接受的数字量;其次,还要能够将计算机发出的控制命令转换为相应的模拟信号,去驱动模拟执行机构。这样两个过程,就需要由模拟量的输入和输出通道来完成。

5.5.2 模拟量输入通道的功能及组成

模拟量输入通道的功能是把被控对象的模拟量信号转换成计算机可以接收的数字量信号。模拟量输入通道一般由多路开关、前置放大器、采样保持器、A/D 转换器、接口和控制电路等组成,其核心是 A/D 转换器,所以,模拟量输入通道也简称为 A/D 通道。模拟量输入通道的组成如图 5-17 所示。

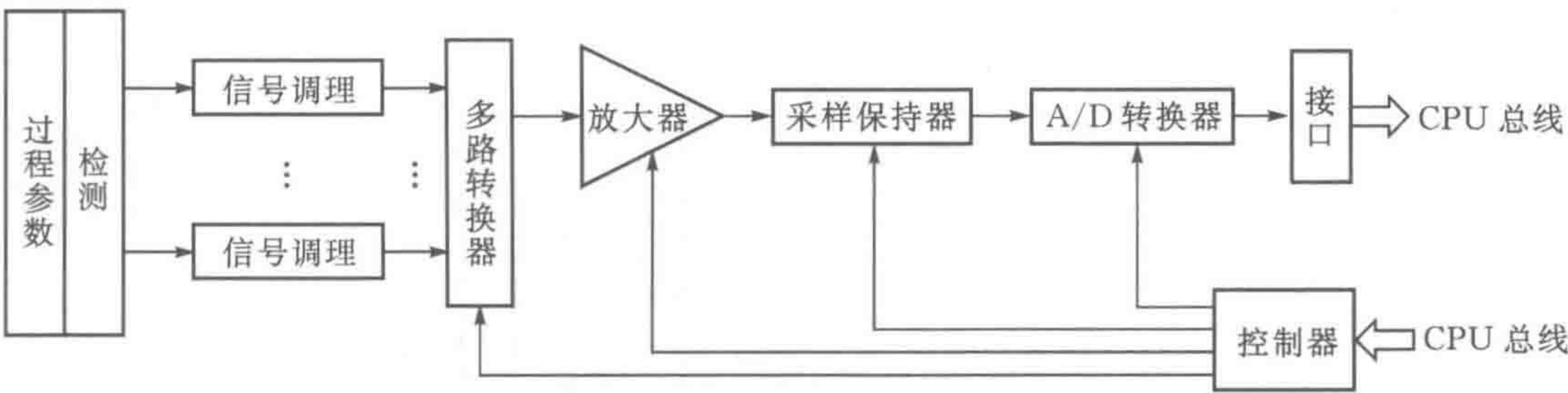


图 5-17 模拟量输入通道的组成

1. 信号调理

信号调理是整个控制系统中很重要的一环,关系到整个系统的精度。信号调理主要包括:小信号放大、信号滤波、信号衰减、阻抗匹配、电平变换、非线性补偿、频/压转换、电流/电压转换等。

2. 多路转换器

由于计算机的工作速度远远快于被测参数的变化,因此一台计算机系统可供几十个检测回路使用,但计算机在某一时刻只能接收一个回路的信号。所以,必须通过多路模拟开关实现多选一的操作,将多路输入信号依次地切换到后级电路。多路转换器把各路模拟量分时接到同一 A/D 转换器进行转换,实现了 CPU 对各路模拟量分时采样。

3. 可编程增益放大器

当多路输入的信号源电平相差较悬殊时,用同一增益的放大器去放大,就有可能使低电平

信号测量精度降低,而高电平信号则可能超出 A/D 转换器的输入范围。采用可编程增益放大器,可通过程序来调节放大倍数,使 A/D 转换信号满量程达到均一化,以提高多路数据采集的精度。

4. 采样保持器

当某一通道进行 A/D 转换时,由于 A/D 转换需要一定的时间,如果输入信号变化较快,而 A/D 转换都要花一定的时间才能完成转换过程,这样就会造成一定的误差,使转换所得到的数字量不能真正代表发出转换命令那一瞬间所要转换的电平。采用采样保持器对变化的模拟信号进行快速“采样”,在保持期间,启动 A/D 转换器,从而保证 A/D 转换时的模拟输入电压恒定,确保 A/D 转换精度。

(1) 采样

采样是把在时间上连续的输入模拟信号 $y(t)$ 转换成在时间上断续的信号 $y^*(t)$,输出脉冲波的包络仍反映输入信号幅度的大小。

①采样过程:按一定的时间间隔 T ,把时间上连续和幅值上也连续的模拟信号,转变成在时刻 $0, T, 2T, \dots, kT$ 的一连串脉冲输出信号的过程称为采样过程,时间间隔 T 称为采样周期,如图 5-18 所示。

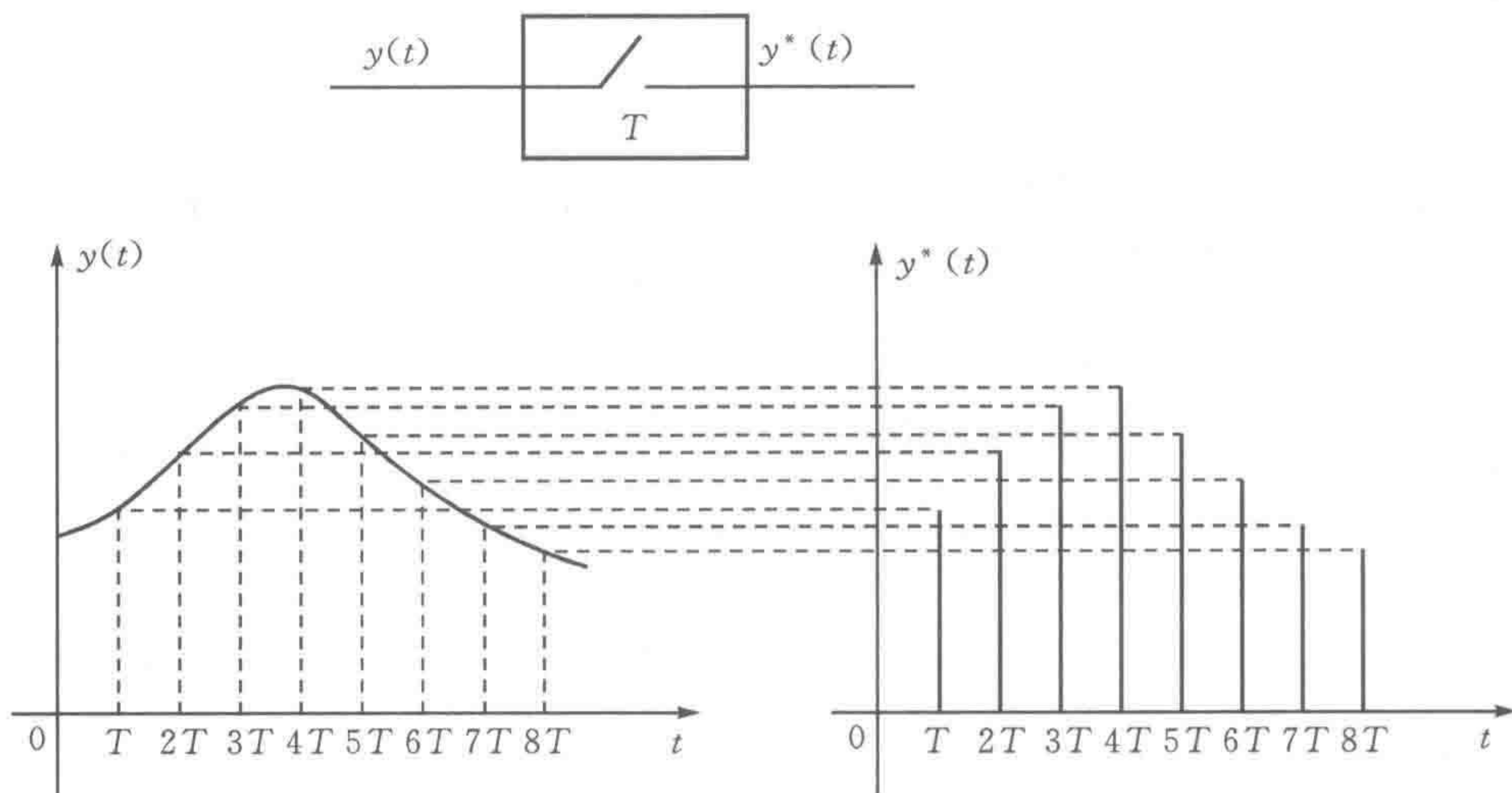


图 5-18 模拟信号采样过程图示

由采样过程可以看出,获得的采样信号仅保留了原模拟信号采样瞬间幅值的大小,而在采样间隔,原模拟信号的幅值信息丢失掉了。由经验可知,采样频率(采样周期的倒数)越高,采样信号 $y^*(t)$ 越接近原信号 $y(t)$,但若采样频率过高,在实时控制系统中将会把许多宝贵的时间用在采样上,从而失去了实时控制的机会。为了使采样信号 $y^*(t)$ 既不失真,又不会因频率太高而浪费时间,我们可依据香农采样定理进行采样。

②香农采样定理:为了使采样信号 $y^*(t)$ 能复现原信号 $y(t)$,采样频率 f 至少要为原信号最高频率 $f_{\max}$ 的两倍,即:

$$f \geq 2f_{\max} \quad (5-1)$$

采样定理给出了 $y^*(t)$ 唯一地复现 $y(t)$ 所必需的最低采样频率。一般实际应用中保证

采样频率为原信号最高频率的 5~10 倍,即:

$$f \geq (5 \sim 10) f_{\max} \tag{5-2}$$

③采样周期的选定:采样定理给出了采样周期的理论上限值,而在理论上,采样周期 T 越小,由离散信号复现连续信号的精度越高,当 $T \rightarrow 0$ 时,离散系统或数字系统就变为连续系统了。但是,在实际系统操作中,采样周期并非越小越好,而是有一个限度。应注意,设备输入/输出、计算机执行程序都需要耗费时间,因此,每次采样间隔不应小于设备输入/输出及计算机执行程序的时间,这是采样周期的下限值 $T_{\min}$,故采样周期应满足:

$$T_{\min} \leq T \leq T_{\max} \tag{5-3}$$

采样周期太小或太大,对系统都是不利的,若 T 太小,会增加计算机的计算负担,而且两次采样的间隔太短,偏差变化太小,使控制器的输出变化不大且调节过于频繁,会使某些执行机构不能及时响应;若 T 太大,调节间隔长,会使动态特性变差,而且干扰输入也得不到及时调节,使系统动态品质恶化,对某些系统,较大的采样周期将导致系统不稳定。因此,采样周期的选择要兼顾系统的动态性能指标、抗干扰能力、计算机的运算速度、执行机构的动作快慢等因素综合考虑。

(2)保持器

保持器的作用是在 A/D 对模拟量进行量化所需的转换时间内,保持采样点的数值不变,以保证 AD 转换精度。采样保持电路如图 5-19 所示。

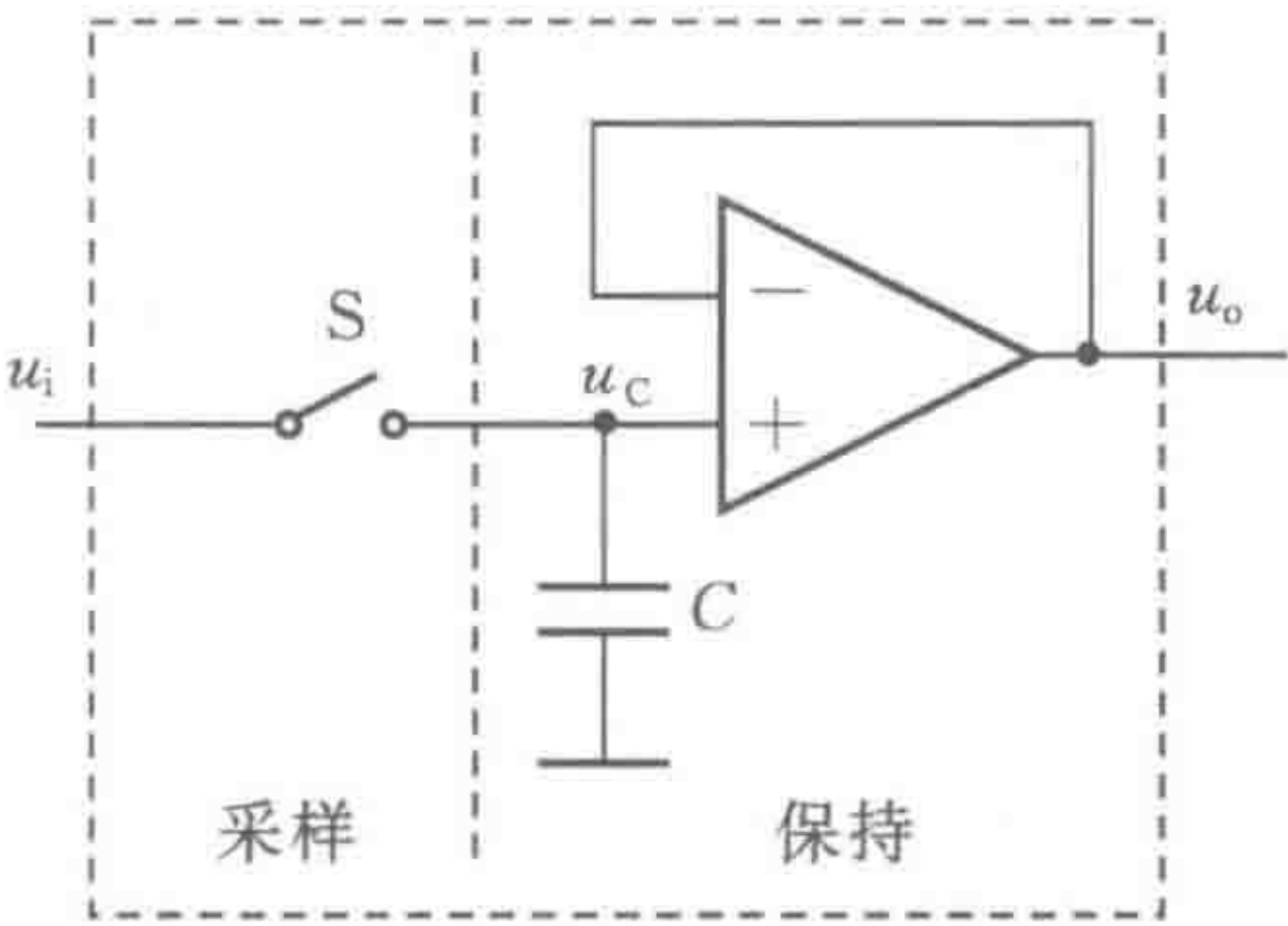


图 5-19 采样保持电路

在图 5-19 所示的采样保持电路中,当开关闭合时,电容快速充电, $u_c = u_i$, u_o 跟随 u_c , 即 $u_o = u_i$; 当 K 断开时, u_o 保持 u_c 。

保持器在采样期间,不启动 A/D 转换器,而一旦进入保持期间,则立即启动 A/D 转换器,从而保证 A/D 转换时的模拟输入电压恒定,以确保 A/D 转换精度。

5. A/D 转换器

A/D 转换器是将模拟信号转换成数字信号的器件,A/D 转换过程也就是幅值量化的过程,就是采用一组二进制码来逼近离散模拟信号的幅值,将其转换为数字信号。二进制数的大小和量化单位有关。

(1)量化单位

字长为 n 的 A/D 转换器,其最低有效位(LSB)所对应的模拟量 q 称为量化单位。

$$q = \frac{\text{Range}}{2^n - 1} \quad (\text{V}) \tag{5-4}$$

公式(5-4)中：

- Range——表示 A/D 转换器允许的输入电压范围；
- n ——是指 A/D 转换后表示数字信号的二进制数的位数。

(2) 量化过程

量化过程就是以 q 为单位去度量采样信号值的归整过程。如图 5-20 所示。

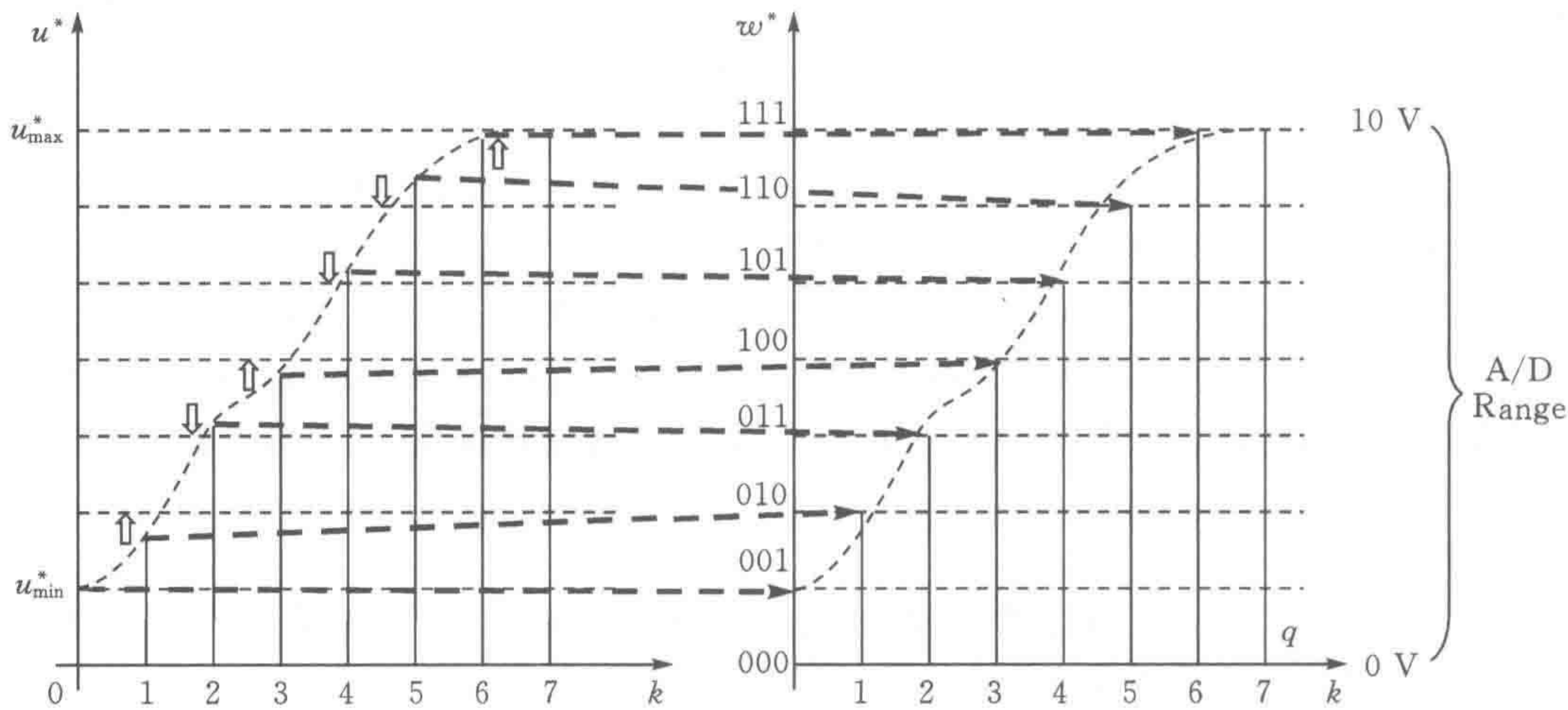


图 5-20 量化过程示意图

由图 5-20 可以看出，有的幅值大小可以由量化单位 q 的整数倍准确表示，有的需近似表示。近似可通过四舍五入法实现。

(3) 量化误差

由于量化过程是一个归整过程，因而存在量化误差，四舍五入法的量化误差为 $(\pm 1/2)q$ 。
A/D 转换器的量程应能够包括信号的幅值范围，量化单位应远小于信号的幅值范围，否则，量化过程的分辨能力会降低。当信号很弱时，信号的幅值范围可能和量化单位处于同一数量级，有时甚至小于量化单位，这样，A/D 转换器就不能分辨信号的变化。因此，在信号调理阶段，常对被测信号进行电压放大等预处理，以适合 A/D 转换的量程，或者调整可编程放大器的增益或改变 A/D 转换器的量程来适应信号调理电路的输出。

(4) A/D 转换触发方式

A/D 转换器在开始转换前，必须加一个触发信号，才能开始工作。常见的触发方式有以下三种：软件触发，板上定时触发，外部脉冲触发。

(5) 转换结束的判断与传输方式

在 A/D 转换器中，当计算机给 A/D 转换器发一个触发信号后，A/D 转换器开始转换，经过一段时间后，A/D 转换才可能结束。只有在 A/D 转换结束后，才能正确读取转换的数据。判断 A/D 转换结束的方法有以下三种。

- ① 中断方式。转换完成后，A/D 转换器主动向 CPU 发出中断请求，CPU 查询到中断申请并响应后，在中断服务程序中读取数据。其特点是中断方式使 A/D 转换器与计算机的并行工作。常用于实时性要求比较强或多参数的数据采集系统。
- ② 查询方式。CPU 主动查询 A/D 转换完成标志位，若完成，从端口读取结果。其特点是

程序设计比较简单,实时性也较强,应用最多。

查询方式软件编程步骤:

- 计算机向 A/D 转换器发出触发信号。
- 查询 A/D 转换结束标志位。
- 未结束,继续查询。
- 结束,读出结果数据。

在本章的学习中,我们采用软件触发 A/D 转换、以查询方式进行数据传输。

③DMA 方式。DMA 传输方式无需 CPU 直接控制传输,也没有中断处理方式那样保留现场和恢复现场的过程,通过硬件为 RAM 与 I/O 设备开辟一条直接传送数据的通路,使 CPU 的效率大为提高。

6. 编码

n 位二进制数可以表示 2^n 个数值,明确规定这 2^n 个二进制数中每一个数所对应的原信号值,这就是对信号进行编码。

对于 12 位的 PCI-1710 采集卡,数字量与模拟量的对应关系如表 5-4 所示。

表 5-4 12 位采集卡数字量与模拟量的对应关系

二进制数	十进制数	双极性 -10~+10 V 量程	单极性 0~+20 V 量程
1111 1111 1111	4095	+10 V	20 V
1111 1111 1110	4094		
...	...	...	...
0111 1111 1111	2047	0 V	10 V
...	...	...	...
0000 0000 0001	1		
0000 0000 0000	0	-10 V	0 V

5.5.3 模拟量输入的实现

以 PCI-1710 采集卡为例,介绍模拟量的输入过程的实现。

工业现场的模拟量状态信息经传感器获取、信号调理电路处理后,再通过模拟量输入通道完成 A/D 转换,生成数字量信号才能输入计算机。在此过程中,由图 5-17 可知,需解决以下问题:对 16 路模拟量输入信号来说,在某一时刻,要决定对哪一路信号进行采集;对幅值不同的各路输入信号共用一个 A/D 转换器,那么,如何提高输入信号的 A/D 转换精度;如何触发 A/D 转换;怎样判断 A/D 转换完成;转换完成后怎样读数;所读取的数字量信号与实际的模拟量信号的对应关系怎样? 下面结合具体使用的寄存器,针对提出的各问题,介绍如何通过编程来解决各问题、实现各功能。

1. 通道的选择

实现通道的选择,由多路转换控制寄存器 BASE+4 和 BASE+5 来完成。多路转换控制

寄存器 BASE+4 和 BASE+5 的数据格式及数据格式说明分别见表 5-5 和表 5-6。

表 5-5 多路转换控制寄存器 BASE+4 和 BASE+5

Bit #	7	6	5	4	3	2	1	0
BASE+5					STO3	STO2	STO1	STO0
BASE+4					STA3	STA2	STA1	STA0

表 5-6 多路转换控制寄存器数据格式说明

STA3~STA0	开始扫描通道编号
STO3~STO0	停止扫描通道编号

寄存器低 4 位表示的范围为 0000~1111,即 0~15,共 16 个数,分别表示 0~15 模拟量输入通道。

当选择单端输入方式时,开始扫描通道编号与停止扫描通道编号是一致的,例如选择模拟量输入 0 号通道输入时,控制字为:

Bit #	7	6	5	4	3	2	1	0
BASE+5					0	0	0	0
BASE+4					0	0	0	0

实现语句为:

```
_outp(BASE+4,0);
_outp(BASE+5,0);
```

2. 通道信号输入范围及增益的设定

设置通道范围及增益大小由寄存器 BASE+2 来完成。寄存器 BASE+2 的数据格式及数据格式说明分别见表 5-7 和表 5-8。增益码见表 5-9。

表 5-7 设置通道范围及增益的寄存器 BASE+2

Bit #	7	6	5	4	3	2	1	0
BASE+2			S/D	B/U		G2	G1	G0

表 5-8 寄存器 BASE+2 数据格式说明

S/D	单端或差分	0 表示通道为单端,1 表示通道为差分
B/U	双极或单极	0 表示通道为双极,1 表示通道为单极
G2 to GO	增益码	

表 5-9 PCI-1710 增益码

增益	输入范围/V	B/U	增益码		
			G2	G1	G0
1	-5~+5	0	0	0	0
2	-2.5~+2.5	0	0	0	1
4	-1.25~+1.25	0	0	1	0
8	-0.625~+0.625	0	0	1	1
0.5	-10~10	0	1	0	0

当选择单端双极性输入,输入范围为-10~+10 V 时,控制字为:

Bit #	7	6	5	4	3	2	1	0
BASE+2			0	0		1	0	0

实现语句为:

```
_outp(BASE+2,0x04);
```

3. A/D 触发模式的选择

选择触发模式由控制寄存器 BASE+6 来完成,控制寄存器 BASE+6 的数据格式及数据格式说明分别见表 5-10 和表 5-11。

表 5-10 控制寄存器 BASE+6

Bit #	7	6	5	4	3	2	1	0
BASE+6	AD16/12	CNT0	CNE/FH	IRQEN	GATE	EXT	PACER	SW

表 5-11 控制寄存器 BASE+6 数据格式说明

SW	软件触发启用位	设为 1 可启用软件触发,设为 0 则禁用
PACER	触发器触发启用位	设为 1 可启用触发器触发,设为 0 则禁用
EXT	外部触发启用位	设为 1 可启用外部触发,设为 0 则禁用
GATE	外部触发门功能启用位	设为 1 可启用外部触发门功能,设为 0 则禁用
IRQEN	中断启用位	设为 1 可启用中断,设为 0 则禁用
ONE/FH	中断源位	设为 0 将在发生 A/D 转换时生成中断,设为 1 则在 FIFO 半满时生成中断
CNT0	计数器 0 时钟源选择位	0 表示计数器 0 的时钟源为内部时钟(100 kHz)、1 表示计数器 0 的时钟源为外部时钟(最大可达 10 MHz)

注:用户不能同时启用软件触发、触发器触发和外部触发。

当选择软件触发方式时,控制字为:

Bit #	7	6	5	4	3	2	1	0
BASE+6	0	0	0	0	0	0	0	1

实现语句为：

```
_outp(BASE+6,0x01);
```

4. 软件触发 A/D 转换

如果选择软件触发 A/D 转换,则向寄存器 BASE+0 写入任意数据就可以触发 A/D 转换。寄存器 BASE+0 的数据格式如表 5-12 所示。

表 5-12 寄存器 BASE+0

Bit #	7	6	5	4	3	2	1	0
BASE+0								

例如,实现语句可以写为：

```
_outp(BASE+0,0);
```

5. A/D 转换是否完成的判定

FIFO 缓存器应用在数据采集卡上,主要用来存储 A/D 转换后的数据。FIFO 缓存器处于何种状态,由状态寄存器 BASE+7 的内容来显示。状态寄存器 BASE+7 的数据格式及数据格式说明分别见表 5-13 和表 5-14。

表 5-13 状态寄存器 BASE+7

Bit #	7	6	5	4	3	2	1	0
BASE+7	CAL				IRQ	F/F	F/H	F/E

表 5-14 状态寄存器 BASE+7 数据格式说明

F/E	FIFO 空标志	此位用于指标 FIFO 是否为空,1 表示 FIFO 为空
F/H	FIFO 半满标志	此位用于指标 FIFO 是否为半满,1 表示 FIFO 半满
F/F	FIFO 满标志	此位用于指标 FIFO 是否为满,1 表示 FIFO 为满
IRQ	中断标志	此位用于指示中断状态,1 表示已发生中断

在数据采集程序开始运行之初,先对 FIFO 缓存器清零,当 A/D 转换完成后,状态寄存器 BASE+7 便显示 FIFO 缓存器处于非空状态,即寄存器 BASE+7 最低位为 0。

实现语句为：

```
while(Status==1)
{
    Status=(_inp(BASE+7))&0x01;
}
if(Status==0)
{
    .....
}
```

6. A/D 转换完成后数据的读取

BASE+0 和 BASE+1 这两个寄存器保存 A/D 转换数据。A/D 转换的 12 位数据存储在

BASE+1 的位 3~位 0,以及 BASE+0 的位 7~位 0。BASE+1 的位 7~位 4 保存源 A/D 通道的编号,见表 5-15 和表 5-16。

表 5-15 A/D 数据寄存器 BASE+0 和 BASE+1

Bit #	7	6	5	4	3	2	1	0
BASE+1	CH3	CH2	CH1	CH0	AD11	AD10	AD9	AD8
BASE+0	AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0

表 5-16 A/D 数据寄存器数据格式说明

AD11~AD0	A/D 转换结果	AD0 是 A/D 数据中最低有效位(LSB),AD11 则是最高有效位(MSB)
CH3~CH0	A/D 通道编号	CH3~CH0 保存接收数据的 A/D 通道的编号,CH3 为 MSB,CH0 为 LSB

实现语句为:

```
tmp=_inpw(BASE+0);
adData=tmp&0xfff;
```

7. 代码转换

读取的数值与原信号值的对应关系由下面公式(5-5)可得

$$V_{_out} = adData * 20.0 / 4095 - 10.0$$
 (5-5)

式(5-5)中:

- adData——为读取的数值;
- V_out——为原信号值。

5.5.4 模拟量输出通道的功能及组成

计算机控制系统中,模拟量输出通道所要完成的功能是把计算机输出的数字量控制信号转换为模拟电压或电流信号,以便驱动相应的执行机构,从而达到控制的目的。

模拟量输出主要由 D/A 转换器和输出保持器组成。多路模拟量输出通道的结构形式,主要取决于输出保持器的结构形式,保持器一般有数字保持方案和模拟保持方案两种,这就决定了模拟量输出通道的两种基本结构形式。

1. 一个通路设置一个 D/A 转换器(结构如图 5-21 所示)

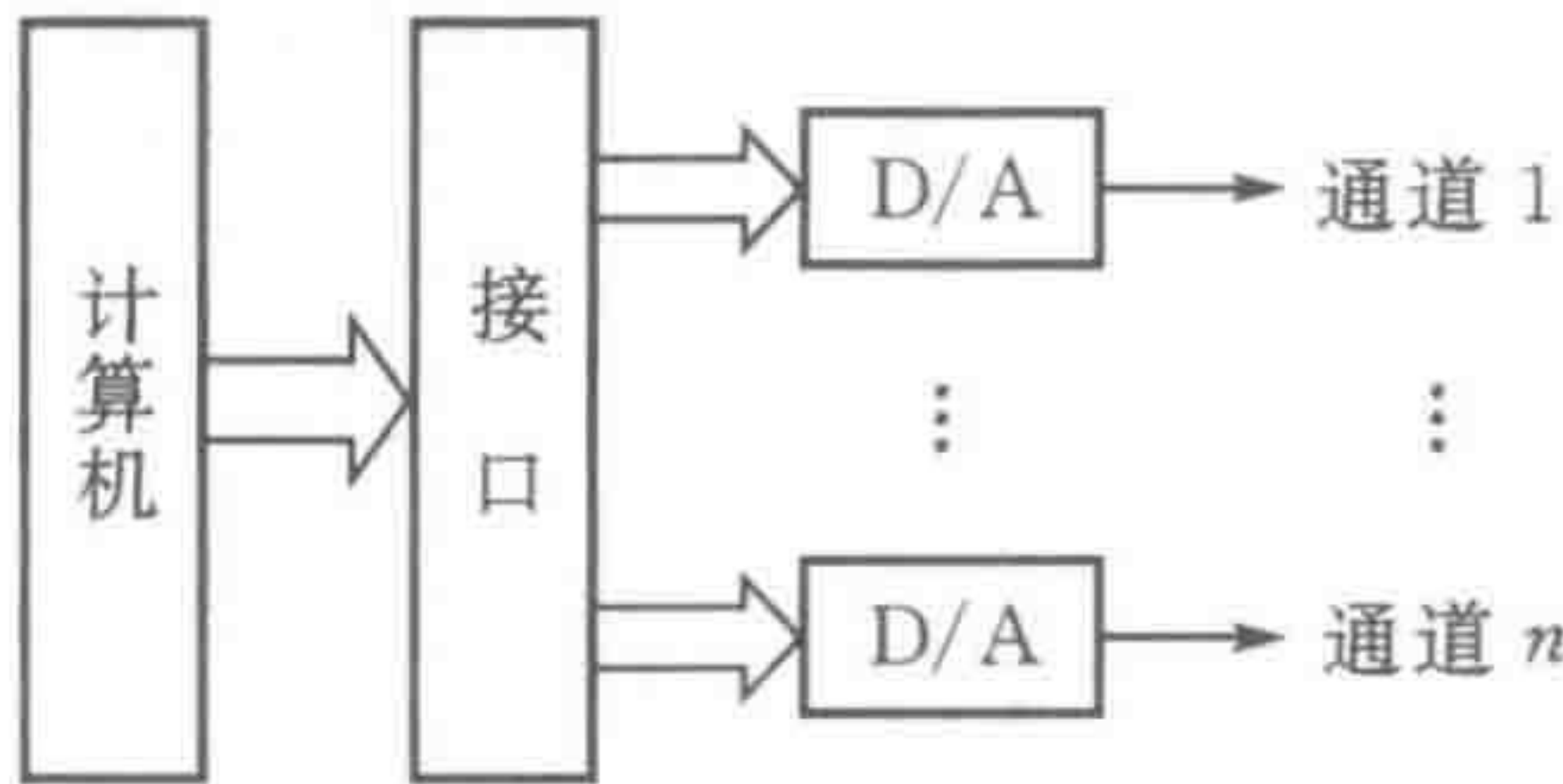


图 5-21 一个通路设置一个 D/A 转换器

该结构的优点是转换速度快、工作可靠。其缺点是使用较多的 D/A 转换器。

2. 多个通路共用一个 D/A 转换器(结构如图 5 - 22 所示)

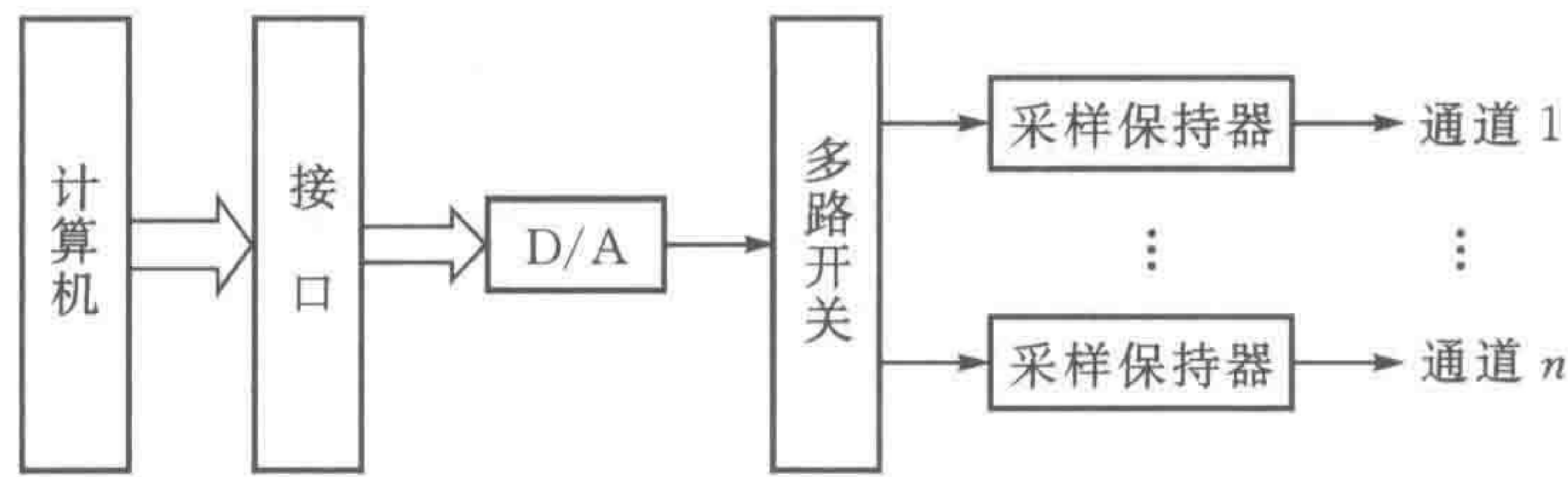


图 5 - 22 多个通路共用一个 D/A 转换器

该结构的优点是节省了 D/A 转换器。其缺点是计算机分时工作,工作可靠性差。

在本章的学习中,我们采用的 PCI - 1710 数据采集卡的模拟量输出通道采用的是一个通路设置一个 D/A 转换器的结构形式。

5.5.5 模拟量输出的实现

1. D/A 参考源的选择

D/A 参考源的选择由寄存器 BASE+14 来完成,寄存器 BASE+14 的数据格式和数据格式说明分别见表 5 - 17 和表 5 - 18。

表 5 - 17 用于设置 D/A 参考源的寄存器 BASE+14

Bit #	7	6	5	4	3	2	1	0
BASE+14					DA1_I/E	DA1_5/10	DA0/I/E	DA0_5/10

表 5 - 18 寄存器 BASE+14 数据格式说明

DA0_5/10	内部参考电压 用于 D/A 输出通道 0	此位控制用于 D/A 输出通道 0 的内部参考电压。0 表示内部参考电压为 5 V,1 表示 10 V
DA0_I/E	内部或外部参考电压 用于 D/A 输出通道 0	此位指标用于 D/A 输出通道 0 的参考电压为内部还是外部。0 表示参考电压来自内部源,1 表示来自外部源
DA1_5/10	内部参考电压 用于 D/A 输出通道 1	此位控制用于 D/A 输出通道 1 的内部参考电压。0 表示内部参考电压为 5 V,1 表示 10 V
DA1_I/E	内部或外部参考电压 用于 D/A 输出通道 0	此位指示用于 D/A 输出通道 1 的参考电压为内部还是外部。0 表示参考电压来自内部源,1 表示来自外部源

当选择 0 号输出通道,参考电压来自内部源,参考电压为 5 V 时,控制字为:

Bit #	7	6	5	4	3	2	1	0
BASE+14							0	0

实现语句为：

```
_outp(BASE+14,0);
```

2. 代码转换

参考电压选择 5 V 时,输出的电压值与其对应的数字量的关系由下公式(5-6)可得：

outData = fVoltage * 4095/5 (5-6)

式(5-6)中：

fVoltage——为输出的电压值；

outData——为对应的数字量。

3. 数据的输出

对于模拟量数据的输出,选择的输出通道不同,所用的数据寄存器就不同。

①D/A 输出 0 号通道。所用的数据寄存器为 BASE+10 和 BASE+11。BASE+10 和 BASE+11 的数据格式见表 5-19。

表 5-19 D/A 数据寄存器 BASE+10 和 BASE+11

Bit #	7	6	5	4	3	2	1	0
BASE+11					DA11	DA10	DA9	DA8
BASE+10	DA7	DA6	DA5	DA4	DA3	DA2	DA1	DA0

②D/A 输出 1 号通道。所用的数据寄存器为 BASE+12 和 BASE+13。BASE+12 和 BASE+13 的数据格式见表 5-20。

表 5-20 D/A 数据寄存器 BASE+12 和 BASE+13

Bit #	7	6	5	4	3	2	1	0
BASE+13					DA11	DA10	DA9	DA8
BASE+12	DA7	DA6	DA5	DA4	DA3	DA2	DA1	DA0

实现语句：

```
Hbyte=(outData>>8)&0x000f;  
Lbyte=outData &0x00ff;  
_outp(Base+10+port * 2, Lbyte);  
_outp(Base+11+port * 2, Hbyte);
```

5.6 训练内容

5.6.1 训练用 DI/DO 电路板介绍

DI/DO 电路板有两种训练板,如图 5-23 DI/DO 电路板 1、图 5-24 DI/DO 电路板 2 所示。

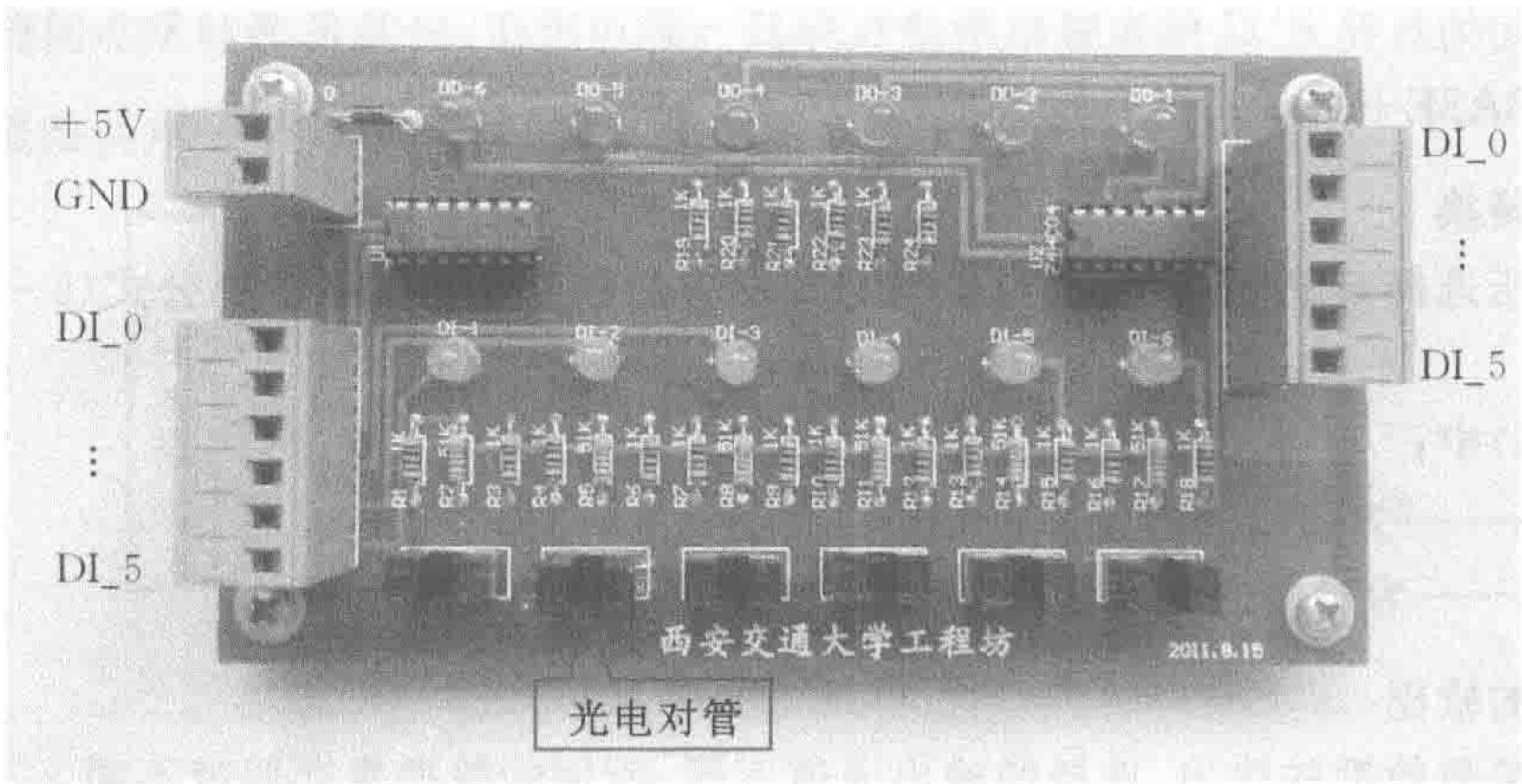


图 5-23 DI/DO 电路板 1 实物图

在 DI/DO 电路板 1 上,有六个光电对管,用来模拟来自工业现场的开关量,光电对管相应位置上的绿色发光二极管用来表示光电对管的状态,当光电对管通光时,相应位置上的绿色发光二极管点亮;当光电对管遮光时,相应位置上的绿色发光二极管熄灭。红色发光二极管用来模拟工业现场的受控开关量状态,当发出的控制量为 1 时,相应位置上的红色发光二极管点亮;当发出的控制量为 0 时,相应位置上的红色发光二极管熄灭。

在 DI/DO 电路板 2 上,有六个不同的开关量传感器,如图 5-24 所示,用光电对管、光敏二极管、微动开关、干簧管、行程开关、金属接近开关等用来模拟来自工业现场的开关量,相应位置上的绿色发光二极管用来表示开关量传感器的状态;用红色发光二极管、蜂鸣器和继电器等用来模拟工业现场的受控开关量的状态。

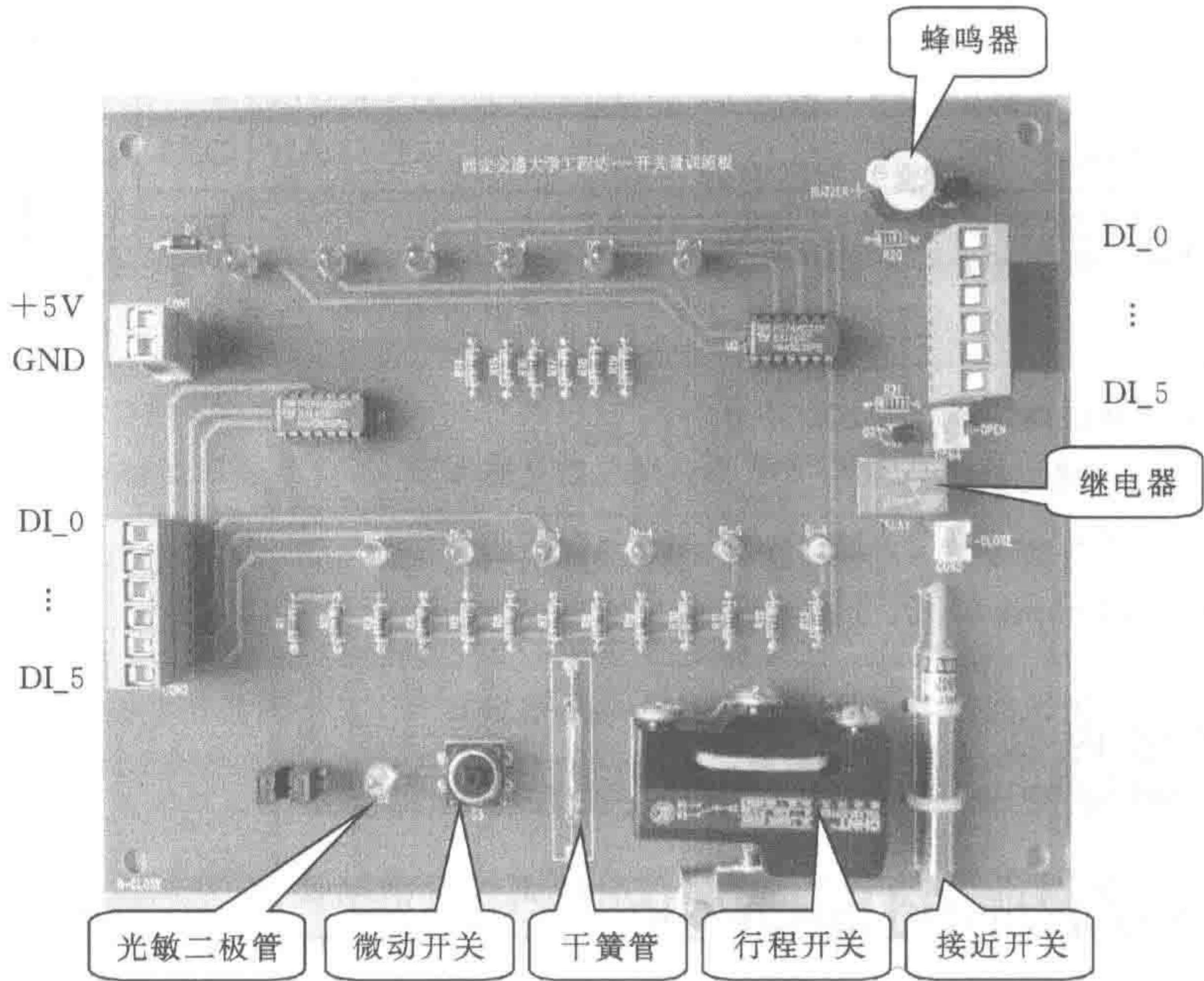


图 5-24 DI/DO 电路板 2 实物图

5.6.2 基础训练

开关量输入、输出基础训练接线图,如图 5-25 所示,任选一种开关量训练板连接线路。

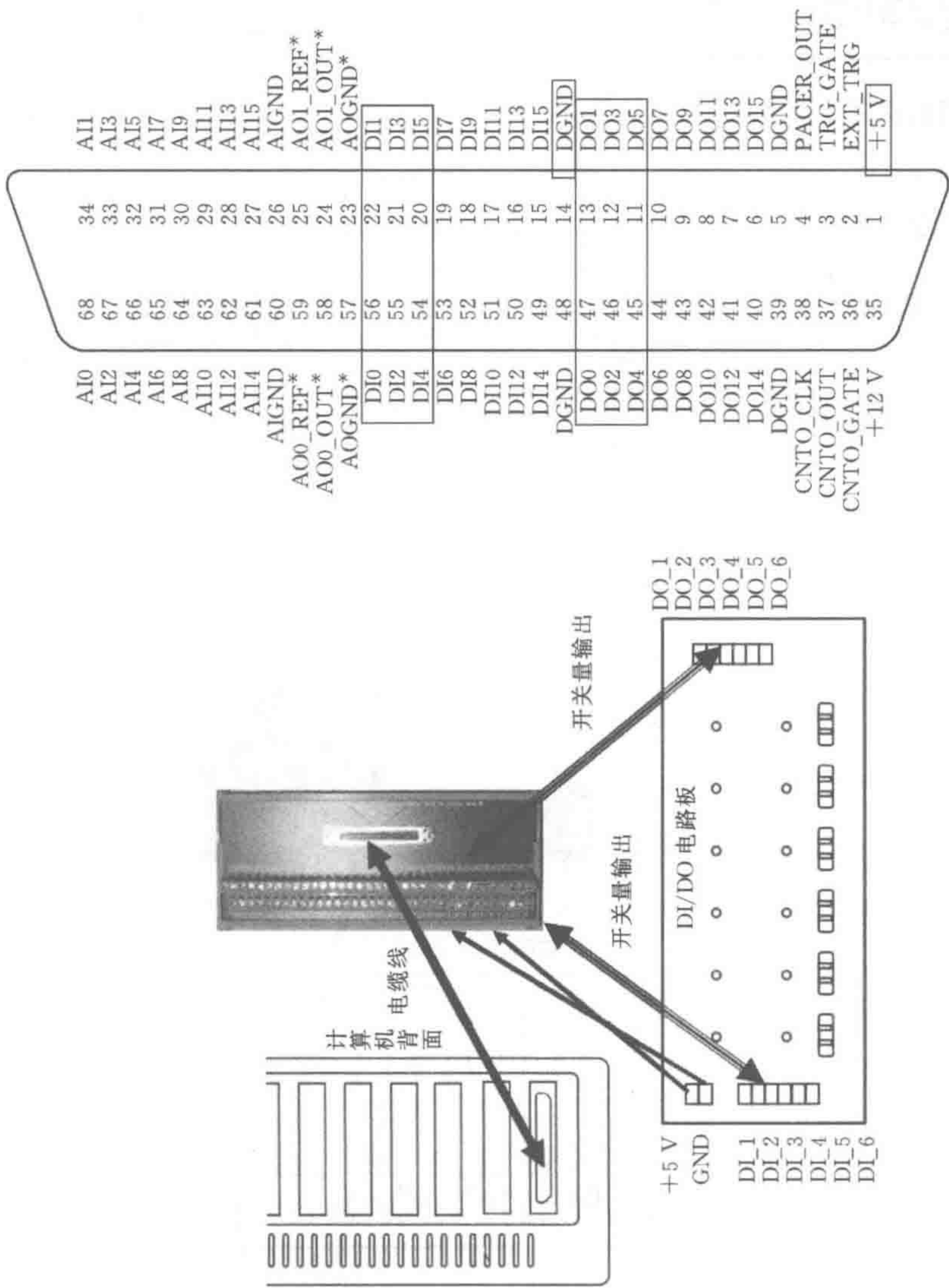


图 5-25 开关量输入输出训练接线图

- ①运行例 1 开关量的输入程序,熟悉 WinIO 库的使用,熟悉 I/O 端口读函数的应用;
- ②仿照例 1,应用 I/O 端口写函数,试编写开关量的输出程序,实现红色发光二极管亮暗的控制。
- ③运行例程 DItest.cpp 程序(见附录 4)。单步执行程序,获取 DI/DO 电路板上开关量的传感器的状态,观察 watch 区域中有关变量值的变化。
- ④运行例程 DOtest.cpp 程序(见附录 5)。单步执行程序,给出输出,控制 DI/DO 电路板上红色发光二极管的状态,观察“watch”区域中有关变量值的变化。

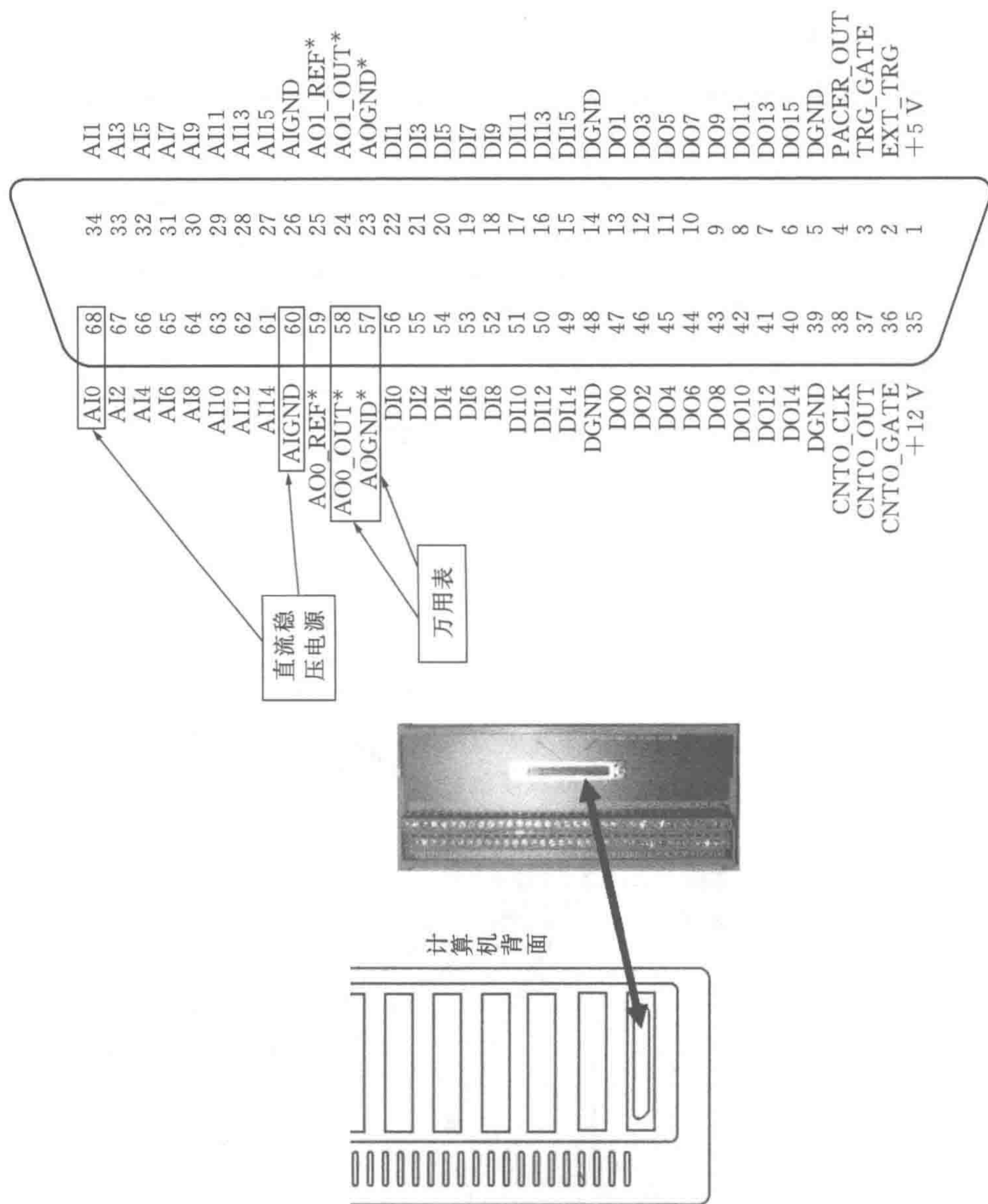


图 5-26 模拟量输入输出基础训练接线图

模拟量输入输出基础训练接线图,如图 5-26 所示,按图正确连接线路。

⑤运行例程 AItest.cpp 程序(见附录 6)。执行程序,该程序将一个直流电压信号采入计算机,以数字形式显示,程序运行中,利用“watch”区域观察有关变量的值,确认程序中的关键语句及其作用。

⑥运行例程 AOtest.cpp 程序(见附录 7)。执行程序,由键盘键入需要输出的直流电压值(0~+5 V)。用万用表在相应输出通道接线端测量,利用“watch”区域观察有关变量的值,确认并记录程序中的关键语句及其作用。

⑦编写 AD 子程序(改写原有 AIttest. cpp 程序)

```
float AD (int channel) /* 0—15 */
```

```
{..... }
```

⑧编写 DA 子程序(改写原有 AOtest. cpp 程序)

```
DA(int channel, float vout) /* 0—1; 0—5 V */
```

```
{..... }
```

5.6.3 综合训练

①光电对管遮光控制 LED 指示灯。

编写程序,采用 DI/DO 电路板 1 实现如下功能:遮挡某一光电对管时(绿灯显示其状态),相应位置上的红色 LED 熄灭。

具体要求:

- 做到实用控制。
- 避免程序死循环。

②城市公交车经常严重超员,请为自动刷卡公交车设计限制超员管理程序,并在 DI/DO 自然板上模拟运行。

具体要求:

- 上下车人员计数。
- 设定车内人员限额。
- 人员满额限制刷卡,禁止上车。

③皮带传输线控制。皮带传输线电动机拖动装置由 M1、M2、M3 三台电动机组成。启动时,为了避免在前段传输皮带上造成物料堆积,要求逆物流方向按一定的时间间隔顺序启动,其启动顺序为:M1→M2→M3 顺序启动,级间间隔时间为 3 s。停车时,为了使传输皮带上不残留物料,要求顺物流方向按一定的时间间隔顺序停止,其停止顺序为:M3→M2→M1 顺序停车,级间间隔时间为 3 s。并且要求:

- 紧急停车时,无条件地将 M1、M2、M3 立即全部同时停车。
- 任何一台电动机发生过载时,其保护停车应该按上述顺序进行。

④请参阅附录 8,熟悉电梯结构及控制原理。试以电梯为控制对象、以计算机为控制器编写控制程序,并在透明电梯模型上运行。

⑤编写程序实现以下功能:直流稳压电源输出 5 V 电压,经 AI 通道采集,转换后,显示在屏幕上,采集的值再经由 AO 通道输出,用万用表在相应的 AO 输出通道端测量,试比较 AI 采集值与 AO 输出值。

⑥编写程序实现由 AO 输出通道输出方波、三角波或正弦波。

5.6.4 拓展训练

1. 信号的显示

信号显示训练板如图 5-27 所示,由八位开关量控制两位数码管显示,其中低四位(DO0~DO3)控制低位数码管,高四位(DO4~DO7)控制高位数码管。单一数码管显示范围为 0~9。参见附录 9。

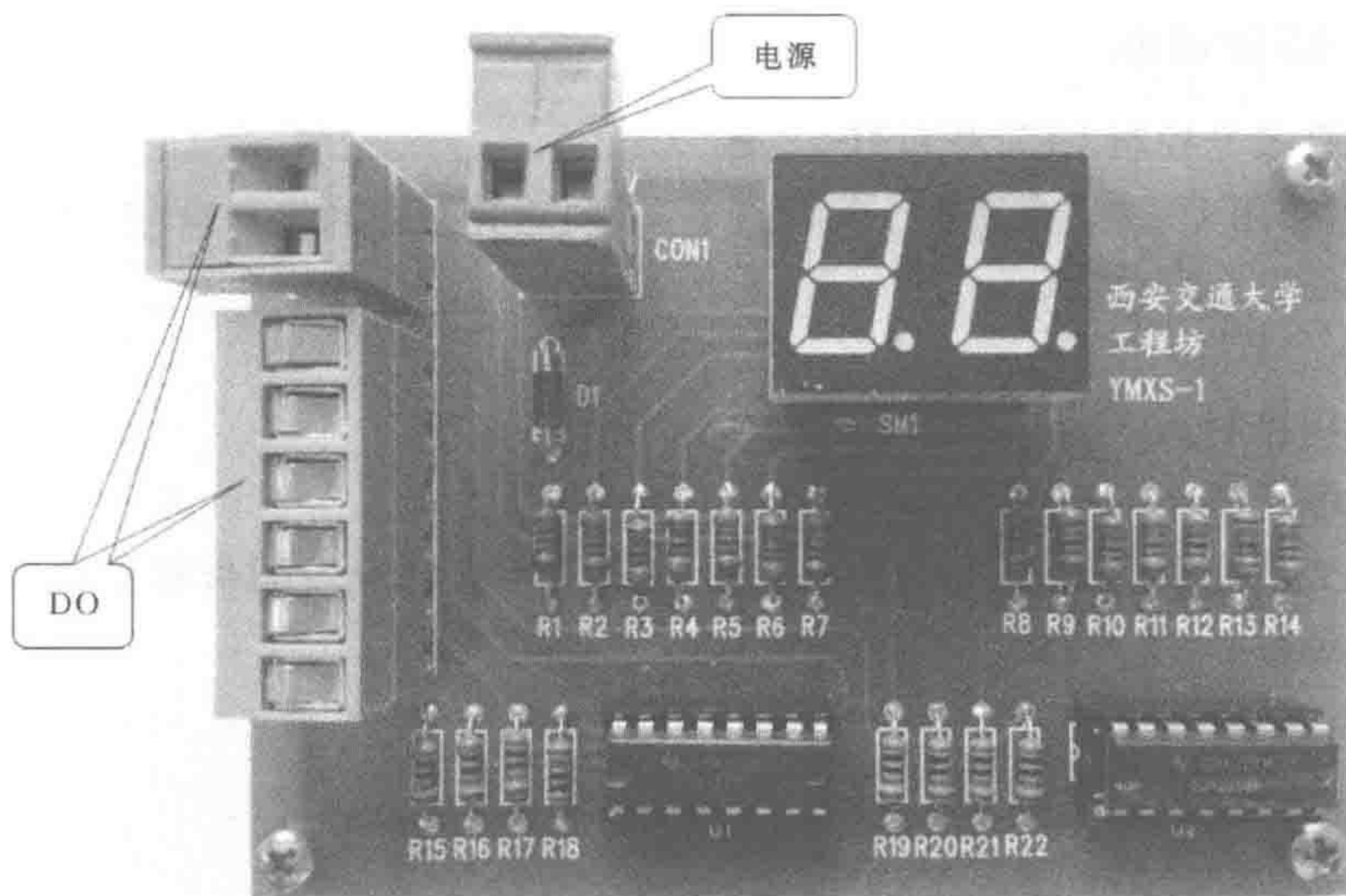


图 5-27 信号显示训练板实物图

- ①请编写程序,实现两位数码管显示十进制数 00~99。
- ②编写小区车库管理程序,要求:
 - 在 DI/DO 训练板上任选两只光电对管,模拟入库门和出库门。
 - 设定车库车位数目。
 - 利用信号显示训练板数码管动态显示当前车库剩余车位。

2. DO 驱动

DO 驱动训练板,如图 5-28 所示,其电路为一个开关量控制直流电机正反转的电路,电机启/停控制电路通过电机启/停控制输入端输入高/低电平控制电机两端是否加电以实现电机的启/停控制;电机正/反转控制电路通过电机正/反转控制输入端输入高/低电平控制电机接入端电压极性的改变以实现电机正/反转控制;驱动电路利用三极管对电流放大,以确保继电器常开触点可靠吸合。参见附录 10。

(1) 电机启/停和正/反转控制

按照图 5-29 所示接线,接线提示如下:

- 将端口 1 与端口 2 连接,端口 3 和 +5 V 连接。
- +5 V 由直流稳压电源提供,电源地接板子 GND2。
- 任选两路开关量输出通道输出开关量控制信号分别接“电机启/停控制输入端”和“电机

正/反转控制输入端”，开关量控制信号的地接板子 GND1。

- 电机接入端连接万用表的两个测量表笔。

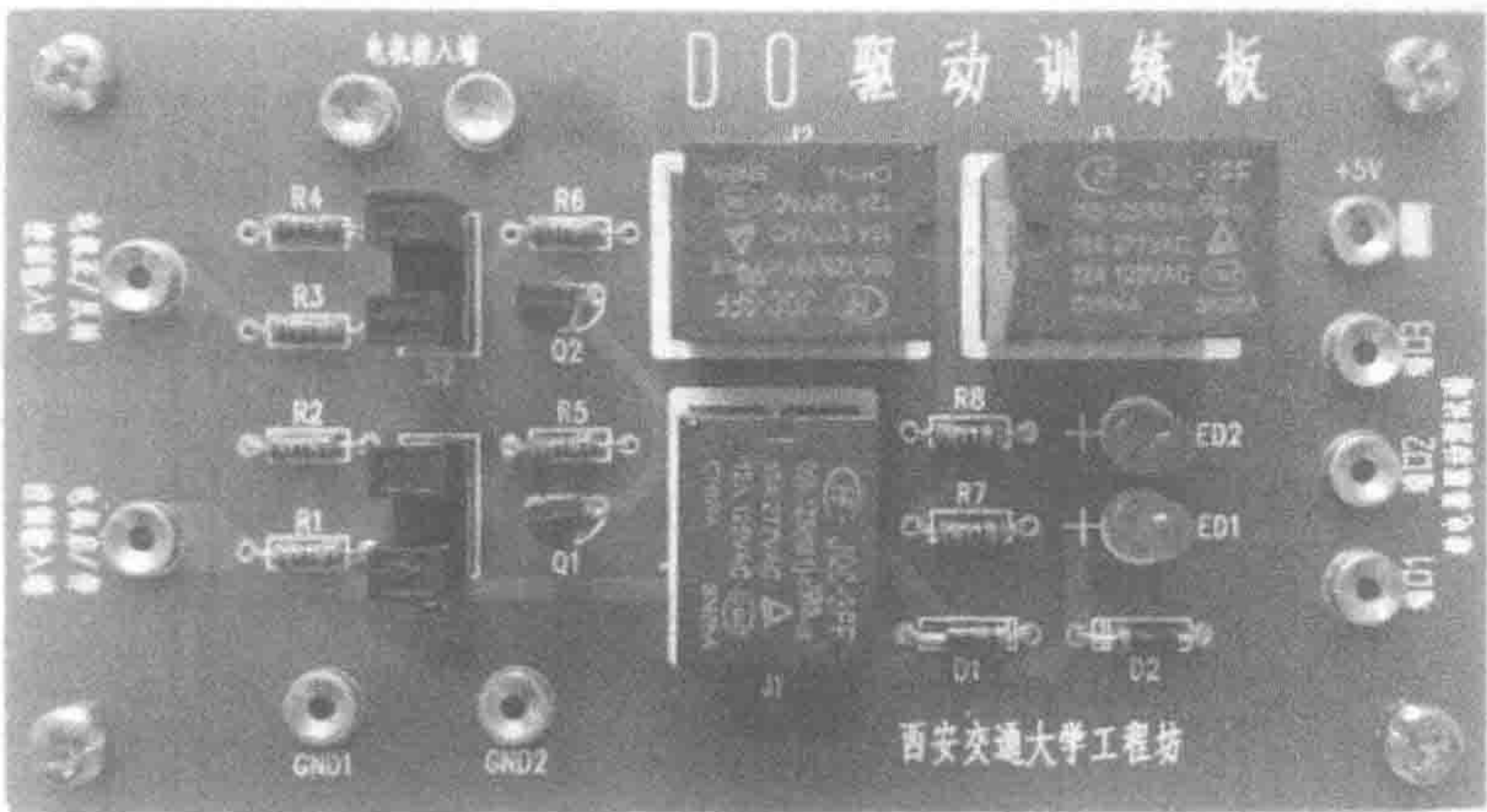


图 5-28 DO 驱动训练板

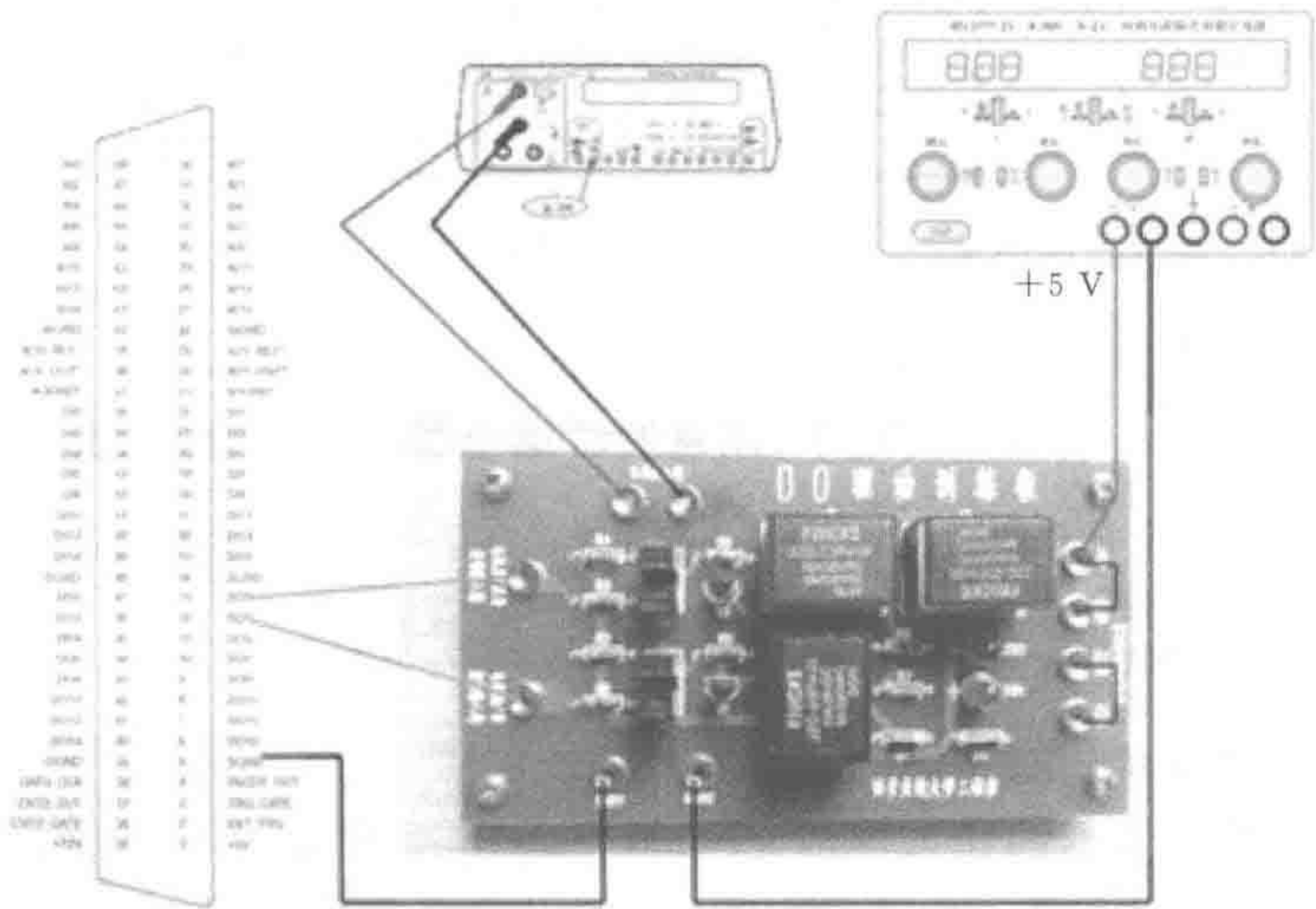


图 5-29 电机启/停与正/反转控制接线图

按照下面提示操作并观察现象：

- ①给“电机启/停控制输入端”输入低电平信号，观察 LED2 的亮暗、并仔细聆听有无继电器吸合的声音。
- ②给“电机启/停控制输入端”输入高电平信号，观察 LED2 的亮暗、并仔细聆听有无继电器吸合的声音。
- ③在“电机启/停控制输入端”输入高电平信号的情况下，先后给“电机正/反转控制输入端”输入高电平信号、低电平信号，用万用表观察电机两个接入端电压极性的变化，同时观察 LED1 的亮暗变化。

(2) “继电器直接加计算机输出的控制信号”现象观察

按照图 5-30 所示接线，接线提示如下：

- 将端口 1 和端口 2 断开，端口 3 和 +5 V 断开。

- +5 V 由直流稳压电源提供,电源地接板子 GND2。
- 在端口 2 和端口 3 两端加开关量控制信号,任选 1 路开关量输出通道输出端接端口 3,开关量控制信号的地接端口 2。

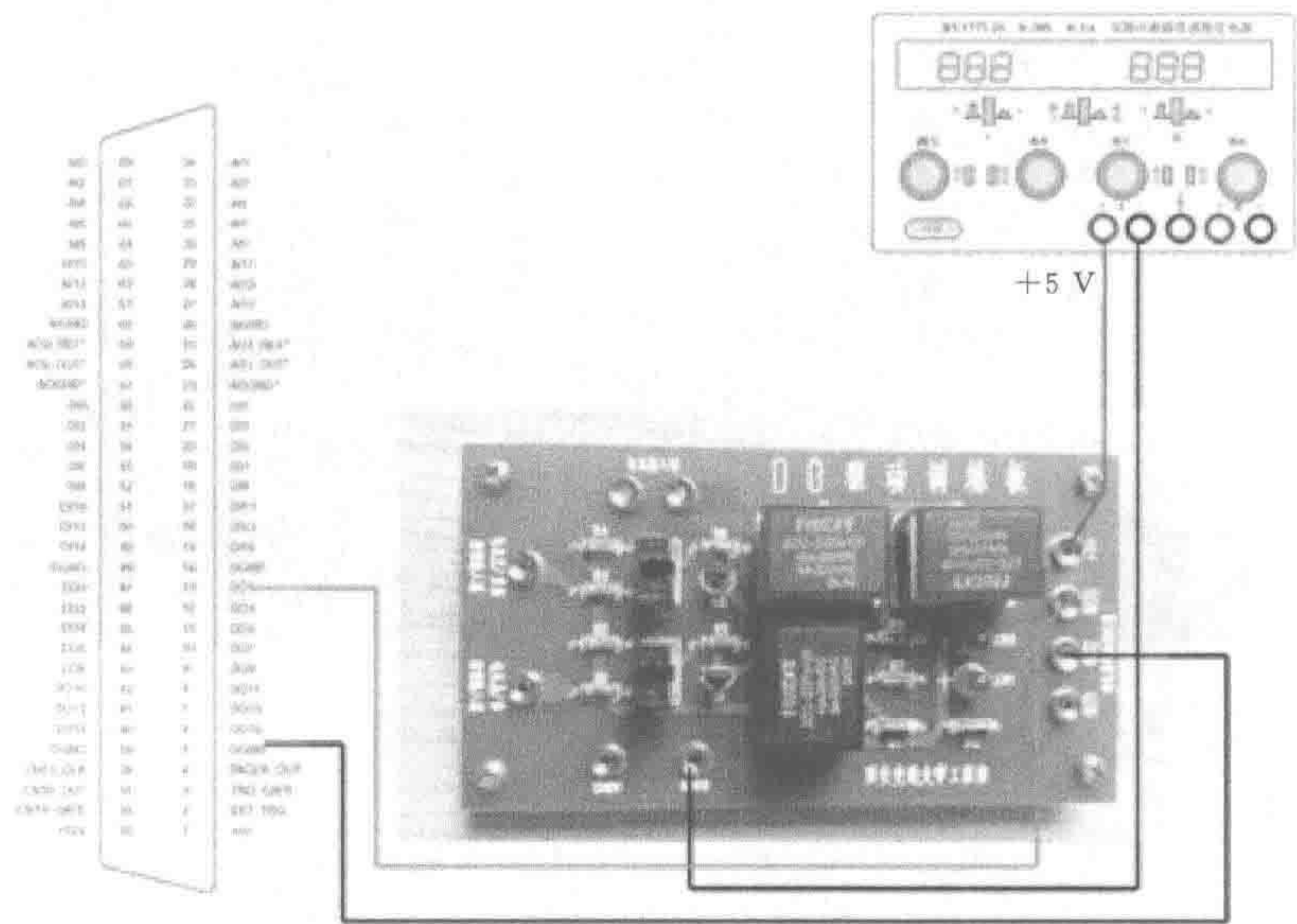


图 5-30 继电器直接接控制信号连线图

按照下面提示操作并观察现象：
开关量输出通道输出端输出高电平，注意聆听继电器有无吸合声，观察 LED2 能否点亮。

5.7 本章小结

- 过程通道是计算机控制系统的重要组成部分。
- 过程通道包括开关量的输入通道、开关量的输出通道、模拟量的输入通道、模拟量的输出通道。
- 介绍了开关量的输入通道、开关量的输出通道的功能；结合 PCI-1710 采集卡，介绍了开关量输入、开关量输出的实现方法。
- 介绍了模拟量的输入通道、模拟量的输出通道的功能；结合 PCI-1710 采集卡，介绍了模拟量输入、模拟量输出的实现方法。

附录 1 常用元器件

任何复杂电路都是基本元器件构成的,了解元器件基本特性和作用对分析电路工作是十分重要的。本附录简单介绍了电阻器、电容器、二极管的电路符号、主要作用、标识方法和主要特性参数。

F1.1 电阻器

电阻器简称为电阻,在电路中用字母 R 表示。电阻在电路中对电流有阻碍作用并且造成能量的消耗,图 F1-1 是电阻的原理图符号,图 F1-2 给出了部分常用电阻的实物图片。

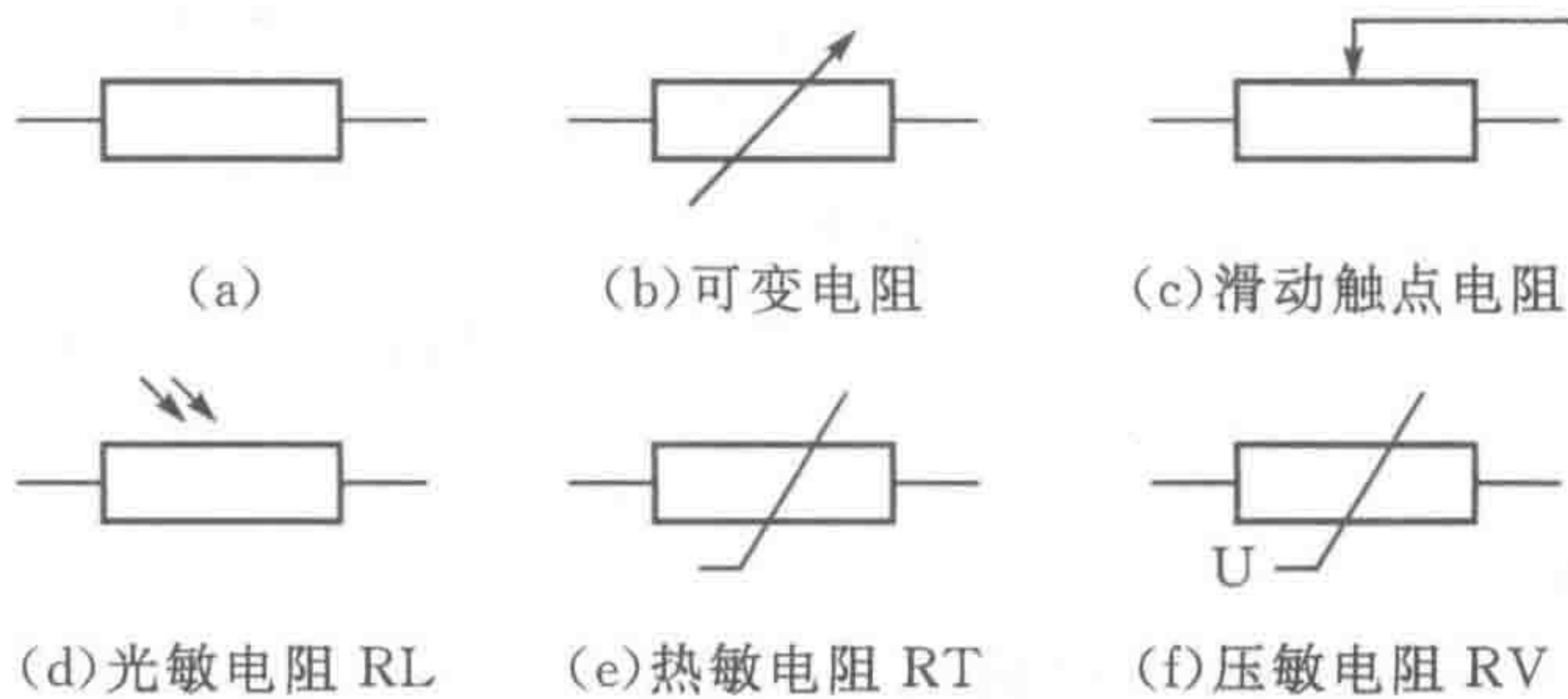


图 F1-1 电阻的原理图符号

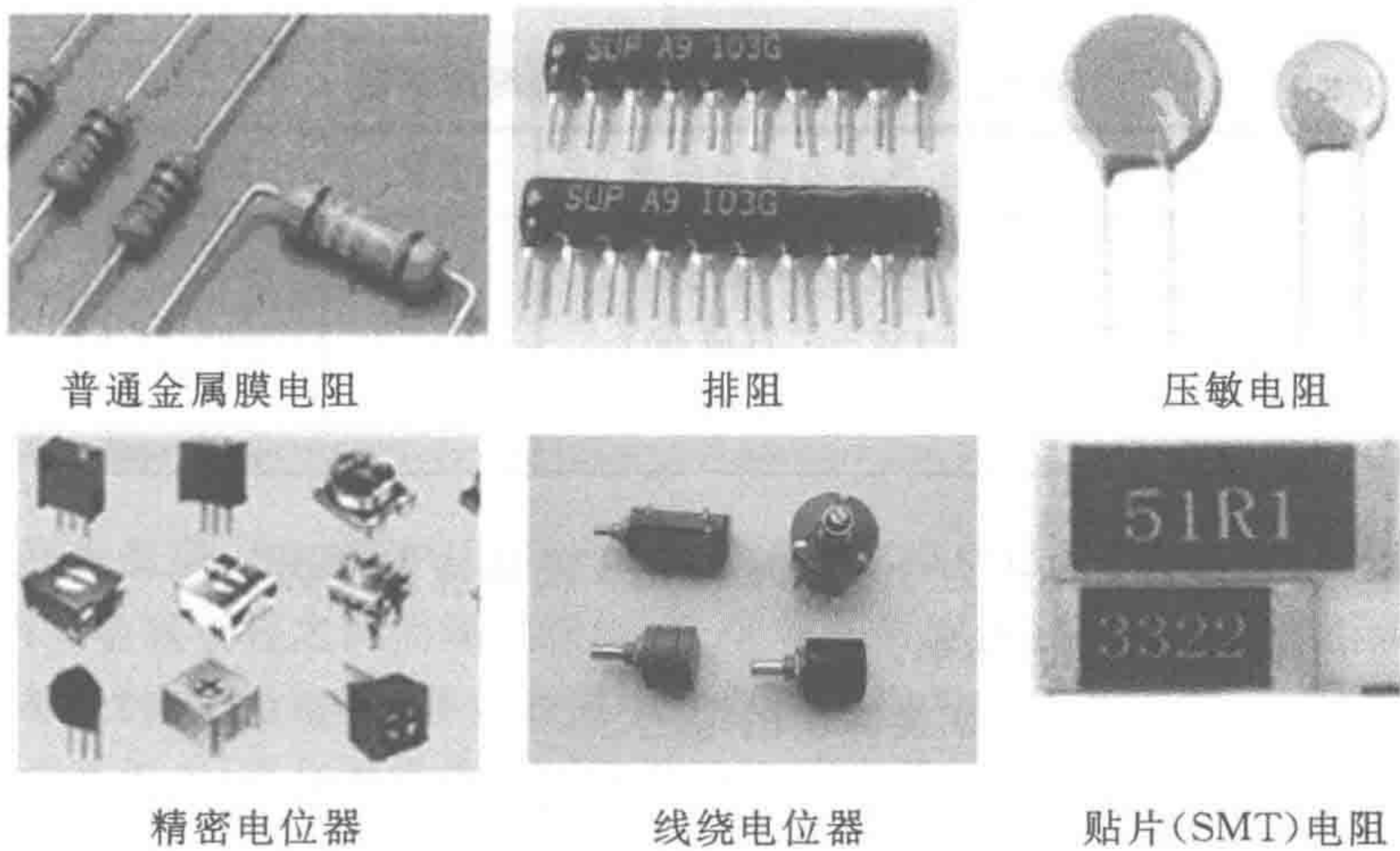


图 F1-2 常用电阻实物

1. 电阻的主要作用

①限流:为了使通过用电器的电流不超过额定值,通常在电路中串联一个电阻。这种可以

限制电流大小的电阻叫做限流电阻。

②分流：当在电路的支路上需同时接入几个额定电流不同的用电器时，可以在额定电流较小的用电器两端并联接入一个电阻，该电阻的作用是分流。

③分压：当工作电源高于用电器的额定电压时，给该用电器串联一个适当阻值的电阻，让它分担一部分电压，用电器便能在额定电压下工作。

④能量转换：电流通过电阻时，会把部分或全部电能转化为内能。如电烙铁、电炉、取暖器等电热器。

⑤阻抗匹配：信号传输过程中负载阻抗和信源内阻抗之间的特定配合关系。对电子设备互连来说，只要后一级的输入阻抗大于前一级的输出阻抗 5~10 倍以上，就认为阻抗匹配良好。

2. 电阻的标识

①直标法：将电阻器的标称值用数字和文字符号直接标在电阻本体上，其允许偏差则用百分数表示，未标偏差值的即为±20%的偏差。

②数码标识法：主要用于贴片等小体积的电阻，在四位(或三位)数码中，从左至右的前 3 位(或前 2 位)数表示有效数字，最后一位表示 0 的个数，如：582 表示 5800 Ω(即 5.8 kΩ)、3322 则表示 33200 Ω(即 33.2 kΩ)，或者用 R 表示小数点的位置(R 表示 0.)，如：51R1 表示 51.1 Ω、R22 表示 0.22 Ω。

③色环标识法：普通电阻用 4 环表示，精密电阻用 5 环表示。放置电阻时色环集中的一端放在左面，电阻的读数从左向右排列，图 F1-3 所示是用色环标示的精密电阻，其阻值为 2560 Ω±2%。

3. 电阻的主要特性参数

①标称阻值：电阻器上面所标示的阻值。

②允许误差：是标称阻值与实际阻值的差值与标称阻值之比的百分数，表示电阻器的精度，电阻的允许误差与精度等级如表 F1-1 所示。

表 F1-1 电阻的允许误差与精度等级

精度等级	I 级	II 级	III 级
允许误差	±0.5%—0.05 ±1%—0.1(或 00) ±2%—0.2(或 0)、±5%	±10%	±20%

③额定功率：在正常的大气压力 90~106.6 kPa 及环境温度为-55~+70 ℃的条件下，电阻长期工作所允许耗散的最大功率。

④额定电压：由阻值和额定功率换算出的电压。

⑤最高工作电压：允许的最大连续工作电压。在低气压工作时，最高工作电压较低。

⑥温度系数：温度每变化 1 ℃所引起的电阻值的相对变化。温度系数越小，电阻的稳定性越好。阻值随温度升高而增大的为正温度系数，反之为负温度系数。

⑦电压系数：在规定的电压范围内，电压每变化 1 V，电阻的相对变化量。

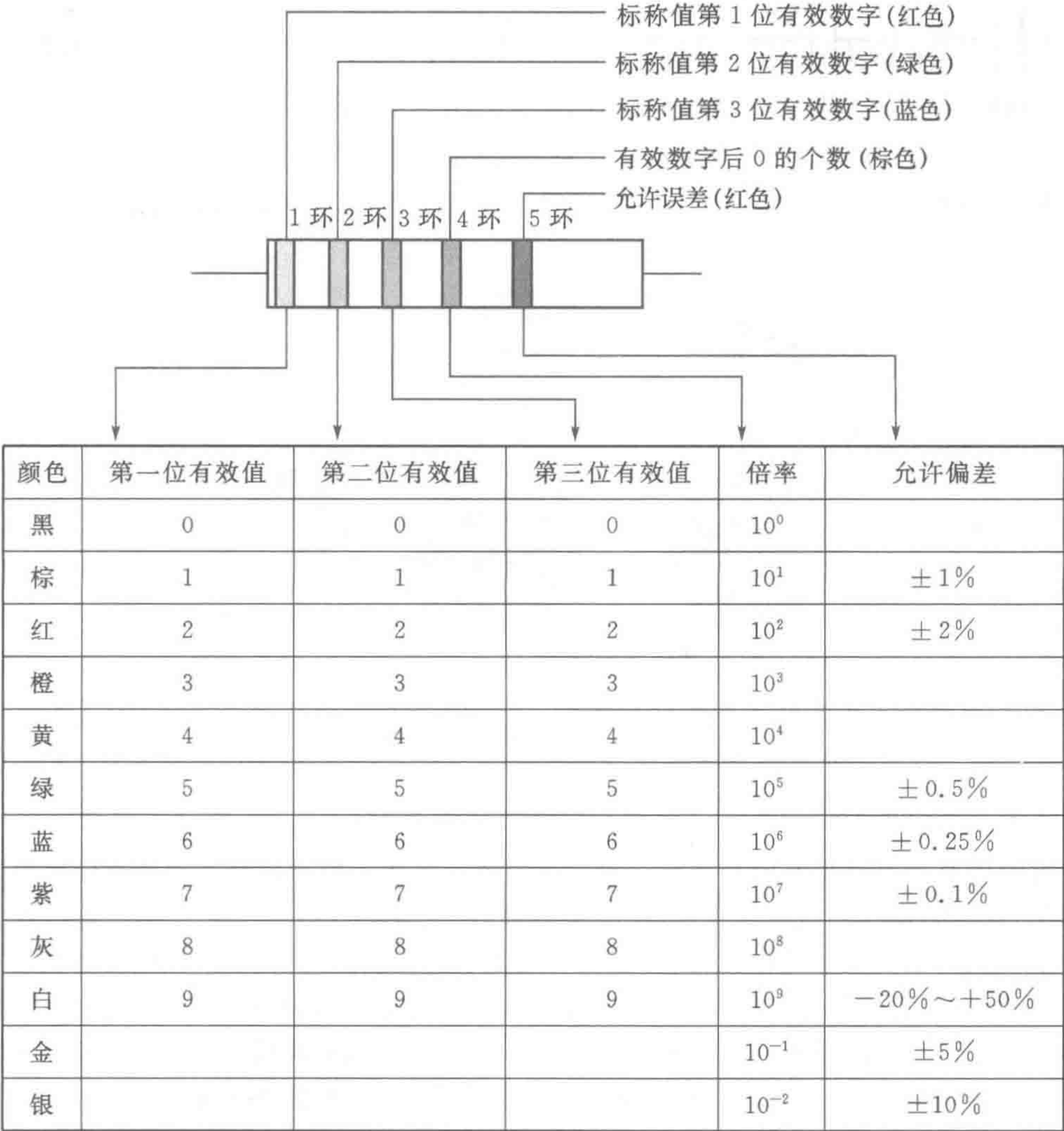


图 F1-3 精密电阻的色环表示

F1.2 电容器

电容器简称为电容,在电路中用字母 C 表示。电容虽然品种、规格各异,但就其构成原理来说,都是在两块金属极板之间间隔绝缘纸、云母等介质组成。在两块极板上加上电压,极板上就分别聚集等量的正、负电荷,并在介质中建立电场而具有电场能量。将电源移去后,电荷可以继续聚集在极板上,电场继续存在。形象地说电容器就是一种容纳电荷的器件,容量的大小反映其能够贮存电能的大小。

图 F1-4 是电容的原理图符号,其中图 F1-4(a)是基本电容符号,图 F1-4(b) 是可调电容符号,图 F1-4(c)是微调电容符号,图 F1-4(d—f)是电解电容(有极性电容)符号,弯片表示负极,空心表示正极。图 F1-5 给了出部分常用电容的实物图片。

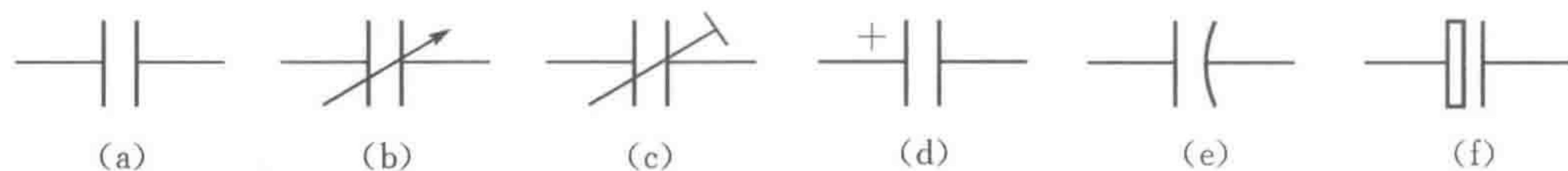


图 F1-4 电容的原理图符号

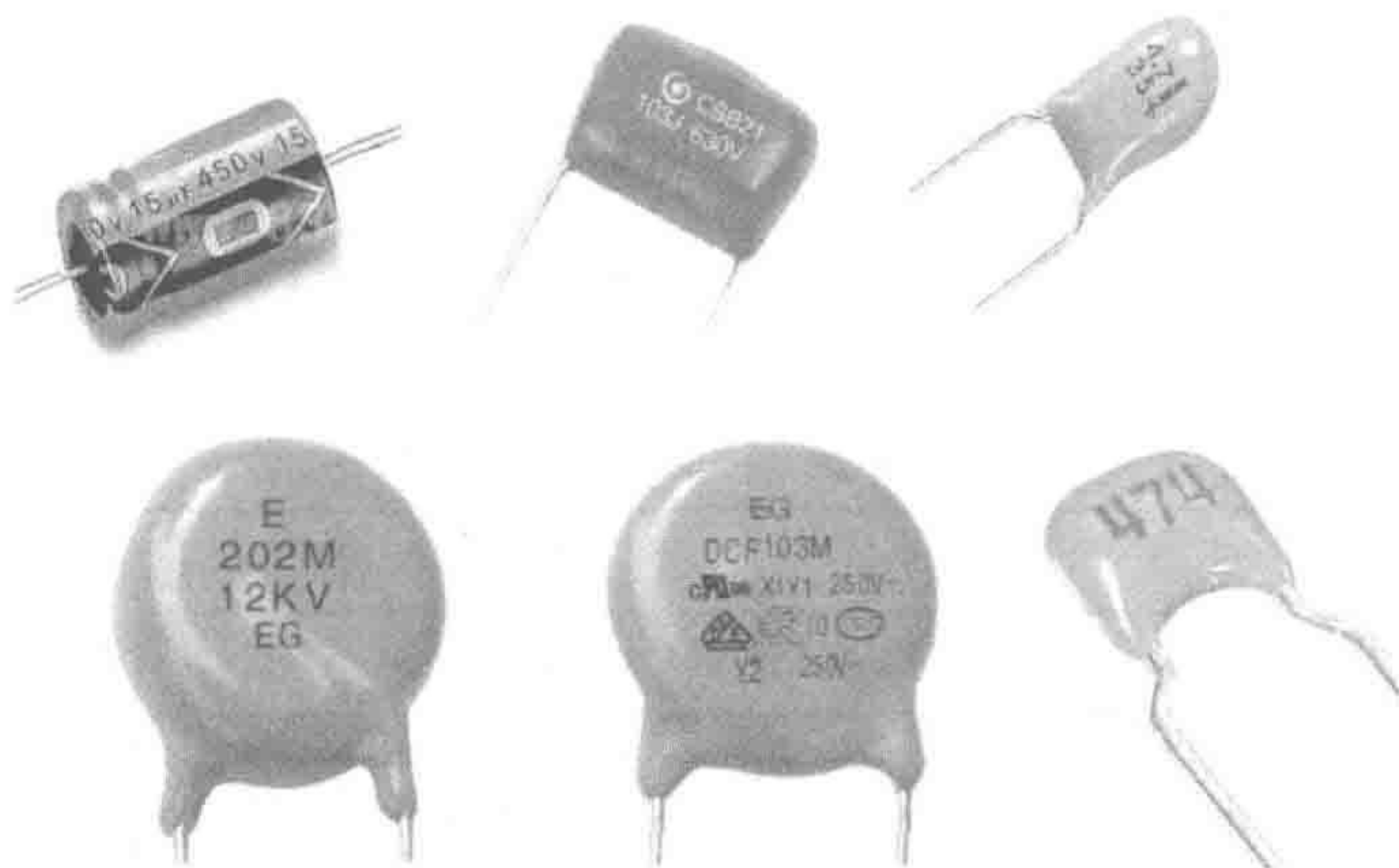


图 F1-5 部分常用电容的实物图片

1. 电容的主要作用

电容器的基本特性是充电与放电。充放电时电容极板上所带电荷对定向移动的电荷具有阻碍作用,这种阻碍作用被称为容抗,在电路中用 X_C 表示, $X_C = 1/(\omega C)$,单位是欧姆。在理想条件下,当 $\omega=0$ 时, X_C 趋向无穷大,这说明直流电将无法通过电容,所以电容器具有“隔直流,通交流”的作用;容抗 X_C 与交流信号的角频率 ω 成反比,在电子电路中,常常利用其频率特性“通高频交流信号,阻低频交流信号”;又由于电容可以存储和释放电荷,而存储和释放电荷都需要一定的时间,所以电容可以充当滤波器,使脉动信号变得更为平滑。

电容的充放电作用所延伸出来的许多电路现象,使得电容器有着诸多用途。例如,在家用电器中可以用它来产生相移,将单相交流电变成相互正交的两相交流电,从而使交流电机工作;在照相闪光灯中,用它来产生高能量的瞬间放电;在声光控照明电路中用它和电阻器一起,决定照明路灯延时时间。

2. 电容器的标识

电容单位“F”的容量非常大,所以一般都采用 μF 、 nF 、 pF 作为电容的单位。一般情况下,小于 9900 pF 用 p 级表示,大于 $0.01 \mu\text{F}$ 级(含 0.01)用 μF 表示。电容器的标识方法有直标、数字标识、数字字母标识和色标法。

①直标:将标称容量和允许偏差直接标在电容器上,直标法有标注单位和不标注单位两种。直标法中有些常把整数单位的“0”省去用 R 表示小数点,如 $\text{R}47 \mu\text{F}$ 表示 $0.47 \mu\text{F}$ 。

②数字标识:只标数字而不标单位(仅限单位是 pF 和 μF)。例如,涤纶电容或瓷介电容上标有“3”、“680”、“0.01”分别表示 3 pF 、 680 pF 、 $0.01 \mu\text{F}$ 。

③数字字母标识:将标称容量的整数部分写在单位之前,小数部分写在单位之后的标示方法。例如 $1\text{p}2$ 、 $3\text{n}3$ 分别标示 1.2 pF 、 3300 pF 。

④色环(或色点)标识:电容器的色标法与电阻相同,不再予以说明。

电容阿拉伯数字后缀的英文字母代表误差值, A:0~±5%, J:±5%, K:±10%, M:±20%, Z:−20%~+50%, 如 222 K=2200 pF±10%。

3. 电容的主要特性参数

①容量与误差:电容量即电容加上电荷后储存电荷的能力大小。电容量误差是指其实际容量与标称容量间的偏差,常用固定电容允许误差的等级见表 F1-2。

表 F1-2 常用固定电容允许误差的等级

允许误差	±2%	±5%	±10%	±20%	(+20% -30%)	(+50% -20%)	(+100%-10%)
级别	02	I	II	III	IV	V	VI

②额定工作电压:是该电容在电路中能够长期可靠地工作而不被击穿所能承受的最大直流电压。如果工作电压超过电容的额定工作电压,电容将被击穿,造成不可修复的损坏。

③温度系数:电容器电容量随温度变化的大小用温度系数(在一定温度范围内,温度每变化 1℃,电容量的相对变化值)来表示。

④绝缘电阻:电容漏电的大小用绝缘电阻来衡量。绝缘电阻越大电容器漏电越小。小电容的绝缘电阻很大,可达几百兆欧或几千兆欧。电解电容器的绝缘电阻一般较小。

⑤损耗:在电场作用下,电容在单位时间内因发热所消耗的能量叫做损耗。理想电容在电路中不应消耗能量。实际上,电容或多或少都要消耗能量。能量的消耗主要由介质损耗和金属部分的损耗组成。

⑥频率特性:电容器的频率特性通常是指电容器的电参数(如电容量、损耗角)随电场频率而变化的性质。由于介电常数在高频时比低频时小,因此在高频下工作的电容器的电容量将相应地减小,损耗将随频率的升高而增加。

4. 电容器的检测

(1)电容漏电阻的离线检测

检测之前先给电容放电,用指针式万用表的 R×1k 挡,直接测量电容引脚的漏电阻,万用表指针先向右偏转,然后缓慢地向左回转,指针停下来所指示的漏电电阻值如果只有几十千欧,说明这一电解电容漏电严重(理想情况接近无穷大)。电解电容的容量越大,万用表指针向右摆动的角度就越大(指针还应该向左回转)。

(2)电容开路或击穿的检测

断开电路,并使电容放电。用万用表 R×1 挡,测量时如果万用表指针向右偏后无回转,且指示阻值很大,说明电容器开路。如果表针向右偏转后所指示的阻值很小(接近短路),说明电容器严重漏电或已击穿。

(3)在线带电检测

如果怀疑电解电容只在通电状态下才存在击穿故障,可以给电路通电,然后用万用表直流电压档测量该电容器两端的直流电压,如果电压很低或为 0 V,则表示是该电容器已击穿。

(4)电解电容的极性判别

对于电解电容的正、负极标志不清楚的,应先判别出它的正、负极。对调万用表笔红、黑端,测量两次,以漏电大(阻值小)的一次为准,黑表笔所接引脚为负极,另一引脚为正极。

F1.3 半导体二极管

半导体二极管,在电路中用字母 D 表示。其主要特性是单向导电性,也就是在正向电压的作用下,导通电阻很小;而在反向电压作用下导通电阻极大或无穷大。硅管的导通电压为 $0.6 \sim 0.8 \text{ V}$,锗管的导通电压为 $0.2 \sim 0.3 \text{ V}$,工程分析时通常采用 0.7 V 作二极管的导通电压值。图 F1-6 是常用半导体二极管的原理图符号,图 F1-7 给了出部分常用二极管的实物图片。



图 F1-6 二极管原理图符号

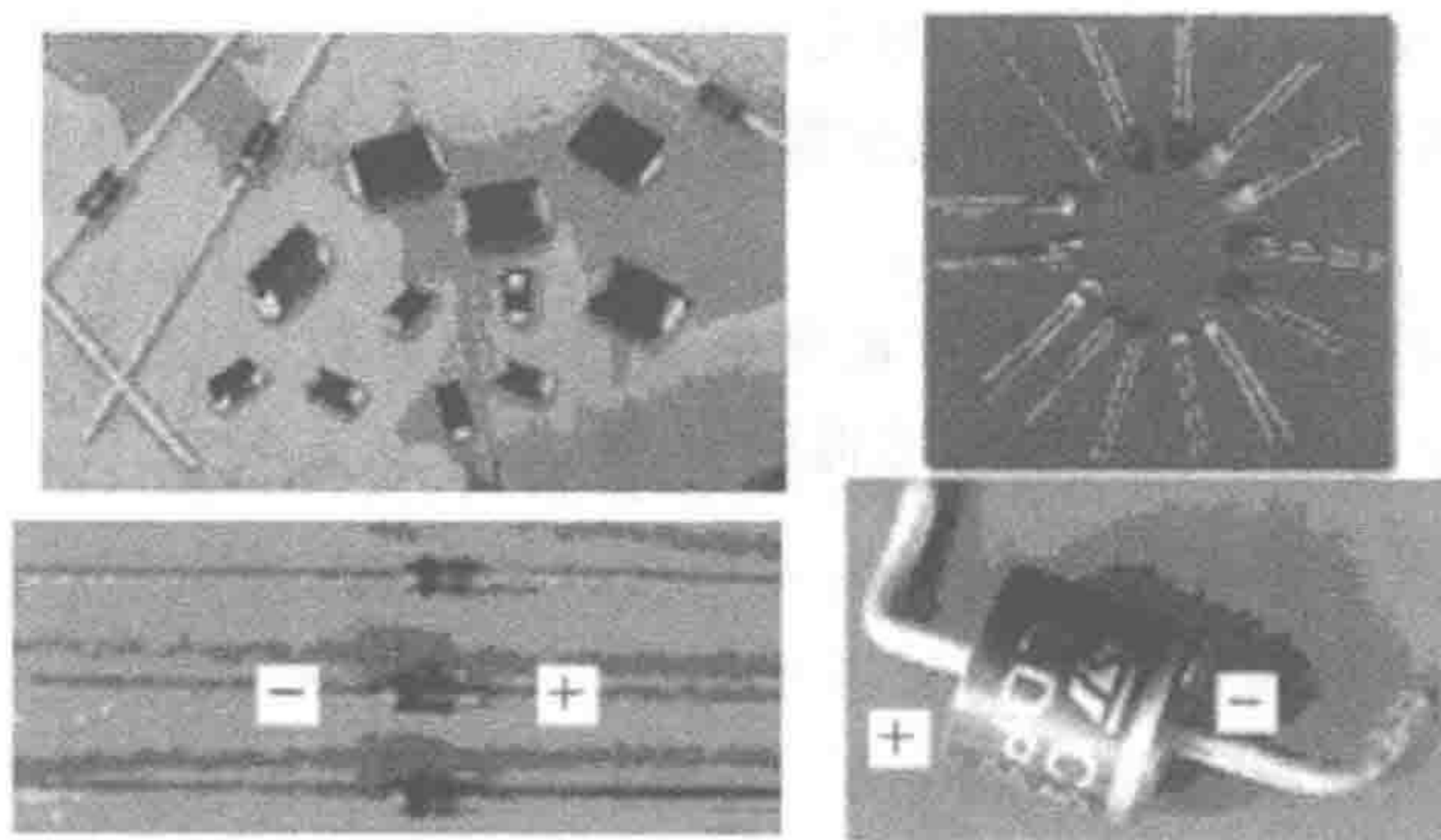


图 F1-7 部分常用二极管实物

1. 二极管的主要作用

正因为二极管具有单向导电性特性,使得它在电子电路中延伸出很多作用,无绳电话机中常把它用在整流、隔离、稳压、极性保护、编码控制、调频调制等电路中。

①开关作用:二极管在正向电压作用下导通之后的电阻很小,相当于开关接通;在反向电压作用下,处于截止状态,如同开关的断开。利用二极管的开关特性,可以组成各种逻辑电路。

②整流作用:利用二极管的单向导电性,可以把方向交替变化的交流电变换成单一方向的脉动直流电。

③限幅作用:二极管正向导通后其压降基本保持不变,利用这一特性,二极管在电路中可作为限幅元件,把信号幅度限制在一定范围内。

④稳压二极管:又称齐纳二极管,在反向击穿电压之前具有很高的阻值。在临界击穿点上,反向电阻会降低到某个很小的值,在这个低阻值区中电流增加而电压则保持恒定。利用这一特性,稳压二极管在电路中作为电压基准元件或稳压器,使用时必须串接合适的限流

电阻。

⑤发光二极管:简称为LED,具有普通二极管的特性,它可以把电能转化成光能。小功率发光二极管的反向击穿电压约为5 V,工作电流为10~30 mA,正向压降值通常在1.5~3 V之间,正向伏安特性曲线很陡,使用时必须串联限流电阻。

⑥光电二极管:也称为光电传感器,是在反向工作电压作用下工作的。无光照时,反向电流(暗电流)极其微弱;有光照时,反向电流(光电流)迅速增大到几十微安。光的变化引起光电二极管电流变化,光的强度越大,反向电流也越大。

⑦变容二极管:加反向工作电压时,变容二极管在电路中相当于可调电容。反向电压变化会引起容量的变化。反向偏压与结电容成反比,在高频调谐、通信等电路中作可变电容器使用。

2. 二极管极性的判别

①小功率二极管的负极(N极),在二极管外表用一种色圈标出来,有部分产品在二极管管脚的正极(P极)标注字母“P”,负极标注字母“N”。

②发光二极管的正、负极可从管脚长短来识别,长脚为正极,短脚为负极。

3. 二极管好坏的判别

用指针万用表 $R \times 100$ 或 $R \times 1\text{ k}$ 的电阻档,测量二极管的正,反向电阻,若两次测得阻值分别为 $1\text{ k}\Omega$ 左右和 $100\text{ k}\Omega$ 以上,说明二极管正常,二极管的正向电阻越小越好,反向电阻越大越好。若正向电阻无穷大,说明二极管内部断路,若反向电阻为零,则表示二极管已被击穿。

附录 2 C++ 控制程序设计

在电气控制系统中,控制规律或者控制策略是由控制器完成的,而在数字控制器中,控制规律通常需要通过程序完成,本书编程采用的是 C++ 语言。C++ 是一种应用广泛的程序设计语言,它在 C 语言的基础上扩展了面向对象的程序设计特点。最主要的是增加了类功能,使它成为面向对象的程序设计语言,从而提高了开发软件的效率。本附录重点介绍本书所列训练项目相关的 C++ 语言知识。

F2.1 概述

C++ 源于 C 语言。C 语言由于其简单、灵活的特点,很快就被用于编写各种不同类型的程序,从而成为世界上最流行的计算机语言之一。很多操作系统都是采用 C 语言编写的,如 UNIX 操作系统以及 UNIX 系统下运行的商业程序一般都是用 C 语言来编写。

C 语言是一种高级编程语言,相对于汇编语言更容易理解和编写,可以编制较为复杂的系统程序。但是也具有低级语言的特点,它类似汇编语言,可以直接对硬件进行操作。

虽然 C 语言是一种高级语言,但是相对于其他高级语言不太容易理解和方便使用,20 世纪 80 年代初,美国贝尔实验室设计并实现了 C 语言的扩充、改进版本,最初的成果称为“带类的 C”,1993 年正式取名为 C++。C++ 改进了 C 的不足之处,支持面向对象的程序设计,在改进的同时保持了 C 的简洁性和高效性。C 语言的大多数特征都成为 C++ 语言的一个子集,C++ 面向对象编程使得程序的模块更加独立,程序的可读性和可理解性更好,代码的结构性更加合理。

F2.2 C++ 语言基础

F2.2.1 标准 C++ 程序的基本组成

一个标准 C++ 程序由预处理命令、函数、语句、变量、输入/输出和注释几个基本部分组成。

```
#include <iostream.h>           //预处理命令
void main(void)                 //主函数
{
    int x;                      //变量定义
    cin>>x;                    //输入赋值
    cout<<"Hello c++"<<endl;  //屏幕输出
}
```

1. 预处理命令

C++程序的预处理命令以“#”开头。C++提供了三类预处理命令：宏定义命令、文件包含命令和条件编译命令。

2. 函数

C++的程序通常是由若干个函数组成，函数可以是 C++提供的库函数，也可以是由用户自己编写的自定义函数。在组成一个程序的若干个函数中，必须有一个并且只能有一个是主函数 main()。执行程序时，系统先找主函数，并且从主函数开始执行，其他函数只能通过主函数或被主函数调用的函数进行调用。

3. 语句

语句是组成程序的基本单元。在 C++语句中，表达式语句最多。表达式语句由一个表达式后面加上分号组成。语句除了有表达式语句和空语句之外，还有分支语句、循环语句和转向语句等，所有语句以分号结束。

4. 变量

多数程序都需要说明和使用变量。变量的类型很多，基本数据类型有整型、字符型和浮点型。

5. 输入和输出

C++程序中总是少不了输入和输出语句，主要用于接收用户的输入以及返回程序运行结果。

6. 注释

注释可以帮助用户读懂程序，不参与程序的运行。C++的注释为“//”之后的内容，直到换行。注释仅供阅读程序使用，是程序的可选部分。另外，C++为了与 C 语言兼容，也能识别 C 语言的注释方式，即一对符号“/*”与“*/”之间的内容。

► F2.2.2 基本数据类型

程序用于处理数据，而数据是以变量或常量的形式存储，每个变量或常量都有数据类型。一般将 C++语言的基本数据类型分成四种：整形(int)、字符型(char)、浮点型(float)和布尔型(bool)。另外为了适应情况需要，将几种类型前面加以修饰，常用修饰符有四种：有符号(signed)、无符号(unsigned)、长型(long)、短型(short)。以 32 位计算机为例，C++常用的数据类型以及取值范围如表 F2-1 所示。

表 F2-1 C++基本数据类型

数据类型	关键字	占字节数	取值范围
字符型	char	1	-128~127
无符号字符型	unsigned char	1	0~255
整型	int	4	-2147483648~2147483647
无符号整型	unsigned int	4	0~4294967294

续表 F2 - 1

数据类型	关键字	占字节数	取值范围
短整型	short int	2	-32768~32767
长整型	long int	4	-2147483648~2147483647
单精度浮点型	float	4	-3.4e38~3.4e38
双精度浮点型	double	8	-1.7e308~1.7e308
无值型	void	0	
逻辑型	bool	1	true 或 false

除了以上基本数据类型外,用户可以根据需要,按照 C++ 语法由基本数据类型构造出一些数据结构,如数组、指针、结构体、共用体、类等。

F2.2.3 运算符和表达式

在程序中,表达式是计算求值的基本单位,它是由运算符和操作数组成的式子。最简单的表达式只有一个常量或变量,当表达式中有两个或多个运算符时,表达式称为复杂表达式。

1. 算术运算符

C++ 定义了五种基本算术运算操作,即加(+)、减(-)、乘(*)、除(/)和取余(%)。C++ 中算术运算符和数学运算的概念及运算方法是一致的,其中乘(*)、除(/)和取余(%)优先于加(+)减(-)。还要注意几个问题,如对于两个整型操作数相除,结果为整数;取余运算只针对整型操作数。

2. 赋值运算符

在 C++ 语言中,赋值操作符“=”所构成的是一个赋值表达式,作用是将赋值符右边的操作数的值赋给左边的变量。例如:pi=3.14159。

3. 复合运算符

在 C++ 语言中,规定了十种复合赋值运算符,常见的算术复合赋值运算符有:+=(加赋值)、-=(减赋值)、*=(乘赋值)、/=(除赋值)、%=(求余赋值)。例如:x+=1,等同于 x=x+1。

4. 关系运算符

关系运算符是双目运算符,是对两个操作数进行比较,运算结果为布尔型,若关系成立,则值为 true,否则为 false。C++ 提供了六种关系运算符:<(小于)、<=(小于或等于)、>(大于)、>=(大于或等于)、==(等于)、!=(不等于)。例如:4<5 的结果为 false。

5. 逻辑运算符

有三种逻辑运算符:! (逻辑非)、&&(逻辑与)、||(逻辑或)。“逻辑非”的规则是,当运算分量为 false 时,结果为 true;运算分量为 true 时,结果为 false。“逻辑与”是当两个运算分量都是“真”时,结果才是“真”,否则为“假”。“逻辑或”是当两个运算分量中有一个是“真”时,结果就为“真”,而只有当它们都为“假”时,结果才为“假”。

6. 位运算符

位是计算机中数据的最小单位,也是在编程中常用的一种运算形式,一般用 0 和 1 表示。一个字符在计算机中用 8 个位表示,8 个位组成一个字节。对于操作数的位运算,通常将十进制数表示为二进制,并逐位进行运算。具体运算规则与实例如表 F2-2 所示。

表 F2-2 位运算规则与实例

位运算符	运算符名称	例子(二进制)	运算结果(二进制)
&	按位与	1010&0110	0010
	按位或	1010 0110	1110
^	按位异或	1010^0110	1100
~	按位取反	~1010	0101
<<	左移位	0110<<1	1100
>>	右移位	1010>>2	0010

按位与的运算规则:参与运算的两个操作数,如果两个对应的位均为 1,对应位与的结果为 1,否则结果为 0。

按位或的运算规则:参与运算的两个操作数,如果两个对应的位只要有一个为 1,对应位或的结果就为 1。

按位异或的运算规则:参与运算的两个操作数,如果两个对应的位不同,对应位异或的结果为 1,否则为 0。

按位取反的运算规则:对于一个二进制数据按位取反,各个位 0 变 1,1 变 0。

移位运算规则:将一个操作数向左移一位,相当于该操作数乘以 2;向右移一位,相当于除以 2。从表中的例子可以看出,对于二进制数据进行移位,移出的位舍弃,空出的位补 0。

7. 表达式

表达式是由运算符和操作数组成的式子,常量、变量、函数等都可以作为操作数,而 C++ 中运算符很丰富,所以表达式种类很多。常见表达式有算术表达式、关系表达式、逻辑表达式、条件表达式、赋值表达式和逗号表达式六种。

在表达式中使用运算符时,要明确运算符的功能,如: &、* ,既可以作为单目运算符,也可以作为双目运算符,表达的意义也不同。同时,运算符优先级决定了在表达式中各个运算符执行的先后顺序。高优先级运算符先于低优先级运算符进行运算。如表达式“a+b*c”,会先计算 b*c,得到的结果再和 a 相加。在优先级相同的情形下,则按从左到右的顺序进行运算。当表达式中出现了括号时,会改变优先级。先计算括号中表达式值,再计算整个表达式的值。

F2.2.4 C++ 的程序结构

算法的基本控制结构也成为流程控制结构,和传统的结构化程序设计一样,C++ 有三种基本结构:顺序结构、选择结构和循环结构。

1. 顺序结构

顺序结构就是按照语句出现的顺序一条一条地执行,先出现的先执行,后出现的后执行,是一种简单的语句,通常包括表达式语句、输入语句与输出语句。

表达式语句是一种简单的语句,任何表达式加上分号都可以构成表达式语句。C++的输入和输出工程通常是由函数 scanf、printf 或者输入、输出流来实现。输入、输出流是指数据传输与流动,C++在 iostream 类中定义了运算符“>>”和“<<”来实现输入与输出功能。

当程序需要键盘输入时,采用操作符“>>”从输入流 cin 中获取键盘输入的数字或者字符,并赋值给变量。具体格式如下所示。

```
#include<iostream. h>           //包含 iostream 头文件
void main(void)
{
    int x;                        //定义整型变量 x
    cin>>x;                       //通过键盘输入赋值
}
```

当需要在屏幕显示输出时,可以用操作符“<<”与 cout 结合,在屏幕上显示字符与数字。执行下所示代码将在屏幕上输出“x=5”,cout 语句中,引号里面的内容为屏幕上限制的字符,而后面的 x 将以数字“5”输出,“endl”的功能是换行。

```
#include <iostream. h>
void main(void)
{
    int x=5;
    cout<<"x="<< x<< endl;
}
```

2. 选择结构

选择结构是用来判断所给定的条件是否满足,并根据判定的结果(真或假)选择执行不同的分支语句。C++中构成选择结构的语句有条件语句(if)和开关语句(switch)

(1) 条件语句

条件语句的基本形式有三种:if 语句、if...else 语句、多重 if...else 语句。

if 语句的形式为(流程图如图 F2-1 所示)

if(<表达式>)

<语句>

if...else 语句的形式为(流程图如图 F2-2 所示):

if (<表达式>)

<语句 1>

else

<语句 2>

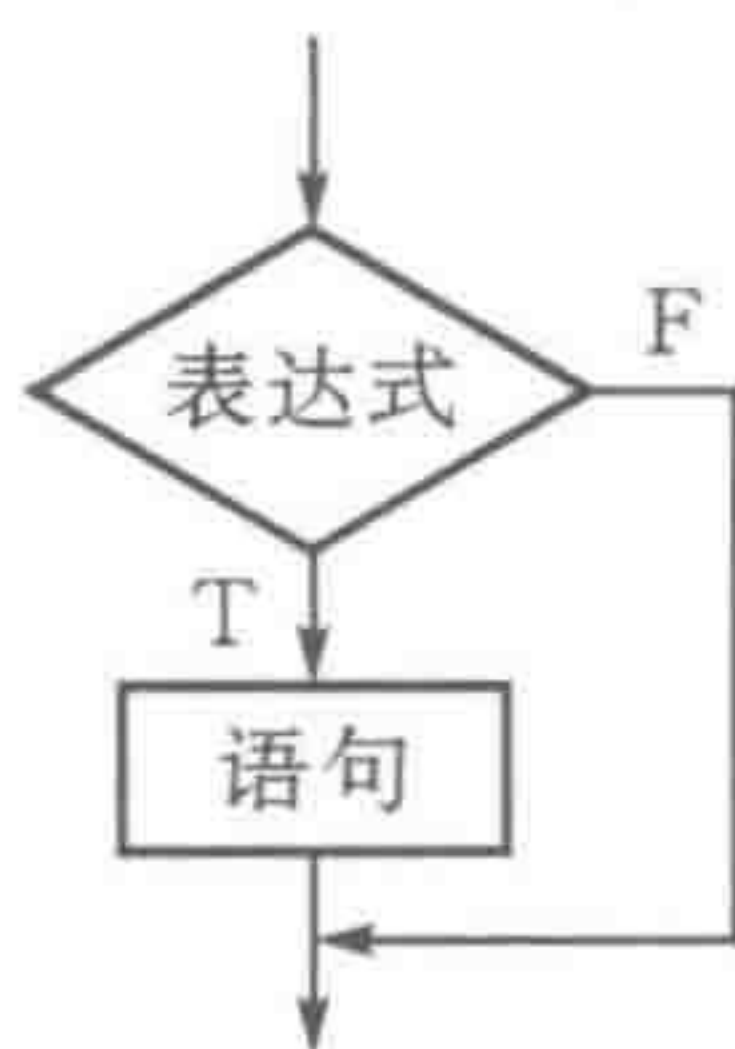


图 F2-1 if 语句流程

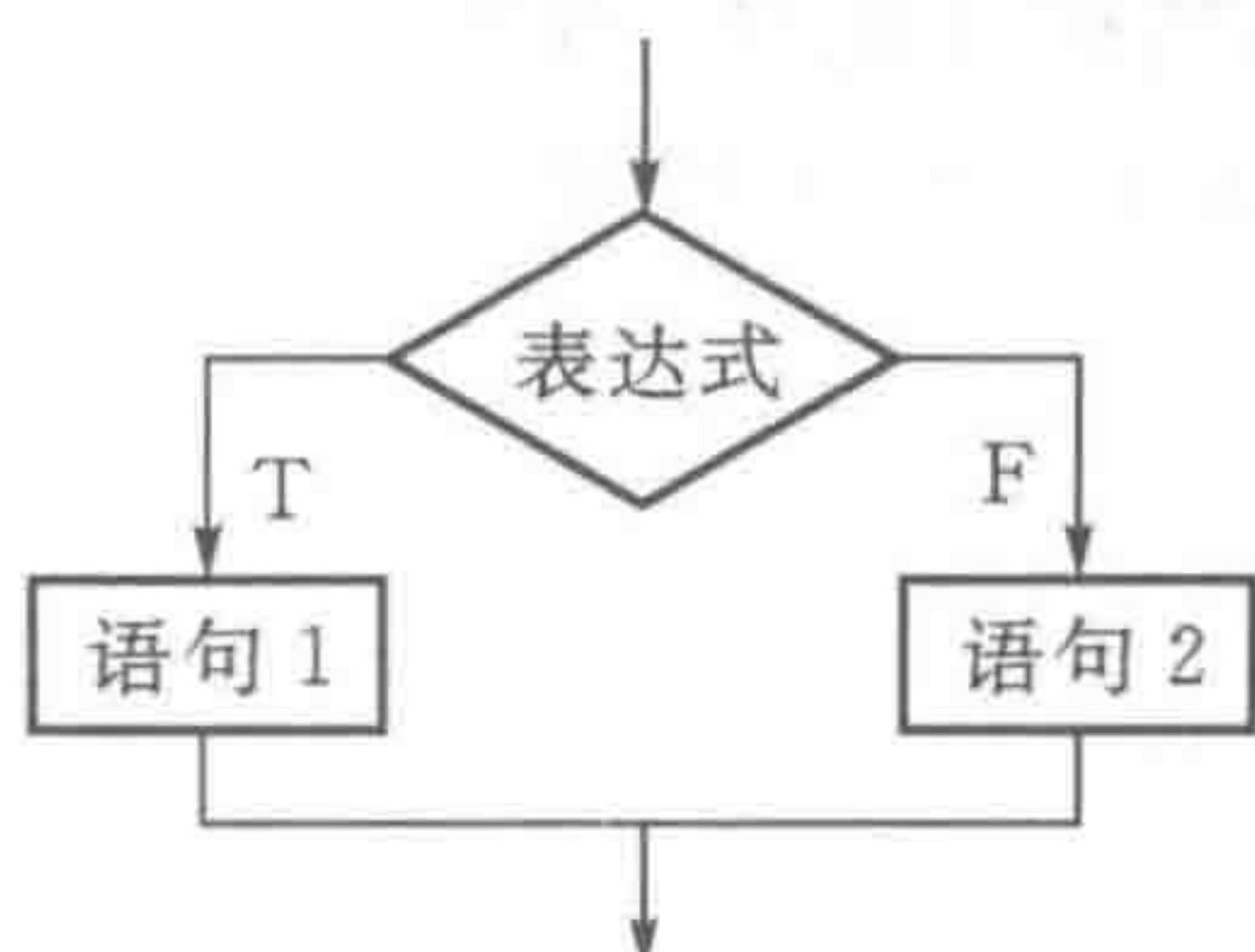


图 F2-2 if...else 语句流程

(2)switch 语句

switch 语句也叫情况语句,是一种多分支语句,语句的形式为:

```
switch(<表达式>)
{
case <常量表达式 1> :<语句 1>; break;
case <常量表达式 2> :<语句 2>; break;
.....
case <常量表达式 n> :<语句 3>; break;
default:<语句 n+1>;
}
```

当表达式的值与 case 中某个表达式的值相等时,就执行该 case 中的所有语句;通过 break 语句,跳出 switch 结构。若表达式的值与 case 中的表达式值都不匹配,则执行 default 后面的语句。

3. 循环语句

C++有三种循环语句:for 循环语句、while 循环语句和 do...while 循环语句。

(1)for 循环语句

for 循环是 C++中最常见的一种循环形式,主要功能是将某段程序代码反复执行若干次,语句形式如下:

```
for(<表达式 1>; <表达式 2>; <表达式 3> )
{<语句序列>;}
```

其中,表达式 1 是对循环控制变量进行初始化,表达式 2 是循环条件,表达式 3 是对循环控制变量进行递增或者递减操作。具体执行过程如图 F2-3 所示:首先循环变量初始化;然后判断循环条件是否成立,成立则执行循环体的语句组,不成立则结束循环;更改循环条件,重复执行,直到循环条件不成立为止。

(2)while 循环语句

while 循环语句是一种简单的循环,可以看成是 for 循环语句表达式 1 和表达式 3 为空的一种形式。具体格式如下:

```
while(条件表达式)
{
<循环体语句>;
}
```

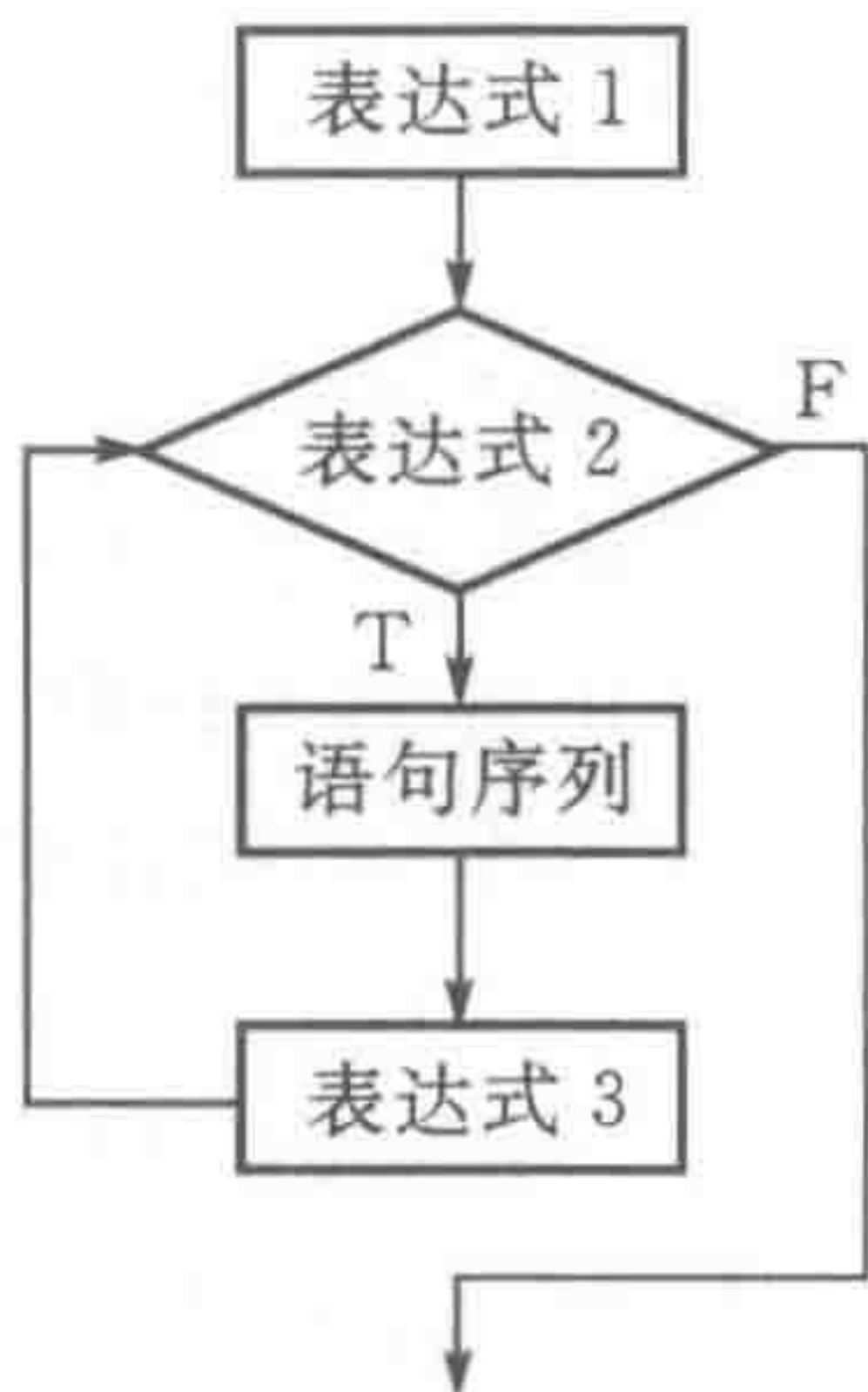


图 F2-3 for 循环流程图

while 首先判断循环条件,当条件为真时,程序重复执行循环体的语句,当条件不成立时,循环结束。流程如图 F2 - 4 所示。

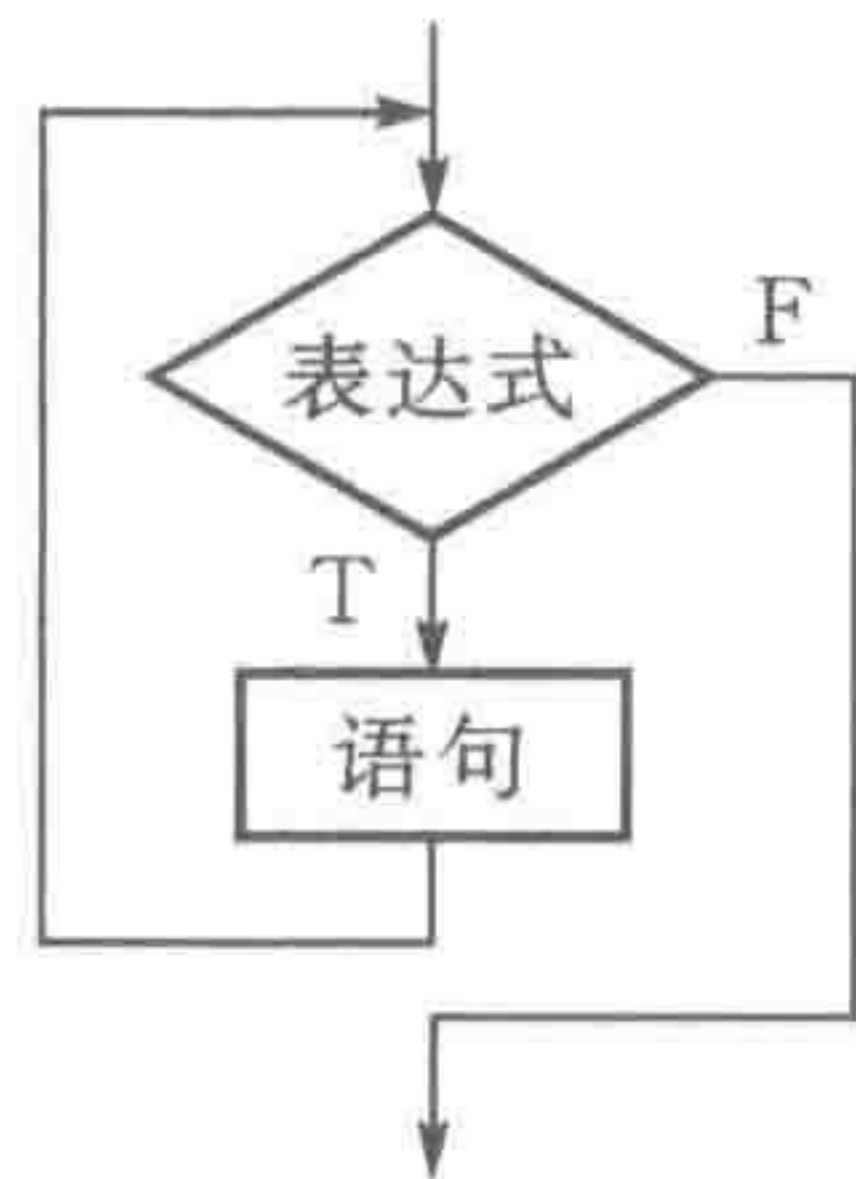


图 F2 - 4 while 语句流程图

(3) do...while 循环语句

do...while 循环语句的形式如下：

```
do
<循环体语句>;
while(<条件表达式>)
```

和 while 循环语句的区别在于:while 语句先判断表达式,然后再执行循环体语句;而 do...while先执行循环体语句,然后判断表达式。do...while 语句循环体语句至少被执行一次,而 while 循环语句中循环体语句有可能一次都不被执行。执行的流程如图 F2 - 5 所示。

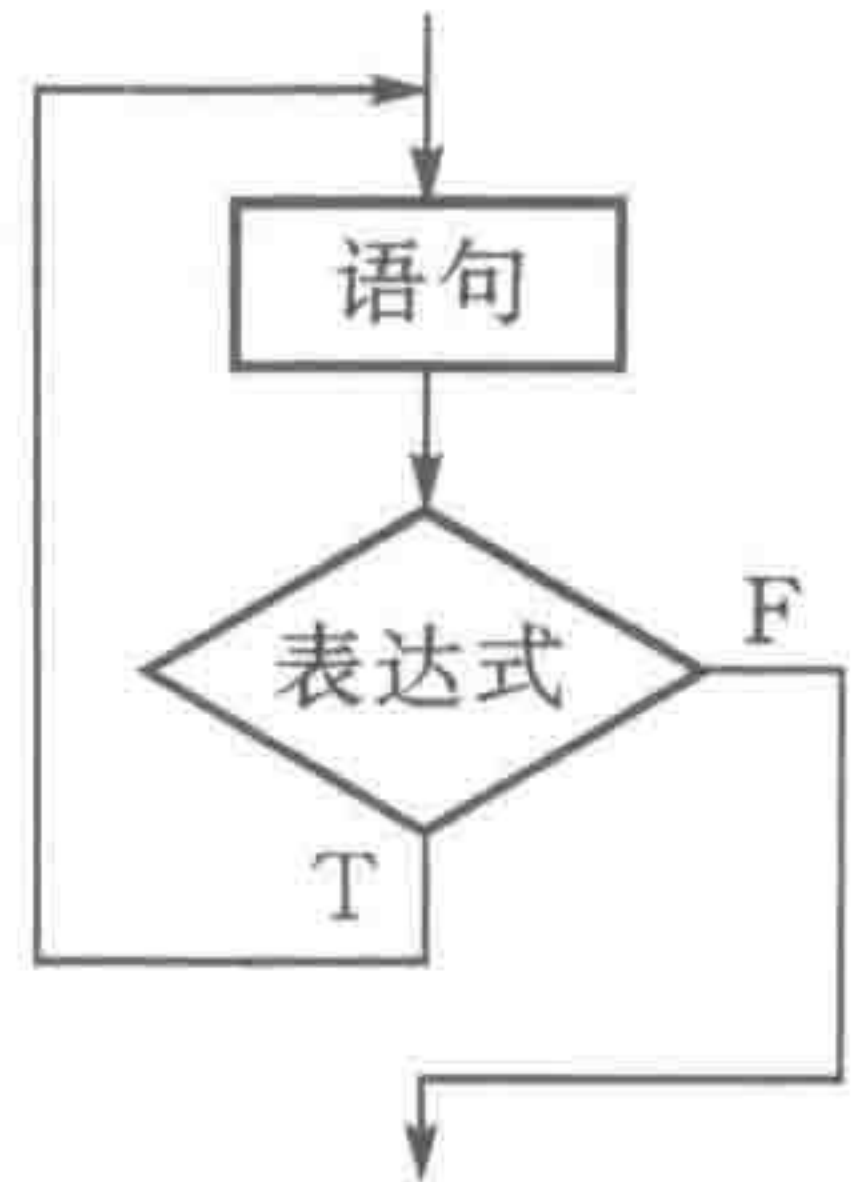


图 F2 - 5 do...while 语句流程图

4. 跳转语句

循环结构中,通常采用条件来判断循环是否继续进行,但是,在某些情况下,需要在循环的中途停止继续执行循环体的剩余语句,从循环中提前退出或者重新回到循环开始的地方进入新的一次循环,或者跳转到其他地方继续执行,这时需用到跳转语句,C++提供了 break、continue 语句。

(1) break 语句

break 语句使程序从当前的循环语句(while 和 for)内跳转出来,接着执行循环语句外面的语句;也可以用来跳出 switch 语句。以下程序可以看出 break 的用法,程序提示输入一个数字,当键盘输入的数字不为“8”时候,继续等待输入;而当输入数字“8”后,程序结束循环。

```
#include <iostream.h>
void main(void)
{
    int x;
    while(1)
    {
        cout<<"请输入一个 10 以下数字"<<endl;
        cin>>x;
        if (x==8)
            break;
    }
}
```

(2) continue 语句

continue 语句只用于循环语句,它类似于 break,但与 break 不同的是,它不是结束整个循环,而是结束本次循环,接着执行下一次循环。

```
#include <iostream.h>
void main(void)
{
    int n;
    for(n=10;n<100;n++)
    {
        if(n%3!=0) continue;
        cout<<n<<endl;
    }
}
```

例程的主要功能是输出 10~100 之间能被 3 整除的数,采用 continue 作为转移语句。而如果将 continue 替换为 break 时,将无法输出,这是因为当 $n=10$ 时,采用 break 跳出来整个 for 循环。

F2.3 函数及常用 C++ 函数

在程序设计中,函数是重要的组成部分,在 C++ 中,函数的主要任务是完成某一个独立的功能的子程序,这样一个复杂的程序就可以按照“自顶向下”的思想分解成若干功能相对独立的子模块。

使用函数的主要目的有:

- ①程序按功能划分成较小的功能模块,可以使程序更简单,调试方便。
- ②避免语句重复。将重复的语句编写成一个函数,不仅可以减少编辑程序的时间。

在调用函数时,C++ 会转移到被调用的函数中执行,执行完后再回到原先程序执行的位置,然后继续执行下一条语句。

C++的函数的定义一般由函数类型、函数名、函数参数表和函数体四个部分构成,其格式如下:

```
函数类型 函数名(参数列表)
{
    函数体;
}
```

函数名:函数名通常要符合 C++命名规范,通常用英文单词或者英文单词的缩写,并尽可能体现函数功能。同时为了与 C++编译器定义的一些函数区分开,通常避免下划线开头。

函数类型:函数类型通常是函数体中使用 return 返回的函数值的类型,具体类型同签名定义的数据类型。如果函数没有返回值,通常定义为 void。

参数列表:参数可以是多个或者没有,通常参数的说明写在函数名后面的括号中,包含函数参数说明和参数名两个部分,参数间通常用逗号隔开。参数列表的主要作用是向函数传递数值或者从函数返回数值。

函数体:花括号中的语句成为函数体,它是完成函数功能的主体。

函数定义与调用举例如下列代码,其只要功能是利用子函数对两个整型数进行求和,并将求和后的数据返回主函数。

```
#include <iostream.h>
int plus(int a,int b)
{
    return (a+b);
}
void main()
{
    int x,y,z;
    x=20;
    y=10;
    z=plus(x,y);
    cout<<"x+y="<<z<<endl;
}
```

在编程时,可以采用以上函数格式进行自定义,从而完成某一功能。在编译器中,也提供了一些库函数,在使用时,不需要进行自定义,只需要进行调用即可,下面将对 C++中常用的函数进行说明。

(1)读端口函数: _inp()

原型: int _inp(unsigned short port);

头文件: conio.h

函数说明: port 参数为指定的输入端口地址。调用后,它从 port 参数指定的端口读入并返回一个字节,输入值可以是在 0~255 范围内的任意无符号整数值。

(2)写端口函数: _outp()

原型: int _outp(unsigned short port, int byte);